

Legyen Ön is Milliomos

1.0

Generated by Doxygen 1.10.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 millionaire.controller.AddQuestionController Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 AddQuestionController()	8
4.1.3 Member Function Documentation	8
4.1.3.1 getCategories()	8
4.1.3.2 getDifficulties()	9
4.1.3.3 handleCancel()	9
4.1.3.4 handleSubmit()	9
4.1.3.5 showDialog()	10
4.2 millionaire.model.AddQuestionModel Class Reference	10
4.2.1 Detailed Description	11
4.2.2 Constructor & Destructor Documentation	11
4.2.2.1 AddQuestionModel()	11
4.2.3 Member Function Documentation	11
4.2.3.1 addQuestion()	11
4.2.3.2 getCategories()	12
4.2.3.3 getDifficulties()	12
4.2.3.4 validateInputs()	13
4.3 millionaire.view.AddQuestionView Class Reference	13
4.3.1 Detailed Description	15
4.3.2 Constructor & Destructor Documentation	16
4.3.2.1 AddQuestionView()	16
4.3.3 Member Function Documentation	16
4.3.3.1 clearFields()	16
4.3.3.2 closeWindow()	16
4.3.3.3 showError()	16
4.3.3.4 showSuccess()	17
4.4 millionaire.controller.GameController Class Reference	17
4.4.1 Detailed Description	18
4.4.2 Constructor & Destructor Documentation	18
4.4.2.1 GameController()	18
4.4.3 Member Function Documentation	19

4.4.3.1	getCurrentLevel()	19
4.4.3.2	getCurrentQuestion()	19
4.4.3.3	getGameView()	19
4.4.3.4	getPlayer()	19
4.4.3.5	handleAnswer()	19
4.4.3.6	setAnswerButtonsEnabled()	20
4.4.3.7	updateGameTime()	20
4.5	millionaire.model.GameModel Class Reference	20
4.5.1	Detailed Description	22
4.5.2	Constructor & Destructor Documentation	22
4.5.3	Member Function Documentation	22
4.5.3.1	getCurrentPrize()	22
4.5.3.2	getCurrentQuestion()	23
4.5.3.3	getLastSafePrize()	23
4.5.3.4	getPlayer()	23
4.5.3.5	getPrizeLevels()	23
4.5.3.6	isGameOver()	23
4.5.3.7	isTimeUp()	23
4.5.3.8	processAnswer()	24
4.5.3.9	selectNextQuestion()	24
4.5.3.10	updatePlayTime()	24
4.6	millionaire.model.GameState Enum Reference	25
4.6.1	Detailed Description	25
4.6.2	Member Function Documentation	26
4.6.2.1	isGameEnding()	26
4.6.3	Member Data Documentation	26
4.6.3.1	CORRECT	26
4.6.3.2	LOST	26
4.6.3.3	WON	26
4.7	millionaire.view.GameView Class Reference	27
4.7.1	Detailed Description	29
4.7.2	Constructor & Destructor Documentation	29
4.7.2.1	GameView()	29
4.7.3	Member Function Documentation	30
4.7.3.1	closeGame()	30
4.7.3.2	getAnswerButtons()	30
4.7.3.3	setAnswerButtonsEnabled()	30
4.7.3.4	showAnswerResult()	30
4.7.3.5	showGameEndDialog()	31
4.7.3.6	stopTimer()	31
4.7.3.7	updateFinalPlaytime()	31

4.7.3.8 updatePrizeLadder()	31
4.7.3.9 updateQuestion()	32
4.8 millionaire.controller.LifelineController Class Reference	32
4.8.1 Detailed Description	33
4.8.2 Constructor & Destructor Documentation	34
4.8.2.1 LifelineController()	34
4.8.3 Member Function Documentation	34
4.8.3.1 getLifelinePanel()	34
4.8.3.2 setGameContext()	34
4.8.3.3 useAudienceHelp()	35
4.8.3.4 useFiftyFifty()	35
4.9 millionaire.model.LifelineModel Class Reference	35
4.9.1 Detailed Description	36
4.9.2 Member Function Documentation	36
4.9.2.1 getAudienceResult()	36
4.9.2.2 getFiftyFiftyResult()	37
4.9.2.3 isAudienceHelpAvailable()	37
4.9.2.4 isFiftyFiftyAvailable()	37
4.10 millionaire.view.LifelineView Class Reference	38
4.10.1 Detailed Description	38
4.10.2 Constructor & Destructor Documentation	39
4.10.2.1 LifelineView()	39
4.10.3 Member Function Documentation	39
4.10.3.1 createLifelinePanel()	39
4.10.3.2 showAudienceResults()	39
4.10.3.3 updateAnswerButtons()	40
4.10.3.4 updateButtonStates()	40
4.11 millionaire.Main Class Reference	40
4.11.1 Detailed Description	41
4.11.2 Member Function Documentation	41
4.11.2.1 isDeveloperMode()	41
4.11.2.2 main()	42
4.12 millionaire.controller.MainMenuController Class Reference	42
4.12.1 Detailed Description	43
4.12.2 Constructor & Destructor Documentation	43
4.12.2.1 MainMenuController()	43
4.12.3 Member Function Documentation	43
4.12.3.1 exitGame()	43
4.12.3.2 showAddQuestionDialog()	44
4.12.3.3 showMenu()	44
4.12.3.4 showRules()	44
4.12.3.5 showScoreboard()	44

4.12.3.6 startNewGame()	44
4.13 millionaire.model.MainMenuModel Class Reference	45
4.13.1 Detailed Description	45
4.13.2 Constructor & Destructor Documentation	45
4.13.2.1 MainMenuModel()	45
4.13.3 Member Function Documentation	46
4.13.3.1 getQuestionRepository()	46
4.14 millionaire.view.MainMenuView Class Reference	46
4.14.1 Detailed Description	47
4.14.2 Constructor & Destructor Documentation	47
4.14.2.1 MainMenuView()	47
4.14.3 Member Function Documentation	48
4.14.3.1 hideWindow()	48
4.14.3.2 showWindow()	48
4.15 millionaire.controller.PlayerController Class Reference	48
4.15.1 Detailed Description	49
4.15.2 Constructor & Destructor Documentation	49
4.15.2.1 PlayerController()	49
4.15.3 Member Function Documentation	49
4.15.3.1 cancelPlayerCreation()	49
4.15.3.2 createPlayer()	50
4.15.3.3 showPlayerCreationDialog()	50
4.16 millionaire.model.PlayerModel Class Reference	51
4.16.1 Detailed Description	53
4.16.2 Constructor & Destructor Documentation	53
4.16.2.1 PlayerModel() [1/2]	53
4.16.2.2 PlayerModel() [2/2]	54
4.16.3 Member Function Documentation	54
4.16.3.1 compareTo()	54
4.16.3.2 equals()	54
4.16.3.3 getCurrentLevel()	55
4.16.3.4 getFormattedPlaytime()	55
4.16.3.5 getFormattedPrize()	55
4.16.3.6 getName()	55
4.16.3.7 getPlaytime()	56
4.16.3.8 getPrize()	56
4.16.3.9 getStartTime()	56
4.16.3.10 incrementLevel()	56
4.16.3.11 setCurrentLevel()	56
4.16.3.12 setName()	57
4.16.3.13 setPlaytime()	57
4.16.3.14 setPrize()	57

4.16.3.15 updatePlaytime()	58
4.17 millionaire.view.PlayerView Class Reference	58
4.17.1 Detailed Description	59
4.17.2 Constructor & Destructor Documentation	60
4.17.2.1 PlayerView()	60
4.17.3 Member Function Documentation	60
4.17.3.1 closeDialog()	60
4.17.3.2 getPlayerName()	60
4.17.3.3 showError()	60
4.18 millionaire.model.PrizeLevel Class Reference	61
4.18.1 Detailed Description	62
4.18.2 Constructor & Destructor Documentation	62
4.18.2.1 PrizeLevel()	62
4.18.3 Member Function Documentation	62
4.18.3.1 getLevel()	62
4.18.3.2 getPrize()	63
4.18.3.3 isSafeSpot()	63
4.19 millionaire.model.Question Class Reference	63
4.19.1 Detailed Description	64
4.19.2 Member Function Documentation	64
4.19.2.1 getCategory()	64
4.19.2.2 getCorrectAnswer()	65
4.19.2.3 getDifficulty()	65
4.19.2.4 getOptionA()	65
4.19.2.5 getOptionB()	65
4.19.2.6 getOptionC()	66
4.19.2.7 getOptionD()	66
4.19.2.8 getQuestion()	66
4.20 millionaire.model.QuestionRepository Class Reference	67
4.20.1 Detailed Description	68
4.20.2 Constructor & Destructor Documentation	68
4.20.2.1 QuestionRepository()	68
4.20.3 Member Function Documentation	68
4.20.3.1 addQuestion()	68
4.20.3.2 getAllQuestions()	68
4.20.3.3 getQuestionsByCategory()	69
4.20.3.4 getQuestionsByDifficulty()	69
4.20.4 Member Data Documentation	69
4.20.4.1 CATEGORIES	69
4.20.4.2 DIFFICULTIES	70
4.21 millionaire.controller.RulesController Class Reference	70
4.21.1 Detailed Description	71

4.21.2 Constructor & Destructor Documentation	71
4.21.2.1 RulesController()	71
4.21.3 Member Function Documentation	71
4.21.3.1 handleCloseRequest()	71
4.21.3.2 showRules()	71
4.22 millionaire.model.RulesModel Class Reference	72
4.22.1 Detailed Description	72
4.22.2 Constructor & Destructor Documentation	72
4.22.2.1 RulesModel()	72
4.22.3 Member Function Documentation	73
4.22.3.1 getRulesContent()	73
4.22.3.2 isContentLoaded()	73
4.23 millionaire.view.RulesView Class Reference	73
4.23.1 Detailed Description	75
4.23.2 Constructor & Destructor Documentation	75
4.23.2.1 RulesView()	75
4.23.3 Member Function Documentation	75
4.23.3.1 closeWindow()	75
4.23.3.2 displayRules()	76
4.23.3.3 showError()	76
4.24 millionaire.controller.ScoreboardController Class Reference	76
4.24.1 Detailed Description	77
4.24.2 Constructor & Destructor Documentation	77
4.24.2.1 ScoreboardController()	77
4.24.3 Member Function Documentation	77
4.24.3.1 addScore()	77
4.24.3.2 handleClose()	78
4.24.3.3 showScoreboard()	78
4.25 millionaire.model.ScoreboardModel Class Reference	78
4.25.1 Detailed Description	79
4.25.2 Constructor & Destructor Documentation	79
4.25.2.1 ScoreboardModel()	79
4.25.3 Member Function Documentation	79
4.25.3.1 addScore()	79
4.25.3.2 getPlayers()	80
4.26 millionaire.view.ScoreboardView Class Reference	80
4.26.1 Detailed Description	81
4.26.2 Constructor & Destructor Documentation	82
4.26.2.1 ScoreboardView()	82
4.26.3 Member Function Documentation	83
4.26.3.1 closeWindow()	83
4.26.3.2 updateScores()	83

4.27 millionaire.controller.TimerController Class Reference	83
4.27.1 Detailed Description	84
4.27.2 Constructor & Destructor Documentation	84
4.27.2.1 TimerController()	84
4.27.3 Member Function Documentation	85
4.27.3.1 getElapsedTime()	85
4.27.3.2 getView()	85
4.27.3.3 startTimer()	85
4.27.3.4 stopTimer()	85
4.28 millionaire.model.TimerModel Class Reference	86
4.28.1 Detailed Description	86
4.28.2 Constructor & Destructor Documentation	87
4.28.2.1 TimerModel()	87
4.28.3 Member Function Documentation	87
4.28.3.1 getElapsedTime()	87
4.28.3.2 getStartTime()	87
4.28.3.3 start()	87
4.28.3.4 stop()	87
4.28.3.5 updateElapsedTime()	88
4.29 millionaire.view.TimerView Class Reference	88
4.29.1 Detailed Description	89
4.29.2 Constructor & Destructor Documentation	89
4.29.2.1 TimerView()	89
4.29.3 Member Function Documentation	90
4.29.3.1 updateDisplay()	90
5 File Documentation	91
5.1 src/main/java/millionaire/controller/AddQuestionController.java File Reference	91
5.2 AddQuestionController.java	91
5.3 src/main/java/millionaire/controller/GameController.java File Reference	92
5.4 GameController.java	92
5.5 src/main/java/millionaire/controller/LifelineController.java File Reference	94
5.6 LifelineController.java	94
5.7 src/main/java/millionaire/controller/MainMenuController.java File Reference	95
5.8 MainMenuController.java	95
5.9 src/main/java/millionaire/controller/PlayerController.java File Reference	96
5.10 PlayerController.java	96
5.11 src/main/java/millionaire/controller/RulesController.java File Reference	97
5.12 RulesController.java	97
5.13 src/main/java/millionaire/controller/ScoreboardController.java File Reference	97
5.14 ScoreboardController.java	98
5.15 src/main/java/millionaire/controller/TimerController.java File Reference	98

5.16 TimerController.java	98
5.17 src/main/java/millionaire/Main.java File Reference	99
5.18 Main.java	99
5.19 src/main/java/millionaire/model/AddQuestionModel.java File Reference	99
5.20 AddQuestionModel.java	100
5.21 src/main/java/millionaire/model/GameModel.java File Reference	100
5.22 GameModel.java	100
5.23 src/main/java/millionaire/model/GameState.java File Reference	102
5.24 GameState.java	102
5.25 src/main/java/millionaire/model/LifelineModel.java File Reference	102
5.26 LifelineModel.java	103
5.27 src/main/java/millionaire/model/MainMenuModel.java File Reference	104
5.28 MainMenuModel.java	104
5.29 src/main/java/millionaire/model/PlayerModel.java File Reference	104
5.30 PlayerModel.java	104
5.31 src/main/java/millionaire/model/PrizeLevel.java File Reference	106
5.32 PrizeLevel.java	106
5.33 src/main/java/millionaire/model/Question.java File Reference	106
5.34 Question.java	106
5.35 src/main/java/millionaire/model/QuestionRepository.java File Reference	108
5.36 QuestionRepository.java	108
5.37 src/main/java/millionaire/model/RulesModel.java File Reference	110
5.38 RulesModel.java	110
5.39 src/main/java/millionaire/model/ScoreboardModel.java File Reference	110
5.40 ScoreboardModel.java	110
5.41 src/main/java/millionaire/model/TimerModel.java File Reference	112
5.42 TimerModel.java	112
5.43 src/main/java/millionaire/view/AddQuestionView.java File Reference	113
5.44 AddQuestionView.java	113
5.45 src/main/java/millionaire/view/GameView.java File Reference	115
5.46 GameView.java	116
5.47 src/main/java/millionaire/view/LifelineView.java File Reference	119
5.48 LifelineView.java	119
5.49 src/main/java/millionaire/view/MainMenuView.java File Reference	121
5.50 MainMenuView.java	121
5.51 src/main/java/millionaire/view/PlayerView.java File Reference	123
5.52 PlayerView.java	123
5.53 src/main/java/millionaire/view/RulesView.java File Reference	124
5.54 RulesView.java	124
5.55 src/main/java/millionaire/view/ScoreboardView.java File Reference	126
5.56 ScoreboardView.java	126
5.57 src/main/java/millionaire/view/TimerView.java File Reference	128

5.58 TimerView.java	128
Index	129

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

millionaire.controller.AddQuestionController	7
millionaire.model.AddQuestionModel	10
Comparable	
millionaire.model.PlayerModel	51
millionaire.controller.GameController	17
millionaire.model.GameModel	20
millionaire.model.GameState	25
JDialog	
millionaire.view.PlayerView	58
millionaire.controller.LifelineController	32
millionaire.model.LifelineModel	35
millionaire.view.LifelineView	38
millionaire.Main	40
millionaire.controller.MainMenuController	42
millionaire.model.MainMenuModel	45
millionaire.controller.PlayerController	48
millionaire.model.PrizeLevel	61
millionaire.model.Question	63
millionaire.model.QuestionRepository	67
millionaire.controller.RulesController	70
millionaire.model.RulesModel	72
millionaire.controller.ScoreboardController	76
millionaire.model.ScoreboardModel	78
millionaire.controller.TimerController	83
millionaire.model.TimerModel	86
JFrame	
millionaire.view.AddQuestionView	13
millionaire.view.GameView	27
millionaire.view.MainMenuView	46
millionaire.view.RulesView	73
millionaire.view.ScoreboardView	80
JPanel	
millionaire.view.TimerView	88

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

millionaire.controller.AddQuestionController	
Új kérdések hozzáadását vezérlő osztály	7
millionaire.model.AddQuestionModel	
Új kérdések hozzáadását kezelő modell osztály	10
millionaire.view.AddQuestionView	
Új kérdések hozzáadására szolgáló grafikus felület	13
millionaire.controller.GameController	
A játék fő vezérlő osztálya	17
millionaire.model.GameModel	
A játék logikáját és állapotát kezelő osztály	20
millionaire.model.GameState	
Enum osztály, amely a játék állapotait reprezentálja	25
millionaire.view.GameView	
A játék fő grafikus felülete	27
millionaire.controller.LifelineController	
A játék segítségét kezelő vezérlő osztály	32
millionaire.model.LifelineModel	
Segítségek (lifeline) logikáját kezelő osztály	35
millionaire.view.LifelineView	
A játék segítségét megjelenítő és kezelő felület	38
millionaire.Main	
A "Legyen Ön is Milliomos!" kvízzjáték főprogramja	40
millionaire.controller.MainMenuController	
A főmenü vezérlő osztálya	42
millionaire.model.MainMenuModel	
A főmenü modell osztálya	45
millionaire.view.MainMenuView	
A játék főmenüjének grafikus felülete	46
millionaire.controller.PlayerController	
A játékos létrehozását kezelő vezérlő osztály	48
millionaire.model.PlayerModel	
.	51
millionaire.view.PlayerView	
Játékos létrehozására szolgáló dialógusablak	58
millionaire.model.PrizeLevel	
Nyereményszintet reprezentáló osztály	61

millionaire.model.Question	
Egy kérdést reprezentáló osztály	63
millionaire.model.QuestionRepository	
Kérdések perzisztens tárolását végző osztály	67
millionaire.controller.RulesController	
A játékszabályokat kezelő vezérlő osztály	70
millionaire.model.RulesModel	
Játékszabályok kezelését végző osztály	72
millionaire.view.RulesView	
A játékszabályok megjelenítésére szolgáló ablak	73
millionaire.controller.ScoreboardController	
A dicsőséglista kezelését végző vezérlő osztály	76
millionaire.model.ScoreboardModel	
Dicsőségtábla kezelését végző osztály	78
millionaire.view.ScoreboardView	
A dicsőséglista megjelenítésére szolgáló ablak	80
millionaire.controller.TimerController	
Az időmérő vezérlője	83
millionaire.model.TimerModel	
Az időmérő adatmodellje	86
millionaire.view.TimerView	
Az időmérő megjelenítése	88

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/main/java/millionaire/ Main.java	99
src/main/java/millionaire/controller/ AddQuestionController.java	91
src/main/java/millionaire/controller/ GameController.java	92
src/main/java/millionaire/controller/ LifelineController.java	94
src/main/java/millionaire/controller/ MainMenuController.java	95
src/main/java/millionaire/controller/ PlayerController.java	96
src/main/java/millionaire/controller/ RulesController.java	97
src/main/java/millionaire/controller/ ScoreboardController.java	97
src/main/java/millionaire/controller/ TimerController.java	98
src/main/java/millionaire/model/ AddQuestionModel.java	99
src/main/java/millionaire/model/ GameModel.java	100
src/main/java/millionaire/model/ GameState.java	102
src/main/java/millionaire/model/ LifelineModel.java	102
src/main/java/millionaire/model/ MainMenuModel.java	104
src/main/java/millionaire/model/ PlayerModel.java	104
src/main/java/millionaire/model/ PrizeLevel.java	106
src/main/java/millionaire/model/ Question.java	106
src/main/java/millionaire/model/ QuestionRepository.java	108
src/main/java/millionaire/model/ RulesModel.java	110
src/main/java/millionaire/model/ ScoreboardModel.java	110
src/main/java/millionaire/model/ TimerModel.java	112
src/main/java/millionaire/view/ AddQuestionView.java	113
src/main/java/millionaire/view/ GameView.java	115
src/main/java/millionaire/view/ LifelineView.java	119
src/main/java/millionaire/view/ MainMenuView.java	121
src/main/java/millionaire/view/ PlayerView.java	123
src/main/java/millionaire/view/ RulesView.java	124
src/main/java/millionaire/view/ ScoreboardView.java	126
src/main/java/millionaire/view/ TimerView.java	128

Chapter 4

Class Documentation

4.1 millionaire.controller.AddQuestionController Class Reference

Új kérdések hozzáadását vezérlő osztály.

Collaboration diagram for millionaire.controller.AddQuestionController:

millionaire.controller.Add QuestionController
+ AddQuestionController (QuestionRepository repository) + void handleSubmit(String question, String optionA, String optionB, String optionC, String optionD, String correctAnswer, String difficulty, String category) + void handleCancel() + List< String > getDifficulties() + List< String > getCategories() + void showDialog()

Public Member Functions

- **AddQuestionController** (**QuestionRepository** repository)
Konstruktor az AddQuestionController létrehozásához.
- void **handleSubmit** (String question, String optionA, String optionB, String optionC, String optionD, String correctAnswer, String difficulty, String category)

A kérdés hozzáadása gombra kattintáskor meghívott metódus.

- void **handleCancel** ()

Bezárja a kérdés hozzáadási ablakot.

- List< String > **getDifficulties** ()

Lekérdezi az elérhető nehézségi szinteket.

- List< String > **getCategories** ()

Lekérdezi az elérhető kategóriákat.

- void **showDialog** ()

Megjeleníti a kérdés hozzáadási ablakot.

4.1.1 Detailed Description

Új kérdések hozzáadását vezérlő osztály.

Ez az osztály felelős a kérdés hozzáadási folyamat vezérléséért:

- Kezeli a felhasználói inputot a kérdés hozzáadási nézetből
- Validálja a bevitt adatokat
- Továbbítja az adatokat a modellnek feldolgozásra
- Visszajelzést ad a felhasználónak a művelet eredményéről

Definition at line 18 of file **AddQuestionController.java**.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 AddQuestionController()

```
millionaire.controller.AddQuestionController.AddQuestionController (
    QuestionRepository repository )
```

Konstruktor az AddQuestionController létrehozásához.

Parameters

<i>repository</i>	A kérdéseket tároló adatbázis
-------------------	-------------------------------

Definition at line 34 of file **AddQuestionController.java**.

4.1.3 Member Function Documentation

4.1.3.1 getCategories()

```
List< String > millionaire.controller.AddQuestionController.getCategories ( )
```

Lekérdezi az elérhető kategóriákat.

Returns

A kategóriák listája

Definition at line 95 of file **AddQuestionController.java**.

4.1.3.2 getDifficulties()

```
List< String > millionaire.controller.AddQuestionController.getDifficulties ( )
```

Lekérdezi az elérhető nehézségi szinteket.

Returns

A nehézségi szintek listája

Definition at line 86 of file **AddQuestionController.java**.

4.1.3.3 handleCancel()

```
void millionaire.controller.AddQuestionController.handleCancel ( )
```

Bezárja a kérdés hozzáadási ablakot.

Definition at line 77 of file **AddQuestionController.java**.

4.1.3.4 handleSubmit()

```
void millionaire.controller.AddQuestionController.handleSubmit (
    String question,
    String optionA,
    String optionB,
    String optionC,
    String optionD,
    String correctAnswer,
    String difficulty,
    String category )
```

A kérdés hozzáadása gombra kattintáskor meghívott metódus.

Ellenőrzi a bemeneti mezők érvényességét, majd hozzáadja a kérdést a repository-hoz. Sikeres hozzáadás esetén visszajelzést ad és törli a mezőket. Hiba esetén hibaüzenetet jelenít meg.

Parameters

<i>question</i>	A kérdés szövege
<i>optionA</i>	Az A válaszlehetőség
<i>optionB</i>	A B válaszlehetőség
<i>optionC</i>	A C válaszlehetőség
<i>optionD</i>	A D válaszlehetőség
<i>correctAnswer</i>	A helyes válasz betűjele
<i>difficulty</i>	A kérdés nehézségi szintje
<i>category</i>	A kérdés kategóriája

Definition at line 56 of file **AddQuestionController.java**.

4.1.3.5 showDialog()

```
void millionaire.controller.AddQuestionController.showDialog ( )
```

Megjeleníti a kérdés hozzáadási ablakot.

Definition at line 103 of file **AddQuestionController.java**.

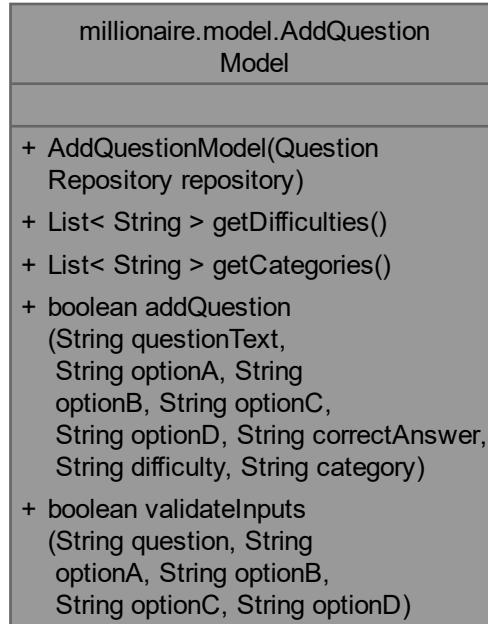
The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **AddQuestionController.java**

4.2 millionaire.model.AddQuestionModel Class Reference

Új kérdések hozzáadását kezelő modell osztály.

Collaboration diagram for millionaire.model.AddQuestionModel:



Public Member Functions

- **AddQuestionModel** (**QuestionRepository** repository)
Konstruktor az AddQuestionModel létrehozásához.
- List< String > **getDifficulties** ()
Visszaadja a kérdések nehézségeit.
- List< String > **getCategories** ()
Visszaadja a kérdések kategóriáit.
- boolean **addQuestion** (String questionText, String optionA, String optionB, String optionC, String optionD, String correctAnswer, String difficulty, String category)
A QuestionBuilder segítségével létrehoz egy új kérdést, majd hozzáadja a kérdéseket tároló repository-hoz.
- boolean **validateInputs** (String question, String optionA, String optionB, String optionC, String optionD)
Ellenőrzi, hogy a megadott bemeneti mezők érvényesek-e.

4.2.1 Detailed Description

Új kérdések hozzáadását kezelő modell osztály.

Ez az osztály felelős az új kérdések létrehozásáért és validálásáért, valamint a kérdések tárolóhoz való hozzáadásáért.

Definition at line 12 of file **AddQuestionModel.java**.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 AddQuestionModel()

```
millionaire.model.AddQuestionModel.AddQuestionModel (
    QuestionRepository repository )
```

Konstruktor az AddQuestionModel létrehozásához.

Parameters

<i>repository</i>	A kérdéseket tároló adatbázis
-------------------	-------------------------------

Definition at line 20 of file **AddQuestionModel.java**.

4.2.3 Member Function Documentation

4.2.3.1 addQuestion()

```
boolean millionaire.model.AddQuestionModel.addQuestion (
    String questionText,
    String optionA,
    String optionB,
    String optionC,
    String optionD,
```

```
String correctAnswer,
String difficulty,
String category )
```

A QuestionBuilder segítségével létrehoz egy új kérdést, majd hozzáadja a kérdéseket tároló repository-hoz.

Parameters

<i>questionText</i>	A kérdés szövege
<i>optionA</i>	A kérdéshez tartozó A válasz
<i>optionB</i>	A kérdéshez tartozó B válasz
<i>optionC</i>	A kérdéshez tartozó C válasz
<i>optionD</i>	A kérdéshez tartozó D válasz
<i>correctAnswer</i>	A helyes válasz betűjele (A, B, C vagy D)
<i>difficulty</i>	A kérdés nehézségi szintje (1-15)
<i>category</i>	A kérdés kategóriája

Returns

True, ha a kérdés hozzáadása a tárolóhoz sikeres, egyébként false

Exceptions

<i>NumberFormatException</i>	Ha a nehézségi szint nem alakítható számmá
<i>IllegalArgumentException</i>	Ha a builder érvénytelen adatokat kap

Definition at line 57 of file **AddQuestionModel.java**.

4.2.3.2 getCategories()

```
List< String > millionaire.model.AddQuestionModel.getCategories ( )
```

Visszaadja a kérdések kategóriáit.

Returns

A kérdések kategóriái.

Definition at line 38 of file **AddQuestionModel.java**.

4.2.3.3 getDifficulties()

```
List< String > millionaire.model.AddQuestionModel.getDifficulties ( )
```

Visszaadja a kérdések nehézségeit.

Returns

A kérdések nehézségei.

Definition at line 29 of file **AddQuestionModel.java**.

4.2.3.4 validateInputs()

```
boolean millionaire.model.AddQuestionModel.validateInputs (
    String question,
    String optionA,
    String optionB,
    String optionC,
    String optionD )
```

Ellenőrzi, hogy a megadott bemeneti mezők érvényesek-e.

Egy bemenet akkor érvényes, ha egyik kötelező mező sem üres. Az üres sztringeket és a csak whitespace karaktereket tartalmazó sztringeket is üresnek tekintjük.

Parameters

<i>question</i>	A feltenni kívánt kérdés
<i>optionA</i>	A kérdéshez tartozó A válasz
<i>optionB</i>	A kérdéshez tartozó B válasz
<i>optionC</i>	A kérdéshez tartozó C válasz
<i>optionD</i>	A kérdéshez tartozó D válasz

Returns

True, ha minden szükséges bemeneti mező ki van töltve, egyébként false

Definition at line 92 of file **AddQuestionModel.java**.

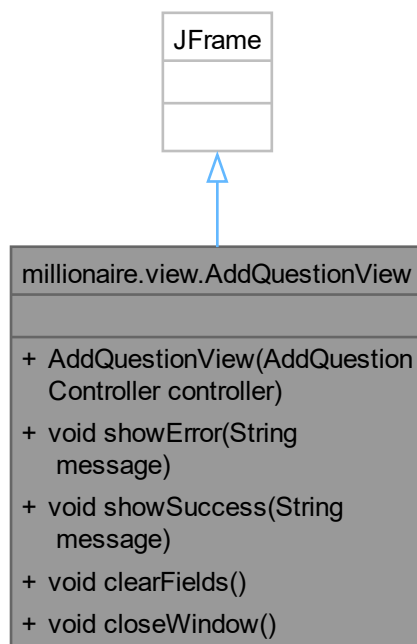
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **AddQuestionModel.java**

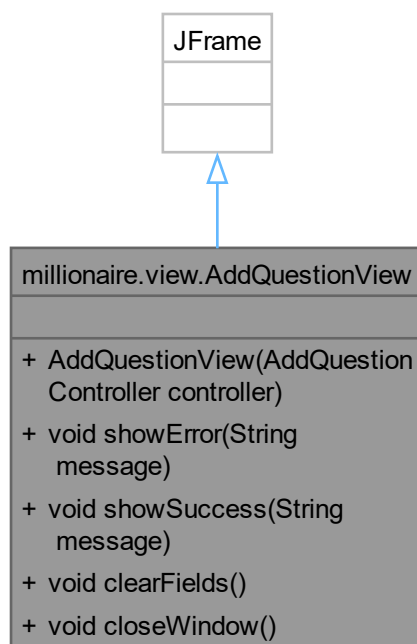
4.3 millionaire.view.AddQuestionView Class Reference

Új kérdések hozzáadására szolgáló grafikus felület.

Inheritance diagram for millionaire.view.AddQuestionView:



Collaboration diagram for millionaire.view.AddQuestionView:



Public Member Functions

- **AddQuestionView** (**AddQuestionController** controller)
Konstruktor az AddQuestionView létrehozásához.
- void **showError** (String message)
Hibaüzenet megjelenítése.
- void **showSuccess** (String message)
Sikeres művelet visszajelzése.
- void **clearFields** ()
Törli az űrlap mezőinek tartalmát.
- void **closeWindow** ()
Bezárja az ablakot.

4.3.1 Detailed Description

Új kérdések hozzáadására szolgáló grafikus felület.

Ez az osztály felelős az új kérdések hozzáadásához szükséges felhasználói felület megjelenítéséért és kezeléséért. A felület tartalmaz:

- Beviteli mezőket a kérdés adatainak megadásához
- Legördülő menüket a kategória és nehézség kiválasztásához
- Gombokat a művelet végrehajtásához vagy megszakításához

Definition at line **18** of file **AddQuestionView.java**.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 AddQuestionView()

```
millionaire.view.AddQuestionView.AddQuestionView (
    AddQuestionController controller )
```

Konstruktor az AddQuestionView létrehozásához.

Parameters

<i>controller</i>	A kérdés hozzáadását kezelő controller
-------------------	--

Definition at line 42 of file **AddQuestionView.java**.

4.3.3 Member Function Documentation

4.3.3.1 clearFields()

```
void millionaire.view.AddQuestionView.clearFields ( )
```

Törli az űrlap mezőinek tartalmát.

Alaphelyzetbe állítja a beviteli mezőket és a legördülő menüket

Definition at line 298 of file **AddQuestionView.java**.

4.3.3.2 closeWindow()

```
void millionaire.view.AddQuestionView.closeWindow ( )
```

Bezárja az ablakot.

Felszabadítja az ablakhoz tartozó erőforrásokat és eltávolítja az ablakot a képernyőről.

Definition at line 315 of file **AddQuestionView.java**.

4.3.3.3 showError()

```
void millionaire.view.AddQuestionView.showError (
    String message )
```

Hibaüzenet megjelenítése.

Felugró ablakban jeleníti meg a hibaüzenetet.

Parameters

<i>message</i>	A megjelenítendő hibaüzenet
----------------	-----------------------------

Definition at line 276 of file **AddQuestionView.java**.

4.3.3.4 showSuccess()

```
void millionaire.view.AddQuestionView.showSuccess (
    String message )
```

Sikeres művelet visszajelzése.

Felugró ablakban jeleníti meg a sikeres művelet üzenetét.

Parameters

<i>message</i>	A megjelenítendő üzenet
----------------	-------------------------

Definition at line 288 of file **AddQuestionView.java**.

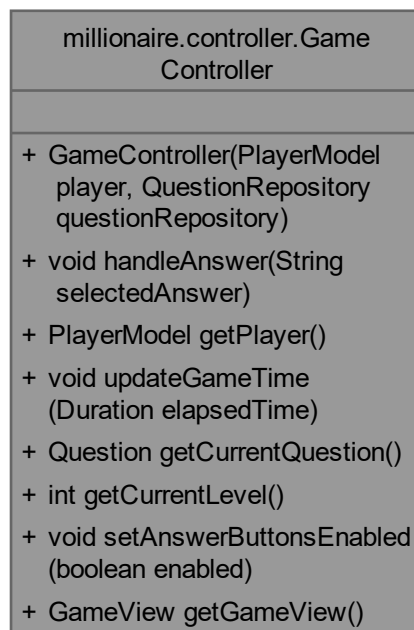
The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **AddQuestionView.java**

4.4 millionaire.controller.GameController Class Reference

A játék fő vezérlő osztálya.

Collaboration diagram for millionaire.controller.GameController:



Public Member Functions

- **GameController** (**PlayerModel** player, **QuestionRepository** questionRepository)
Konstruktor a GameController létrehozásához.
- void **handleAnswer** (String selectedAnswer)
Kezeli a játékos által választott választ.
- **PlayerModel** **getPlayer** ()
Visszaadja az aktuális játékos modelljét.
- void **updateGameTime** (Duration elapsedTime)
Frissíti a játékidőt a modellben.
- **Question** **getCurrentQuestion** ()
Visszaadja az aktuális kérdést.
- int **getCurrentLevel** ()
Visszaadja az aktuális szintet.
- void **setAnswerButtonsEnabled** (boolean enabled)
Beállítja a válaszgombok engedélyezettségi állapotát.
- **GameView** **getGameView** ()
Visszaadja a játék nézetét.

4.4.1 Detailed Description

A játék fő vezérlő osztálya.

Ez az osztály felelős a teljes játékmenet vezérléséért:

- Koordinálja a játék állapotának változásait
- Kezeli a játékos válaszait
- Irányítja a segítségék használatát
- Frissíti a játék nézetét
- Kezeli a játék végét és az eredmények mentését

Definition at line 21 of file **GameController.java**.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 GameController()

```
millionaire.controller.GameController.GameController (
    PlayerModel player,
    QuestionRepository questionRepository )
```

Konstruktor a GameController létrehozásához.

Parameters

<i>player</i>	A játékos modellje
<i>questionRepository</i>	A kérdéseket tároló adatbázis

Definition at line 33 of file **GameController.java**.

4.4.3 Member Function Documentation

4.4.3.1 getCurrentLevel()

```
int millionaire.controller.GameController.getCurrentLevel ( )
```

Visszaadja az aktuális szintet.

Returns

Az aktuális szint

Definition at line 219 of file **GameController.java**.

4.4.3.2 getCurrentQuestion()

```
Question millionaire.controller.GameController.getCurrentQuestion ( )
```

Visszaadja az aktuális kérdést.

Returns

Az aktuális kérdés

Definition at line 210 of file **GameController.java**.

4.4.3.3 getGameView()

```
GameView millionaire.controller.GameController.getGameView ( )
```

Visszaadja a játék nézetét.

Returns

A játék nézete

Definition at line 237 of file **GameController.java**.

4.4.3.4 getPlayer()

```
PlayerModel millionaire.controller.GameController.getPlayer ( )
```

Visszaadja az aktuális játékos modelljét.

Returns

A játékos modellje

Definition at line 181 of file **GameController.java**.

4.4.3.5 handleAnswer()

```
void millionaire.controller.GameController.handleAnswer (
    String selectedAnswer )
```

Kezeli a játékos által választott választ.

Letiltja a választógombokat a feldolgozás idejére, megjeleníti a válasz helyességét, majd késleltetés után folytatja a játékot vagy befejezi azt.

Parameters

<i>selectedAnswer</i>	A választott válasz betűjele
-----------------------	------------------------------

Definition at line **61** of file **GameController.java**.

4.4.3.6 setAnswerButtonsEnabled()

```
void millionaire.controller.GameController.setAnswerButtonsEnabled (
    boolean enabled )
```

Beállítja a választógombok engedélyezettségi állapotát.

Parameters

<i>enabled</i>	Az engedélyezettség állapota
----------------	------------------------------

Definition at line **228** of file **GameController.java**.

4.4.3.7 updateGameTime()

```
void millionaire.controller.GameController.updateGameTime (
    Duration elapsedTime )
```

Frissíti a játékidőt a modellben.

Parameters

<i>elapsedTime</i>	Az eltelt játékidő
--------------------	--------------------

Definition at line **190** of file **GameController.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **GameController.java**

4.5 millionaire.model.GameModel Class Reference

A játék logikáját és állapotát kezelő osztály.

Collaboration diagram for millionaire.model.GameModel:

millionaire.model.GameModel
+ GameModel(QuestionRepository questionRepository, PlayerModel player) + Question selectNextQuestion() + GameState processAnswer (String selectedAnswer) + long getLastSafePrize() + long getCurrentPrize() + PlayerModel getPlayer() + Question getCurrentQuestion() + boolean isGameOver() + void updatePlayTime (Duration elapsedTime) + boolean isTimeUp(Duration currentTime) + static List< PrizeLevel > getPrizeLevels()

Public Member Functions

- **GameModel** (**QuestionRepository** questionRepository, **PlayerModel** player)
Konstruktor a játékmódel létrehozásához.
- **Question** **selectNextQuestion** ()
Kiválaszt egy következő kérdést az aktuális szintnek megfelelően.
- **GameState** **processAnswer** (String selectedAnswer)
Feldolgozza a játékos választát.
- long **getLastSafePrize** ()
Meghatározza az utolsó elért garantált nyereményt.
- long **getCurrentPrize** ()
Visszaadja az aktuális szinthez tartozó nyereményösszeget.
- **PlayerModel** **getPlayer** ()
Visszaadja az aktuális játékost.
- **Question** **getCurrentQuestion** ()
Visszaadja az aktuális kérdést.
- boolean **isGameOver** ()
Visszaadja, hogy véget ért-e a játék.
- void **updatePlayTime** (Duration elapsedTime)
Frissíti a játékos játékidéjét.
- boolean **isTimeUp** (Duration currentTime)
Ellenőrzi, hogy a játék időlimitje lejárt-e (ha van)

Static Public Member Functions

- static List< **PrizeLevel** > **getPrizeLevels** ()
Létrehozza a nyereményszintek listáját.

4.5.1 Detailed Description

A játék logikáját és állapotát kezelő osztály.

Ez az osztály felelős a játék fő működéséért, beleértve:

- Kérdések kiválasztását
- Válaszok feldolgozását
- Nyeremények kezelését
- Játékállapot követését

Definition at line 17 of file **GameModel.java**.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 GameModel()

```
millionaire.model.GameModel.GameModel (
    QuestionRepository questionRepository,
    PlayerModel player )
```

Konstruktor a játékmódel létrehozásához.

Parameters

<i>questionRepository</i>	A kérdéseket tároló adatbázis
<i>player</i>	Az aktuális játékos modellje

Definition at line 42 of file **GameModel.java**.

4.5.3 Member Function Documentation

4.5.3.1 getCurrentPrize()

```
long millionaire.model.GameModel.getCurrentPrize ( )
```

Visszaadja az aktuális szinthez tartozó nyereményösszeget.

Returns

Az aktuális nyereményösszeg

Definition at line 113 of file **GameModel.java**.

4.5.3.2 getCurrentQuestion()

```
Question millionaire.model.GameModel.getCurrentQuestion ( )
```

Visszaadja az aktuális kérdést.

Definition at line 123 of file **GameModel.java**.

4.5.3.3 getLastSafePrize()

```
long millionaire.model.GameModel.getLastSafePrize ( )
```

Meghatározza az utolsó elért garantált nyereményt.

Returns

Az utolsó garantált nyeremény összege

Definition at line 99 of file **GameModel.java**.

4.5.3.4 getPlayer()

```
PlayerModel millionaire.model.GameModel.getPlayer ( )
```

Visszaadja az aktuális játékost.

Definition at line 119 of file **GameModel.java**.

4.5.3.5 getPrizeLevels()

```
static List< PrizeLevel > millionaire.model.GameModel.getPrizeLevels ( ) [static]
```

Létrehozza a nyereményszintek listáját.

Returns

Nyereményszintek listája a megfelelő információkkal

Definition at line 134 of file **GameModel.java**.

4.5.3.6 isGameOver()

```
boolean millionaire.model.GameModel.isGameOver ( )
```

Visszaadja, hogy véget ért-e a játék.

Definition at line 127 of file **GameModel.java**.

4.5.3.7 isTimeUp()

```
boolean millionaire.model.GameModel.isTimeUp (   
    Duration currentTime )
```

Ellenőrzi, hogy a játék időlimitje lejárt-e (ha van)

Parameters

<i>currentTime</i>	Az aktuális játékidő
--------------------	----------------------

Returns

true ha lejárt az idő, false ha nem

Definition at line **160** of file **GameModel.java**.

4.5.3.8 processAnswer()

```
GameState millionaire.model.GameModel.processAnswer (
    String selectedAnswer )
```

Feldolgozza a játékos választát.

Parameters

<i>selectedAnswer</i>	A játékos által választott válasz betűjele
-----------------------	--

Returns

A játék állapota a válasz feldolgozása után

Definition at line **72** of file **GameModel.java**.

4.5.3.9 selectNextQuestion()

```
Question millionaire.model.GameModel.selectNextQuestion ( )
```

Kiválaszt egy következő kérdést az aktuális szintnek megfelelően.

Returns

A kiválasztott kérdés

Exceptions

<i>IllegalStateException</i>	Ha nincs elérhető kérdés az adott szinten
------------------------------	---

Definition at line **57** of file **GameModel.java**.

4.5.3.10 updatePlayTime()

```
void millionaire.model.GameModel.updatePlayTime (
    Duration elapsedTime )
```

Frissíti a játékos játékidéjét.

Parameters

<i>elapsedTime</i>	Az eltelt játékidő
--------------------	--------------------

Definition at line **150** of file **GameModel.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **GameModel.java**

4.6 millionaire.model.GameState Enum Reference

Enum osztály, amely a játék állapotait reprezentálja.

Collaboration diagram for millionaire.model.GameState:

millionaire.model.GameState
+ CORRECT
+ LOST
+ WON
+ boolean isGameEnding()

Public Member Functions

- boolean **isGameEnding** ()

Megadja, hogy a játék olyan állapotban van-e, amikor véget kell érnie.

Public Attributes

- CORRECT
- LOST
- WON

4.6.1 Detailed Description

Enum osztály, amely a játék állapotait reprezentálja.

Definition at line **7** of file **GameState.java**.

4.6.2 Member Function Documentation

4.6.2.1 isGameEnding()

```
boolean millionaire.model.GameState.isGameEnding ( )
```

Megadja, hogy a játék olyan állapotban van-e, amikor véget kell érnie.

Returns

True, ha a játék véget ért, false egyébként.

Definition at line 30 of file **GameState.java**.

4.6.3 Member Data Documentation

4.6.3.1 CORRECT

```
millionaire.model.GameState.CORRECT
```

Ha nem az utolsó kérdésre válaszolt a játékos, és helyes választ adott. A játék folytatódik a következő kérdéssel.

Definition at line 12 of file **GameState.java**.

4.6.3.2 LOST

```
millionaire.model.GameState.LOST
```

Ha a játékos rossz választ adott, vagy az utolsó kérdésre helytelen választ adott. A játék véget ér, a játékos vagy nem nyer semmit, vagy a legutóbb elért biztos nyereményt kapja.

Definition at line 18 of file **GameState.java**.

4.6.3.3 WON

```
millionaire.model.GameState.WON
```

A játékos a 15. kérdésre helyes választ adott, így megnyerte a főnyereményt. A játék véget ér, a játékos megszerezte a főnyereményt.

Definition at line 24 of file **GameState.java**.

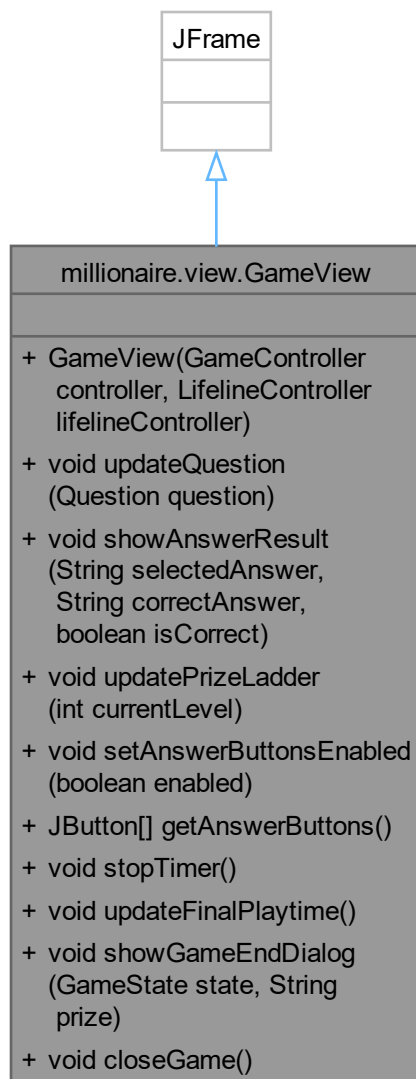
The documentation for this enum was generated from the following file:

- src/main/java/millionaire/model/ **GameState.java**

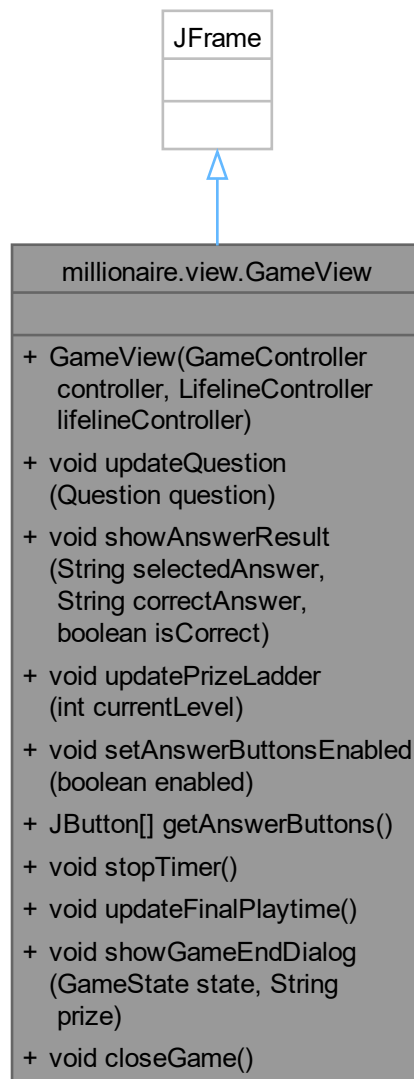
4.7 millionaire.view.GameView Class Reference

A játék fő grafikus felülete.

Inheritance diagram for millionaire.view.GameView:



Collaboration diagram for millionaire.view.GameView:



Public Member Functions

- **GameView** (**GameController** controller, **LifelineController** lifelineController)
Konstruktor a GameView létrehozásához.
- void **updateQuestion** (**Question** question)
Frissíti a kérdést és a válaszlehetőségeket.
- void **showAnswerResult** (String selectedAnswer, String correctAnswer, boolean isCorrect)
Megjeleníti a választott válasz eredményét.
- void **updatePrizeLadder** (int currentLevel)
Frissíti a nyereménylétrát.
- void **setAnswerButtonsEnabled** (boolean enabled)

- *Beállítja a válaszgombok engedélyezettségi állapotát.*
 • JButton[] **getAnswerButtons** ()
Visszaadja a válaszgombokat.
- void **stopTimer** ()
Leállítja az időmérőt.
- void **updateFinalPlaytime** ()
Frissíti a végleges játékidjét a játékosnak.
- void **showGameEndDialog** (GameState state, String prize)
Megjeleníti a játék végeredményét.
- void **closeGame** ()
Bezárja a játék ablakát.

4.7.1 Detailed Description

A játék fő grafikus felülete.

Ez az osztály felelős a játék fő képernyőjének megjelenítéséért és kezeléséért. A felület tartalmazza:

- A kérdés megjelenítését
- A négy válaszlehetőség gombjait
- A nyereményletrát
- A segítség ikonjait
- A játékállapot megjelenítését

Definition at line 26 of file **GameView.java**.

4.7.2 Constructor & Destructor Documentation

4.7.2.1 GameView()

```
millionaire.view.GameView.GameView (
    GameController controller,
    LifelineController lifelineController )
```

Konstruktor a GameView létrehozásához.

Parameters

<i>controller</i>	A játékot vezérlő controller
<i>lifelineController</i>	A segítségeket kezelő controller

Definition at line 52 of file **GameView.java**.

4.7.3 Member Function Documentation

4.7.3.1 closeGame()

```
void millionaire.view.GameView.closeGame ( )
```

Bezárja a játék ablakát.

Felszabadítja az erőforrásokat és eltávolítja az ablakot a képernyőről.

Definition at line **473** of file **GameView.java**.

4.7.3.2 getAnswerButtons()

```
JButton[] millionaire.view.GameView.getAnswerButtons ( )
```

Visszaadja a válaszgombokat.

Returns

A válaszgombok tömbje

Definition at line **425** of file **GameView.java**.

4.7.3.3 setAnswerButtonsEnabled()

```
void millionaire.view.GameView.setAnswerButtonsEnabled (
    boolean enabled )
```

Beállítja a válaszgombok engedélyezettségi állapotát.

Parameters

<i>enabled</i>	Az engedélyezettség állapota
----------------	------------------------------

Definition at line **413** of file **GameView.java**.

4.7.3.4 showAnswerResult()

```
void millionaire.view.GameView.showAnswerResult (
    String selectedAnswer,
    String correctAnswer,
    boolean isCorrect )
```

Megjeleníti a választott válasz eredményét.

- Letiltja a válaszgombokat
- Színezi a választott választ (zöld/piros)
- Helytelen válasz esetén megjelöli a helyes választ

Parameters

<i>selectedAnswer</i>	A választott válasz betűjele
<i>correctAnswer</i>	A helyes válasz betűjele
<i>isCorrect</i>	A válasz helyességét jelző flag

Definition at line 348 of file **GameView.java**.

4.7.3.5 showGameEndDialog()

```
void millionaire.view.GameView.showGameEndDialog (
    GameState state,
    String prize )
```

Megjeleníti a játék végeredményét.

Felugró ablakban jeleníti meg:

- A játék végeredményét (nyert/vesztett)
- Az elért nyereményt

Parameters

<i>state</i>	A játék végállapota
<i>prize</i>	Az elért nyeremény

Definition at line 456 of file **GameView.java**.

4.7.3.6 stopTimer()

```
void millionaire.view.GameView.stopTimer ( )
```

Leállítja az időmérőt.

Definition at line 433 of file **GameView.java**.

4.7.3.7 updateFinalPlaytime()

```
void millionaire.view.GameView.updateFinalPlaytime ( )
```

Frissíti a végleges játékidjét a játékosnak.

Definition at line 441 of file **GameView.java**.

4.7.3.8 updatePrizeLadder()

```
void millionaire.view.GameView.updatePrizeLadder (
    int currentLevel )
```

Frissíti a nyeremenylétrát.

Kiemeli az aktuális szintet és frissíti a megjelenítést

Parameters

<i>currentLevel</i>	Az aktuális szint száma
---------------------	-------------------------

Definition at line 369 of file **GameView.java**.

4.7.3.9 updateQuestion()

```
void millionaire.view.GameView.updateQuestion (
    Question question )
```

Frissíti a kérdést és a válaszlehetőségeket.

Megjeleníti:

- Az új kérdés szövegét
- Az új válaszlehetőségeket
- Alaphelyzetbe állítja a válaszgombokat

Parameters

<i>question</i>	Az új kérdés objektum
-----------------	-----------------------

Definition at line 321 of file **GameView.java**.

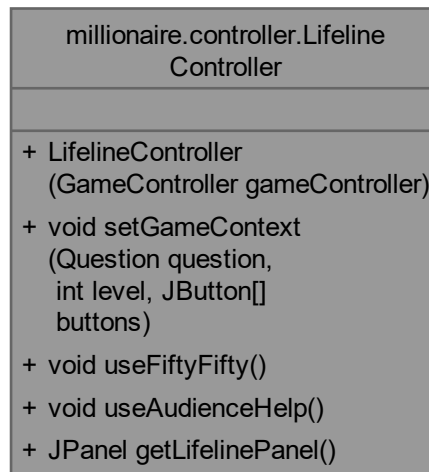
The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **GameView.java**

4.8 millionaire.controller.LifelineController Class Reference

A játék segítségét kezelő vezérlő osztály.

Collaboration diagram for millionaire.controller.LifelineController:



Public Member Functions

- **LifelineController (GameController gameController)**
Konstruktor a LifelineController létrehozásához.
- void **setGameContext (Question question, int level, JButton[] buttons)**
Beállítja a játék aktuális kontextusát.
- void **useFiftyFifty ()**
A felezés segítség használata.
- void **useAudienceHelp ()**
A közönség segítségének használata.
- JPanel **getLifelinePanel ()**
Visszaadja a segítségek paneljét.

4.8.1 Detailed Description

A játék segítségeit kezelő vezérlő osztály.

Ez az osztály felelős a játék során használható segítségek kezeléséért:

- Felezés segítség használata
- Közönség segítségének használata
- Segítségek állapotának követése
- Segítségek eredményének megjelenítése

Definition at line **23** of file **LifelineController.java**.

4.8.2 Constructor & Destructor Documentation

4.8.2.1 LifelineController()

```
millionaire.controller.LifelineController.LifelineController (
    GameController gameController )
```

Konstruktor a LifelineController létrehozásához.

Parameters

<i>gameController</i>	A fő játékvezérlő
-----------------------	-------------------

Definition at line 36 of file LifelineController.java.

4.8.3 Member Function Documentation

4.8.3.1 getLifelinePanel()

```
JPanel millionaire.controller.LifelineController.getLifelinePanel ( )
```

Visszaadja a segítségék paneljét.

Returns

A segítségék panelje

Definition at line 120 of file LifelineController.java.

4.8.3.2 setGameContext()

```
void millionaire.controller.LifelineController.setGameContext (
    Question question,
    int level,
    JButton[] buttons )
```

Beállítja a játék aktuális kontextusát.

Frissíti:

- Az aktuális kérdést
- Az aktuális szintet
- A válaszgombokat
- A segítségék elérhetőségét

Parameters

<i>question</i>	Az aktuális kérdés
<i>level</i>	Az aktuális szint
<i>buttons</i>	A válasszgombok

Definition at line 56 of file **LifelineController.java**.

4.8.3.3 useAudienceHelp()

```
void millionaire.controller.LifelineController.useAudienceHelp ( )
```

A közönség segítségének használata.

Generál egy szavazási eredményt, megjeleníti az eredményt.

Definition at line 88 of file **LifelineController.java**.

4.8.3.4 useFiftyFifty()

```
void millionaire.controller.LifelineController.useFiftyFifty ( )
```

A felezés segítség használata.

Eltávolít két helytelen választ, frissíti a gombok állapotát.

Definition at line 69 of file **LifelineController.java**.

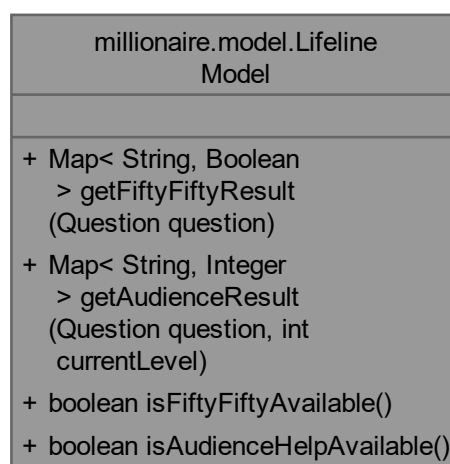
The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **LifelineController.java**

4.9 millionaire.model.LifelineModel Class Reference

Segítségek (lifeline) logikáját kezelő osztály.

Collaboration diagram for millionaire.model.LifelineModel:



Public Member Functions

- Map< String, Boolean > **getFiftyFiftyResult** (**Question** question)
Felezés segítség végrehajtása.
- Map< String, Integer > **getAudienceResult** (**Question** question, int currentLevel)
Közönség segítségének szimulálása.
- boolean **isFiftyFiftyAvailable** ()
Ellenőrzi, hogy a felezés segítség még használható-e.
- boolean **isAudienceHelpAvailable** ()
Ellenőrzi, hogy a közönség segítség még használható-e.

4.9.1 Detailed Description

Segítségek (lifeline) logikáját kezelő osztály.

Ez az osztály felelős a játékban használható segítségek (felezés, közönség) működésének kezeléséért és a válaszok generálásáért.

Definition at line 12 of file **LifelineModel.java**.

4.9.2 Member Function Documentation

4.9.2.1 getAudienceResult()

```
Map< String, Integer > millionaire.model.LifelineModel.getAudienceResult (
    Question question,
    int currentLevel )
```

Közönség segítségének szimulálása.

A játék aktuális szintjétől függően generál szavazati arányokat. A magasabb szinteken a közönség kevésbé biztos a helyes válaszban.

Parameters

<i>question</i>	Az aktuális kérdés
<i>currentLevel</i>	A játék aktuális szintje

Returns

Map, ami tartalmazza a válaszlehetőségekhez tartozó százalékos arányokat

Exceptions

<i>IllegalStateException</i>	Ha a közönség segítséget már használták
------------------------------	---

Definition at line 63 of file **LifelineModel.java**.

4.9.2.2 getFiftyFiftyResult()

```
Map< String, Boolean > millionaire.model.LifelineModel.getFiftyFiftyResult (
    Question question )
```

Felezés segítség végrehajtása.

Két rossz választ eltávolít a négy lehetőség közül, meghagyva a helyes választ és egy véletlenszerűen kiválasztott rossz választ.

Parameters

<i>question</i>	Az aktuális kérdés
-----------------	--------------------

Returns

Map, ami tartalmazza minden válaszlehetőséghez, hogy látható-e

Exceptions

<i>IllegalStateException</i>	Ha a felezés segítséget már használták
------------------------------	--

Definition at line 28 of file **LifelineModel.java**.

4.9.2.3 isAudienceHelpAvailable()

```
boolean millionaire.model.LifelineModel.isAudienceHelpAvailable ( )
```

Ellenőrzi, hogy a közönség segítség még használható-e.

Returns

true ha a segítség még nem volt használva, egyébként false

Definition at line 142 of file **LifelineModel.java**.

4.9.2.4 isFiftyFiftyAvailable()

```
boolean millionaire.model.LifelineModel.isFiftyFiftyAvailable ( )
```

Ellenőrzi, hogy a felezés segítség még használható-e.

Returns

true ha a segítség még nem volt használva, egyébként false

Definition at line 133 of file **LifelineModel.java**.

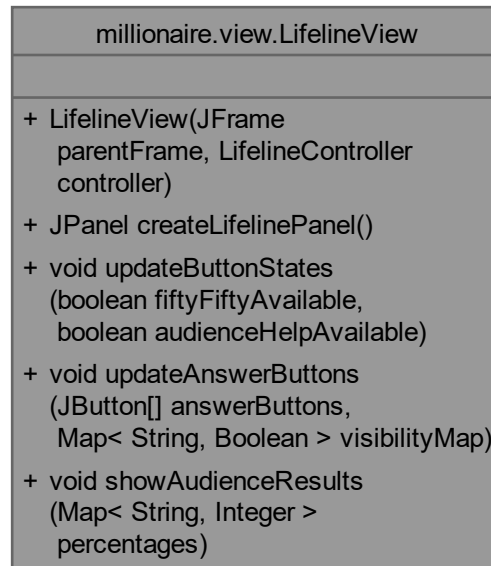
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **LifelineModel.java**

4.10 millionaire.view.LifelineView Class Reference

A játék segítségait megjelenítő és kezelő felület.

Collaboration diagram for millionaire.view.LifelineView:



Public Member Functions

- **LifelineView** (JFrame parentFrame, **LifelineController** controller)
Konstruktor a LifelineView létrehozásához.
- JPanel **createLifelinePanel** ()
Létrehozza a segítségok paneljét.
- void **updateButtonStates** (boolean fiftyFiftyAvailable, boolean audienceHelpAvailable)
Frissíti a segítség gombok állapotát.
- void **updateAnswerButtons** (JButton[] answerButtons, Map< String, Boolean > visibilityMap)
Frissíti a válaszgombokat a felezés segítség használatakor.
- void **showAudienceResults** (Map< String, Integer > percentages)
Megjeleníti a közönség szavazatainak eredményét.

4.10.1 Detailed Description

A játék segítségait megjelenítő és kezelő felület.

Ez az osztály felelős a játék során használható segítségok (felezés, közönség) grafikus megjelenítéséért és interakcióinak kezeléséért. A felület tartalmazza:

- A segítségeket aktiváló gombokat
- A segítségok állapotának megjelenítését
- A közönség szavazatainak vizualizációját

Definition at line **18** of file **LifelineView.java**.

4.10.2 Constructor & Destructor Documentation

4.10.2.1 LifelineView()

```
millionaire.view.LifelineView.LifelineView (
    JFrame parentFrame,
    LifelineController controller )
```

Konstruktor a LifelineView létrehozásához.

Parameters

<i>parentFrame</i>	A szülő ablak
<i>controller</i>	A segítségeket kezelő controller

Definition at line 36 of file **LifelineView.java**.

4.10.3 Member Function Documentation

4.10.3.1 createLifelinePanel()

```
JPanel millionaire.view.LifelineView.createLifelinePanel ( )
```

Létrehozza a segítségek paneljét.

Létrehozza a segítség gombokat tartalmazó panelt, megfelelő elrendezéssel és stílusbeállításokkal.

Returns

A létrehozott panel a segítség gombokkal

Definition at line 106 of file **LifelineView.java**.

4.10.3.2 showAudienceResults()

```
void millionaire.view.LifelineView.showAudienceResults (
    Map< String, Integer > percentages )
```

Megjeleníti a közönség szavazatainak eredményét.

Létrehoz egy dialógusablakot, amely megjeleníti a közönség szavazatainak százalékos megoszlását.

Parameters

<i>percentages</i>	A válaszokra adott szavazatok százalékos megoszlása
--------------------	---

Definition at line 163 of file **LifelineView.java**.

4.10.3.3 updateAnswerButtons()

```
void millionaire.view.LifelineView.updateAnswerButtons (
    JButton[] answerButtons,
    Map< String, Boolean > visibilityMap )
```

Frissíti a válaszgombokat a felezés segítség használatakor.

A felezés segítség használatakor:

- Letiltja a kiválasztott rossz válaszokat
- Törli a letiltott válaszok szövegét

Parameters

<i>answerButtons</i>	A válaszgombok tömbje
<i>visibilityMap</i>	A válaszok láthatóságát tartalmazó térkép

Definition at line **145** of file **LifelineView.java**.

4.10.3.4 updateButtonStates()

```
void millionaire.view.LifelineView.updateButtonStates (
    boolean fiftyFiftyAvailable,
    boolean audienceHelpAvailable )
```

Frissíti a segítség gombok állapotát.

Frissíti a gombok:

- Engedélyezettségi állapotát
- Színeit A felhasznált segítések gombjai letiltásra kerülnek.

Parameters

<i>fiftyFiftyAvailable</i>	A felezés segítség elérhetősége
<i>audienceHelpAvailable</i>	A közönség segítség elérhetősége

Definition at line **126** of file **LifelineView.java**.

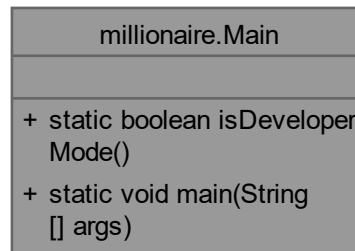
The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **LifelineView.java**

4.11 millionaire.Main Class Reference

A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.

Collaboration diagram for millionaire.Main:



Static Public Member Functions

- static boolean **isDeveloperMode** ()
Visszaadja, hogy a program fejlesztői módban fut-e.
- static void **main** (String[] args)
A program belépési pontja.

4.11.1 Detailed Description

A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.

Author

Zeisz Ottó (HP9G2J)

Date

2024. november 24.

Ez az osztály a program belépési pontja, amely elindítja a főmenüt és biztosítja, hogy a felhasználói felület az Event Dispatch Thread-en fusson.

Definition at line **15** of file **Main.java**.

4.11.2 Member Function Documentation

4.11.2.1 isDeveloperMode()

```
static boolean millionaire.Main.isDeveloperMode ( ) [static]
```

Visszaadja, hogy a program fejlesztői módban fut-e.

Returns

true ha fejlesztői módban fut, false ha nem

Definition at line **22** of file **Main.java**.

4.11.2.2 main()

```
static void millionaire.Main.main (
    String[] args ) [static]
```

A program belépési pontja.

Parameters

<i>args</i>	Parancssori argumentumok: Ha args[0] == "dev", akkor fejlesztői módban indul
-------------	--

Definition at line **31** of file **Main.java**.

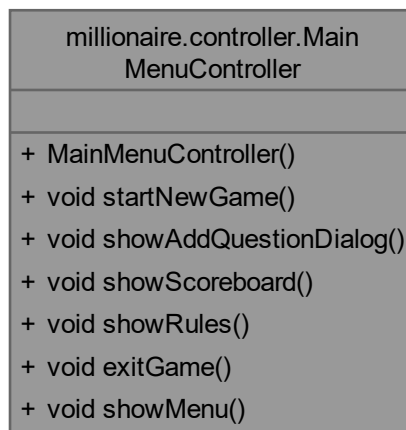
The documentation for this class was generated from the following file:

- src/main/java/millionaire/ **Main.java**

4.12 millionaire.controller.MainMenuController Class Reference

A főmenü vezérlő osztálya.

Collaboration diagram for millionaire.controller.MainMenuController:



Public Member Functions

- **MainMenuController ()**
Konstruktor a MainMenuController létrehozásához.
- void **startNewGame ()**
Új játék indítása.

- void **showAddQuestionDialog** ()
Megjeleníti a kérdés hozzáadási dialógust.
- void **showScoreboard** ()
Megjeleníti a dicsőséglistát.
- void **showRules** ()
Megjeleníti a játékszabályokat.
- void **exitGame** ()
Kilép a játékból.
- void **showMenu** ()
Megjeleníti a főmenüt.

4.12.1 Detailed Description

A főmenü vezérlő osztálya.

Ez az osztály felelős a főmenü működésének vezérléséért:

- Új játék indítása
- Kérdés hozzáadási felület megnyitása
- Dicsőséglista megjelenítése
- Játékszabályok megjelenítése
- Játékból való kilépés

Definition at line 18 of file **MainMenuController.java**.

4.12.2 Constructor & Destructor Documentation

4.12.2.1 MainMenuController()

```
millionaire.controller.MainMenuController.MainMenuController ( )
```

Konstruktor a MainMenuController létrehozásához.

Definition at line 25 of file **MainMenuController.java**.

4.12.3 Member Function Documentation

4.12.3.1 exitGame()

```
void millionaire.controller.MainMenuController.exitGame ( )
```

Kilép a játékból.

Definition at line 79 of file **MainMenuController.java**.

4.12.3.2 showAddQuestionDialog()

```
void millionaire.controller.MainMenuController.showAddQuestionDialog ( )
```

Megjeleníti a kérdés hozzáadási dialógust.

Definition at line 51 of file **MainMenuController.java**.

4.12.3.3 showMenu()

```
void millionaire.controller.MainMenuController.showMenu ( )
```

Megjeleníti a főmenüt.

Definition at line 87 of file **MainMenuController.java**.

4.12.3.4 showRules()

```
void millionaire.controller.MainMenuController.showRules ( )
```

Megjeleníti a játékszabályokat.

Definition at line 70 of file **MainMenuController.java**.

4.12.3.5 showScoreboard()

```
void millionaire.controller.MainMenuController.showScoreboard ( )
```

Megjeleníti a dicsőséglistát.

Definition at line 61 of file **MainMenuController.java**.

4.12.3.6 startNewGame()

```
void millionaire.controller.MainMenuController.startNewGame ( )
```

Új játék indítása.

- Megjeleníti a játékos létrehozási dialógust
- Sikeres létrehozás esetén elindítja a játékot

Definition at line 37 of file **MainMenuController.java**.

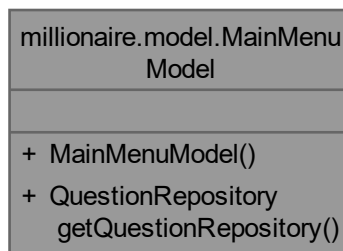
The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **MainMenuController.java**

4.13 millionaire.model.MainMenuModel Class Reference

A főmenü modell osztálya.

Collaboration diagram for millionaire.model.MainMenuModel:



Public Member Functions

- **MainMenuModel ()**
Konstruktor a MainMenuModel létrehozásához.
- **QuestionRepository getQuestionRepository ()**
Visszaadja a kérdések tárolóját.

4.13.1 Detailed Description

A főmenü modell osztálya.

Ez az osztály felelős a főmenü adatmodelljének kezeléséért, elsősorban a kérdések tárolójának inicializálásáért és elérhetővé tételéért.

Definition at line 10 of file **MainMenuModel.java**.

4.13.2 Constructor & Destructor Documentation

4.13.2.1 MainMenuModel()

```
millionaire.model.MainMenuModel.MainMenuModel ( )
```

Konstruktor a MainMenuModel létrehozásához.

Inicializálja a kérdések tárolóját.

Definition at line 18 of file **MainMenuModel.java**.

4.13.3 Member Function Documentation

4.13.3.1 `getRepository()`

```
QuestionRepository millionaire.model.MainMenuModel.getRepository ( )
```

Visszaadja a kérdések tárolóját.

Returns

A kérdéseket tároló `QuestionRepository` példány

Definition at line **26** of file **MainMenuModel.java**.

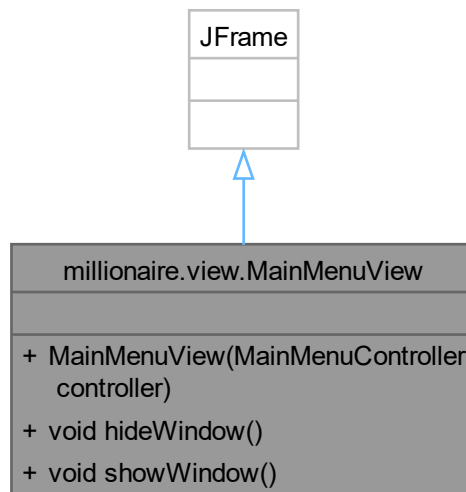
The documentation for this class was generated from the following file:

- `src/main/java/millionaire/model/ MainMenuModel.java`

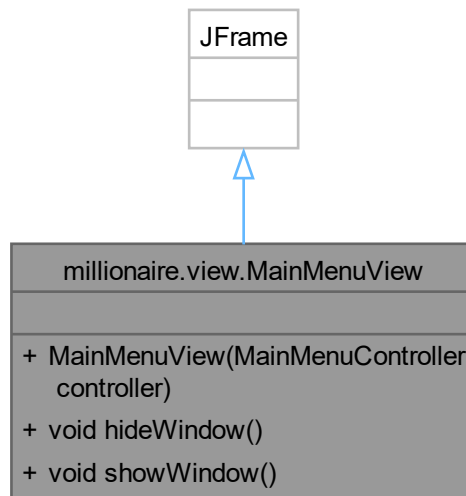
4.14 `millionaire.view.MainMenuView` Class Reference

A játék főmenüjének grafikus felülete.

Inheritance diagram for `millionaire.view.MainMenuView`:



Collaboration diagram for millionaire.view.MainMenuView:



Public Member Functions

- **MainMenuView (MainMenuController controller)**
Konstruktor a MainMenuView létrehozásához.
- void **hideWindow ()**
Elrejti a főmenü ablakát.
- void **showWindow ()**
Megjeleníti a főmenü ablakát.

4.14.1 Detailed Description

A játék főmenüjének grafikus felülete.

Ez az osztály felelős a játék főmenüjének megjelenítéséért és kezeléséért. A főmenü tartalmazza:

- A játék címét
- Az összes főmenü opciót (új játék, kérdés hozzáadása, dicsőséglista, kézikönyv, kilépés)
- Az opciók közötti navigáció kezelését

Definition at line 17 of file **MainMenuView.java**.

4.14.2 Constructor & Destructor Documentation

4.14.2.1 MainMenuView()

```
millionaire.view.MainMenuView.MainMenuView (
    MainMenuController controller )
```

Konstruktor a MainMenuView létrehozásához.

Inicializálja az ablakot és beállítja annak komponenseit.

Parameters

<i>controller</i>	A főmenüt vezérlő controller
-------------------	------------------------------

Definition at line 36 of file **MainMenuView.java**.

4.14.3 Member Function Documentation

4.14.3.1 hideWindow()

```
void millionaire.view.MainMenuView.hideWindow ( )
```

Elrejtí a főmenü ablakát.

Az ablakot láthatatlanná teszi, de nem zárja be. Új játék indításakor használatos.

Definition at line 238 of file **MainMenuView.java**.

4.14.3.2 showWindow()

```
void millionaire.view.MainMenuView.showWindow ( )
```

Megjeleníti a főmenü ablakát.

Az ablakot láthatóvá teszi. A játékból való visszatéréskor használatos.

Definition at line 248 of file **MainMenuView.java**.

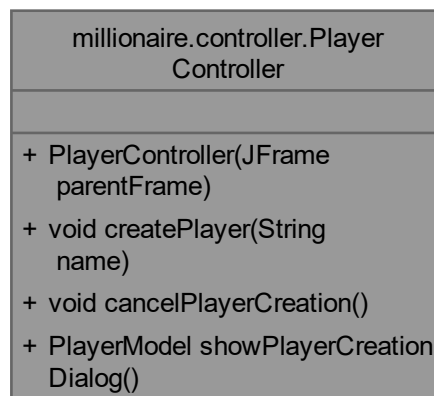
The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **MainMenuView.java**

4.15 millionaire.controller.PlayerController Class Reference

A játékos létrehozását kezelő vezérlő osztály.

Collaboration diagram for millionaire.controller.PlayerController:



Public Member Functions

- **PlayerController** (JFrame parentFrame)
Konstruktor a PlayerController létrehozásához.
- void **createPlayer** (String name)
Létrehoz egy új játékost.
- void **cancelPlayerCreation** ()
Megszakítja a játékos létrehozását.
- **PlayerModel** **showPlayerCreationDialog** ()
Megjeleníti a játékos létrehozási dialógust.

4.15.1 Detailed Description

A játékos létrehozását kezelő vezérlő osztály.

Ez az osztály felelős:

- A játékos létrehozási folyamat vezérléséért
- A játékos nevének validálásáért
- A játékos objektum létrehozásáért

Definition at line 17 of file **PlayerController.java**.

4.15.2 Constructor & Destructor Documentation

4.15.2.1 PlayerController()

```
millionaire.controller.PlayerController.PlayerController (  
    JFrame parentFrame )
```

Konstruktor a PlayerController létrehozásához.

Parameters

<i>parentFrame</i>	A szülő ablak
--------------------	---------------

Definition at line 27 of file **PlayerController.java**.

4.15.3 Member Function Documentation

4.15.3.1 cancelPlayerCreation()

```
void millionaire.controller.PlayerController.cancelPlayerCreation ( )
```

Megszakítja a játékos létrehozását.

Definition at line 55 of file **PlayerController.java**.

4.15.3.2 createPlayer()

```
void millionaire.controller.PlayerController.createPlayer (
    String name )
```

Létrehoz egy új játékost.

Ellenőrzi a név érvényességét, létrehozza a játékos objektumot.

Parameters

<i>name</i>	A játékos neve
-------------	----------------

Definition at line 40 of file **PlayerController.java**.

4.15.3.3 showPlayerCreationDialog()

```
PlayerModel millionaire.controller.PlayerController.showPlayerCreationDialog ( )
```

Megjeleníti a játékos létrehozási dialógust.

Returns

A létrehozott játékos, vagy null ha megszakították

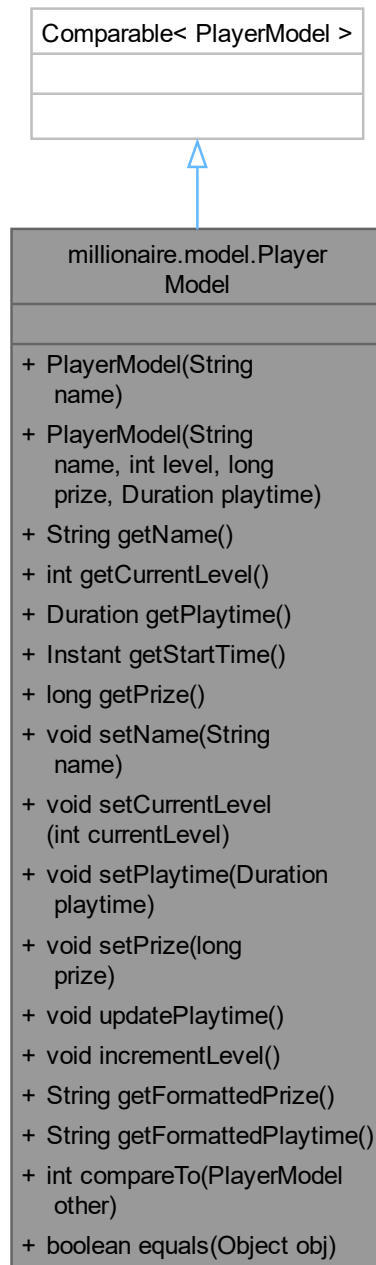
Definition at line 65 of file **PlayerController.java**.

The documentation for this class was generated from the following file:

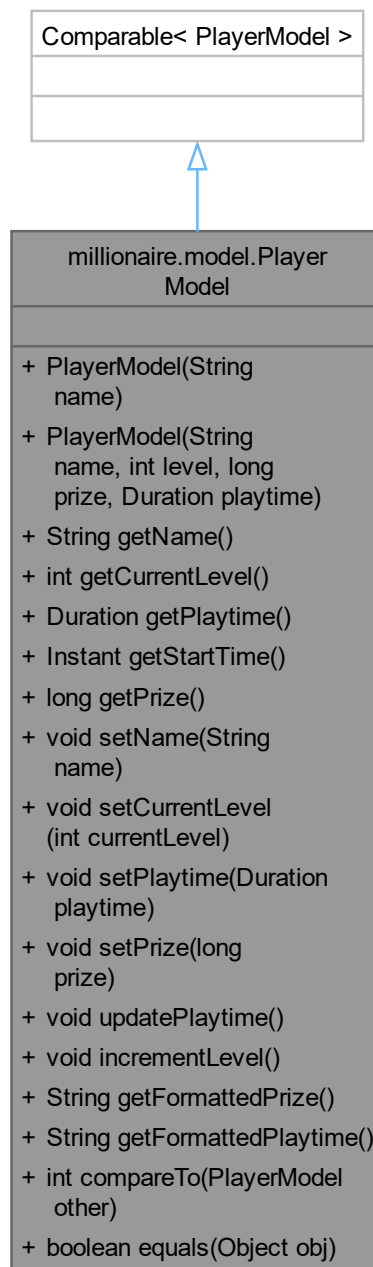
- src/main/java/millionaire/controller/ **PlayerController.java**

4.16 millionaire.model.PlayerModel Class Reference

Inheritance diagram for millionaire.model.PlayerModel:



Collaboration diagram for millionaire.model.PlayerModel:



Public Member Functions

- **PlayerModel** (String name)
Konstruktor új játékos létrehozásához.
- **PlayerModel** (String name, int level, long prize, Duration playtime)
Konstruktor meglévő játékos adatainak betöltéséhez.
- String **getName** ()

- Visszaadja az aktuális játékos nevét.*
- int **getCurrentLevel** ()
Visszaadja az aktuális játékos jelenlegi szintjét (1-15).
- Duration **getPlaytime** ()
Visszaadja az aktuális játékos játékidőjét.
- Instant **getStartTime** ()
Visszaadja az aktuális játékos játékának kezdetének időpontját.
- long **getPrize** ()
Visszaadja az aktuális játékos nyereményét.
- void **setName** (String name)
Beállítja az aktuális játékos nevét.
- void **setCurrentLevel** (int currentLevel)
Beállítja az aktuális játékos jelenlegi szintjét.
- void **setPlaytime** (Duration playtime)
Beállítja az aktuális játékos játékidőjét.
- void **setPrize** (long prize)
Beállítja az aktuális játékos nyereményét.
- void **updatePlaytime** ()
Játékidő frissítése a kezdési idő alapján.
- void **incrementLevel** ()
Növeli az aktuális játékos szintjét eggyel, ha helyesen válaszol egy kérdésre.
- String **getFormattedPrize** ()
Visszaadja az aktuális játékos nyereményét formázva.
- String **getFormattedPlaytime** ()
Játékidő formázott megjelenítése.
- int **compareTo** (**PlayerModel** other)
Játékosok összehasonlítása először a szintjük, majd a játékidőjük alapján, ha a szintjük megegyezik.
- boolean **equals** (Object obj)
Két játékos egyenlőségének vizsgálata a nevük alapján.

4.16.1 Detailed Description

Definition at line 9 of file **PlayerModel.java**.

4.16.2 Constructor & Destructor Documentation

4.16.2.1 PlayerModel() [1/2]

```
millionaire.model.PlayerModel.PlayerModel (
    String name )
```

Konstruktor új játékos létrehozásához.

Parameters

<i>name</i>	A játékos neve
-------------	----------------

Definition at line 21 of file **PlayerModel.java**.

4.16.2.2 PlayerModel() [2/2]

```
millionaire.model.PlayerModel.PlayerModel (
    String name,
    int level,
    long prize,
    Duration playtime )
```

Konstruktor meglévő játékos adatainak betöltéséhez.

Parameters

<i>name</i>	A játékos neve
<i>level</i>	A játékos szintje
<i>prize</i>	A játékos nyereménye
<i>playtime</i>	A játékos játékidője

Definition at line 37 of file **PlayerModel.java**.

4.16.3 Member Function Documentation

4.16.3.1 compareTo()

```
int millionaire.model.PlayerModel.compareTo (
    PlayerModel other )
```

Játékosok összehasonlítása először a szintjük, majd a játékidőjük alapján, ha a szintjük megegyezik.

Parameters

<i>other</i>	A másik játékos, amivel összehasonlítjuk az aktuális játékost.
--------------	--

Returns

Az összehasonlítás eredménye.

Definition at line 155 of file **PlayerModel.java**.

4.16.3.2 equals()

```
boolean millionaire.model.PlayerModel.equals (
    Object obj )
```

Két játékos egyenlőségének vizsgálata a nevük alapján.

Parameters

<i>obj</i>	A másik játékos, amivel összehasonlítjuk az aktuális játékost.
------------	--

Returns

Az egyenlőség eredménye.

Definition at line 171 of file **PlayerModel.java**.

4.16.3.3 getCurrentLevel()

```
int millionaire.model.PlayerModel.getCurrentLevel ( )
```

Visszaadja az aktuális játékos jelenlegi szintjét (1-15).

Returns

A játékos jelenlegi szintje.

Definition at line 57 of file **PlayerModel.java**.

4.16.3.4 getFormattedPlaytime()

```
String millionaire.model.PlayerModel.getFormattedPlaytime ( )
```

Játékidő formázott megjelenítése.

Returns

A játékidő "perc:másodperc" formátumban

Definition at line 138 of file **PlayerModel.java**.

4.16.3.5 getFormattedPrize()

```
String millionaire.model.PlayerModel.getFormattedPrize ( )
```

Visszaadja az aktuális játékos nyereményét formázva.

Returns

A játékos nyereménye formázva.

Definition at line 129 of file **PlayerModel.java**.

4.16.3.6 getName()

```
String millionaire.model.PlayerModel.getName ( )
```

Visszaadja az aktuális játékos nevét.

Returns

A játékos neve.

Definition at line 50 of file **PlayerModel.java**.

4.16.3.7 `getPlaytime()`

```
Duration millionaire.model.PlayerModel.getPlaytime ( )
```

Visszaadja az aktuális játékos játékidéjét.

Returns

A játékos játékidéje.

Definition at line **64** of file **PlayerModel.java**.

4.16.3.8 `getPrize()`

```
long millionaire.model.PlayerModel.getPrize ( )
```

Visszaadja az aktuális játékos nyereményét.

Returns

A játékos nyereménye.

Definition at line **78** of file **PlayerModel.java**.

4.16.3.9 `getStartTime()`

```
Instant millionaire.model.PlayerModel.getStartTime ( )
```

Visszaadja az aktuális játékos játékának kezdetének időpontját.

Returns

A játék kezdetének időpontja.

Definition at line **71** of file **PlayerModel.java**.

4.16.3.10 `incrementLevel()`

```
void millionaire.model.PlayerModel.incrementLevel ( )
```

Növeli az aktuális játékos szintjét eggyel, ha helyesen válaszol egy kérdésre.

Definition at line **122** of file **PlayerModel.java**.

4.16.3.11 `setCurrentLevel()`

```
void millionaire.model.PlayerModel.setCurrentLevel (
    int currentLevel )
```

Beállítja az aktuális játékos jelenlegi szintjét.

Parameters

<i>currentLevel</i>	A játékos jelenlegi szintje.
---------------------	------------------------------

Definition at line **92** of file **PlayerModel.java**.

4.16.3.12 setName()

```
void millionaire.model.PlayerModel.setName (  
    String name )
```

Beállítja az aktuális játékos nevét.

Returns

A játékos neve.

Definition at line **85** of file **PlayerModel.java**.

4.16.3.13 setPlaytime()

```
void millionaire.model.PlayerModel.setPlaytime (  
    Duration playtime )
```

Beállítja az aktuális játékos játékidéjét.

Parameters

<i>playtime</i>	A játékos játékidéje.
-----------------	-----------------------

Definition at line **99** of file **PlayerModel.java**.

4.16.3.14 setPrize()

```
void millionaire.model.PlayerModel.setPrize (  
    long prize )
```

Beállítja az aktuális játékos nyereményét.

Parameters

<i>prize</i>	A játékos nyereménye.
--------------	-----------------------

Definition at line **105** of file **PlayerModel.java**.

4.16.3.15 updatePlaytime()

```
void millionaire.model.PlayerModel.updatePlaytime ( )
```

Játékidő frissítése a kezdési idő alapján.

Definition at line **111** of file **PlayerModel.java**.

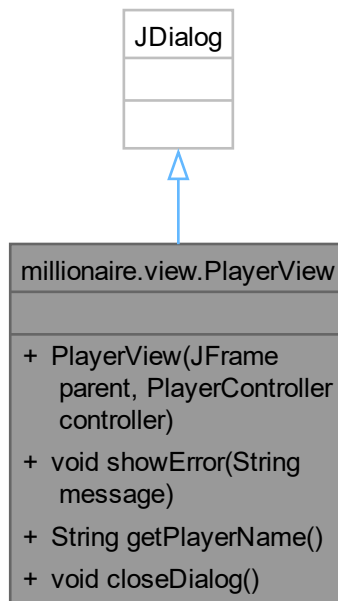
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **PlayerModel.java**

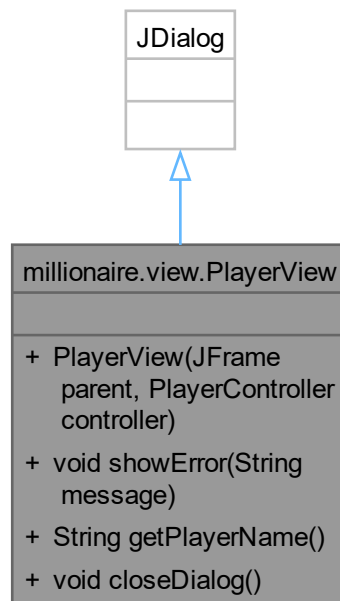
4.17 millionaire.view.PlayerView Class Reference

Játékos létrehozására szolgáló dialógusablak.

Inheritance diagram for millionaire.view.PlayerView:



Collaboration diagram for millionaire.view.PlayerView:



Public Member Functions

- **PlayerView** (JFrame parent, **PlayerController** controller)
Konstruktor a PlayerView létrehozásához.
- void **showError** (String message)
Hibaüzenet megjelenítése.
- String **getPlayerName** ()
A játékos nevének lekérdezése.
- void **closeDialog** ()
Bezárja a dialógusablakot.

4.17.1 Detailed Description

Játékos létrehozására szolgáló dialógusablak.

Ez az osztály felelős az új játékos létrehozásához szükséges felhasználói felület megjelenítéséért és kezeléséért. A felület tartalmazza:

- Beviteli mezőt a játékos nevének megadásához
- OK és Mégse gombokat a művelet végrehajtásához vagy megszakításához

Definition at line 16 of file **PlayerView.java**.

4.17.2 Constructor & Destructor Documentation

4.17.2.1 PlayerView()

```
millionaire.view.PlayerView.PlayerView (
    JFrame parent,
    PlayerController controller )
```

Konstruktor a PlayerView létrehozásához.

Létrehoz egy modális dialógusablakot a játékos nevének bekérésére.

Parameters

<i>parent</i>	A szülő ablak
<i>controller</i>	A játékos létrehozását kezelő controller

Definition at line 35 of file **PlayerView.java**.

4.17.3 Member Function Documentation

4.17.3.1 closeDialog()

```
void millionaire.view.PlayerView.closeDialog ( )
```

Bezárja a dialógusablakot.

Felszabadítja az ablakhoz tartozó erőforrásokat és eltávolítja az ablakot a képernyőről.

Definition at line 137 of file **PlayerView.java**.

4.17.3.2 getPlayerName()

```
String millionaire.view.PlayerView.getPlayerName ( )
```

A játékos nevének lekérdezése.

Visszaadja a beviteli mezőbe írt nevet, eltávolítva az esetleges felesleges szóközöket.

Returns

A megadott játékosnév

Definition at line 127 of file **PlayerView.java**.

4.17.3.3 showError()

```
void millionaire.view.PlayerView.showError (
    String message )
```

Hibaüzenet megjelenítése.

Felugró ablakban jeleníti meg a hibaüzenetet.

Parameters

<i>message</i>	A megjelenítendő hibaüzenet
----------------	-----------------------------

Definition at line **115** of file **PlayerView.java**.

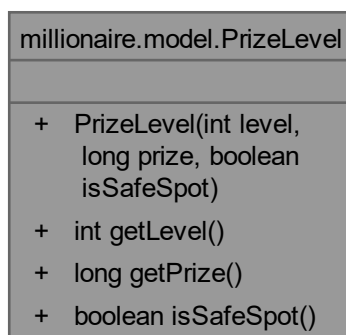
The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **PlayerView.java**

4.18 millionaire.model.PrizeLevel Class Reference

Nyereményszintet reprezentáló osztály.

Collaboration diagram for millionaire.model.PrizeLevel:



Public Member Functions

- **PrizeLevel** (int level, long prize, boolean isSafeSpot)
Konstruktor egy nyereményszint létrehozásához.
- int **getLevel** ()
Megadja a játékos által elért szintet.
- long **getPrize** ()
Megadja a játékos által elért nyereményt.
- boolean **isSafeSpot** ()
Megadja, hogy a játékos által elért szint szakaszhatároló-e.

4.18.1 Detailed Description

Nyereményszintet reprezentáló osztály.

Ez az osztály egy nyereményszintet reprezentál a játékban. Minden szinthez tartozik:

- egy sorszám (1-15)
- egy nyereményösszeg
- egy jelző, hogy szakaszhatároló-e (garantált nyereménnyel járó szint)

Definition at line 13 of file **PrizeLevel.java**.

4.18.2 Constructor & Destructor Documentation

4.18.2.1 PrizeLevel()

```
millionaire.model.PrizeLevel.PrizeLevel (
    int level,
    long prize,
    boolean isSafeSpot )
```

Konstruktor egy nyereményszint létrehozásához.

Parameters

<i>level</i>	A szint sorszáma (1-15)
<i>prize</i>	A szinthez tartozó nyereményösszeg
<i>isSafeSpot</i>	Igaz, ha ez egy szakaszhatároló szint (garantált nyereménnyel)

Definition at line 25 of file **PrizeLevel.java**.

4.18.3 Member Function Documentation

4.18.3.1 getLevel()

```
int millionaire.model.PrizeLevel.getLevel ( )
```

Megadja a játékos által elért szintet.

Returns

Az elért szint.

Definition at line 35 of file **PrizeLevel.java**.

4.18.3.2 getPrize()

```
long millionaire.model.PrizeLevel.getPrize ( )
```

Megadja a játékos által elért nyereményt.

Returns

Az elért nyeremény.

Definition at line 41 of file **PrizeLevel.java**.

4.18.3.3 isSafeSpot()

```
boolean millionaire.model.PrizeLevel.isSafeSpot ( )
```

Megadja, hogy a játékos által elért szint szakaszhatároló-e.

Returns

Igaz, ha a szint szakaszhatároló (garantált nyereménnyel), egyébként hamis.

Definition at line 47 of file **PrizeLevel.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **PrizeLevel.java**

4.19 millionaire.model.Question Class Reference

Egy kérdést reprezentáló osztály.

Collaboration diagram for millionaire.model.Question:

millionaire.model.Question
+ int getDifficulty() + String getQuestion() + String getOptionA() + String getOptionB() + String getOptionC() + String getOptionD() + String getCorrectAnswer() + String getCategory()

Classes

- class **QuestionBuilder**

Builder osztály a Question példányok létrehozásához.

Public Member Functions

- int **getDifficulty** ()
A kérdéshez tartozó nehézségi szintet adja vissza.
- String **getQuestion** ()
A kérdés szövegét adja vissza.
- String **getOptionA** ()
A kérdéshez tartozó 'A' válaszlehetőség szövegét adja vissza.
- String **getOptionB** ()
A kérdéshez tartozó 'B' válaszlehetőség szövegét adja vissza.
- String **getOptionC** ()
A kérdéshez tartozó 'C' válaszlehetőség szövegét adja vissza.
- String **getOptionD** ()
A kérdéshez tartozó 'D' válaszlehetőség szövegét adja vissza.
- String **getCorrectAnswer** ()
A kérdéshez tartozó helyes válasz betűjelét adja vissza.
- String **getCategory** ()
A kérdés kategóriáját adja vissza.

4.19.1 Detailed Description

Egy kérdést reprezentáló osztály.

Ez az osztály egy játékbeli kérdést reprezentál, annak minden tulajdonságával:

- nehézségi szint (1-15)
- kérdés szövege
- négy válaszlehetőség (A, B, C, D)
- helyes válasz betűjele
- kategória

Az osztály immutable (megváltoztathatatlan), és Builder pattern segítségével példányosítható.

Definition at line 16 of file **Question.java**.

4.19.2 Member Function Documentation

4.19.2.1 getCategory()

```
String millionaire.model.Question.getCategory ( )
```

A kérdés kategóriáját adja vissza.

Returns

A kérdés kategóriája.

Definition at line 96 of file **Question.java**.

4.19.2.2 getCorrectAnswer()

```
String millionaire.model.Question.getCorrectAnswer ( )
```

A kérdéshez tartozó helyes válasz betűjelét adja vissza.

Returns

A helyes válasz betűjele.

Definition at line 89 of file **Question.java**.

4.19.2.3 getDifficulty()

```
int millionaire.model.Question.getDifficulty ( )
```

A kérdéshez tartozó nehézségi szintet adja vissza.

Returns

A nehézségi szint.

Definition at line 47 of file **Question.java**.

4.19.2.4 getOptionA()

```
String millionaire.model.Question.getOptionA ( )
```

A kérdéshez tartozó 'A' válaszlehetőség szövegét adja vissza.

Returns

Az 'A' válaszlehetőség szövege.

Definition at line 61 of file **Question.java**.

4.19.2.5 getOptionB()

```
String millionaire.model.Question.getOptionB ( )
```

A kérdéshez tartozó 'B' válaszlehetőség szövegét adja vissza.

Returns

Az 'B' válaszlehetőség szövege.

Definition at line 68 of file **Question.java**.

4.19.2.6 `getOptionC()`

```
String millionaire.model.Question.getOptionC ( )
```

A kérdéshez tartozó 'C' válaszlehetőség szövegét adja vissza.

Returns

Az 'C' válaszlehetőség szövege.

Definition at line **75** of file **Question.java**.

4.19.2.7 `getOptionD()`

```
String millionaire.model.Question.getOptionD ( )
```

A kérdéshez tartozó 'D' válaszlehetőség szövegét adja vissza.

Returns

Az 'D' válaszlehetőség szövege.

Definition at line **82** of file **Question.java**.

4.19.2.8 `getQuestion()`

```
String millionaire.model.Question.getQuestion ( )
```

A kérdés szövegét adja vissza.

Returns

A kérdés szövege.

Definition at line **54** of file **Question.java**.

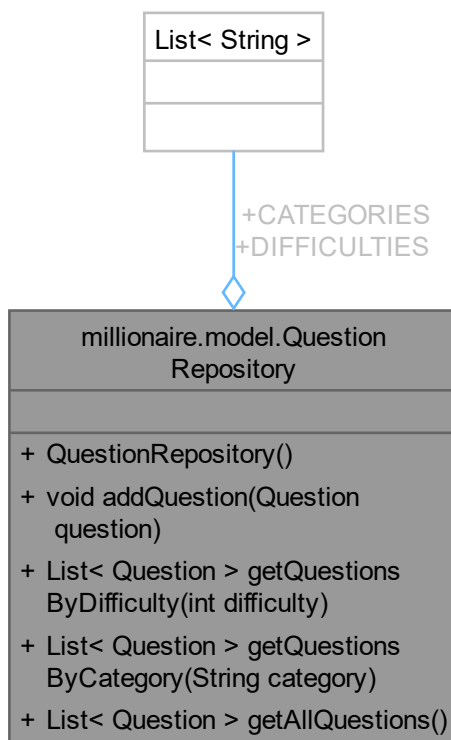
The documentation for this class was generated from the following file:

- `src/main/java/millionaire/model/Question.java`

4.20 millionaire.model.QuestionRepository Class Reference

Kérdések perzisztens tárolását végző osztály.

Collaboration diagram for millionaire.model.QuestionRepository:



Public Member Functions

- **QuestionRepository ()**
Konstruktor a QuestionRepository létrehozásához.
- void **addQuestion (Question question)**
Hozzáad egy új kérdést a gyűjteményhez.
- List< **Question** > **getQuestionsByDifficulty** (int difficulty)
Lekérdezi az adott nehézségi szintű kérdéseket.
- List< **Question** > **getQuestionsByCategory** (String category)
Lekérdezi az adott kategóriába tartozó kérdéseket.
- List< **Question** > **getAllQuestions ()**
Visszaadja az összes tárolt kérdést.

Static Public Attributes

- static final List< String > **CATEGORIES**
Elérhető kategóriák listája.
- static final List< String > **DIFFICULTIES**
Elérhető nehézségi szintek listája.

4.20.1 Detailed Description

Kérdések perzisztens tárolását végző osztály.

Ez az osztály felelős:

- A kérdések JSON fájlból való betöltéséért
- A kérdések JSON fájlba mentéséért
- A kérdések memóriában való tárolásáért
- A kérdések szűréséért különböző szempontok alapján

Definition at line 24 of file **QuestionRepository.java**.

4.20.2 Constructor & Destructor Documentation

4.20.2.1 QuestionRepository()

```
millionaire.model.QuestionRepository.QuestionRepository ( )
```

Konstruktor a QuestionRepository létrehozásához.

Inicializálja az adattagokat és betölti a kérdéseket a JSON fájlból.

Definition at line 47 of file **QuestionRepository.java**.

4.20.3 Member Function Documentation

4.20.3.1 addQuestion()

```
void millionaire.model.QuestionRepository.addQuestion (
    Question question )
```

Hozzáad egy új kérdést a gyűjteményhez.

Parameters

<i>question</i>	A hozzáadandó kérdés
-----------------	----------------------

Definition at line 128 of file **QuestionRepository.java**.

4.20.3.2 getAllQuestions()

```
List< Question > millionaire.model.QuestionRepository.getAllQuestions ( )
```

Visszaadja az összes tárolt kérdést.

Returns

Az összes kérdés másolata egy új listában

Definition at line 160 of file **QuestionRepository.java**.

4.20.3.3 getQuestionsByCategory()

```
List< Question > millionaire.model.QuestionRepository.getQuestionsByCategory (
    String category )
```

Lekérdezi az adott kategóriába tartozó kérdéseket.

Parameters

<i>category</i>	A kívánt kategória neve
-----------------	-------------------------

Returns

A kategóriához tartozó kérdések listája

Definition at line 151 of file **QuestionRepository.java**.

4.20.3.4 getQuestionsByDifficulty()

```
List< Question > millionaire.model.QuestionRepository.getQuestionsByDifficulty (
    int difficulty )
```

Lekérdezi az adott nehézségi szintű kérdéseket.

Parameters

<i>difficulty</i>	A kívánt nehézségi szint (1-15)
-------------------	---------------------------------

Returns

A megadott nehézségű kérdések listája

Definition at line 139 of file **QuestionRepository.java**.

4.20.4 Member Data Documentation

4.20.4.1 CATEGORIES

```
final List<String> millionaire.model.QuestionRepository.CATEGORIES [static]
```

Initial value:

```
= Arrays.asList(
    "TÖRTÉNELEM", "SPORT", "FÖLDRAJZ", "IRODALOM", "TUDOMÁNY", "ZENE", "FILM", "KONYHA", "ÁLTALÁNOS"
)
```

Elérhető kategóriák listája.

Definition at line 31 of file **QuestionRepository.java**.

4.20.4.2 DIFFICULTIES

```
final List<String> millionaire.model.QuestionRepository.DIFFICULTIES [static]
```

Initial value:

```
= Arrays.asList(  
    "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15"  
)
```

Elérhető nehézségi szintek listája.

Definition at line 37 of file **QuestionRepository.java**.

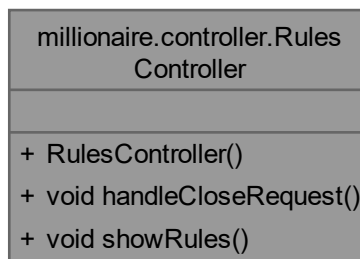
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **QuestionRepository.java**

4.21 millionaire.controller.RulesController Class Reference

A játékszabályokat kezelő vezérlő osztály.

Collaboration diagram for millionaire.controller.RulesController:



Public Member Functions

- **RulesController ()**
A játékszabályok vezérlőjének konstruktora.
- void **handleCloseRequest ()**
Kezeli az ablak bezárásának kérését.
- void **showRules ()**
Megjeleníti a játékszabályok ablakot.

4.21.1 Detailed Description

A játékszabályokat kezelő vezérlő osztály.

Ez az osztály felelős:

- A játékszabályok betöltéséért
- A szabályok megjelenítéséért
- A szabályok ablak kezeléséért

Definition at line 15 of file **RulesController.java**.

4.21.2 Constructor & Destructor Documentation

4.21.2.1 RulesController()

```
millionaire.controller.RulesController.RulesController ( )
```

A játékszabályok vezérlőjének konstruktora.

Létrehozza a játékszabályok modelljét és nézetét, majd inicializálja a nézetet.

Definition at line 25 of file **RulesController.java**.

4.21.3 Member Function Documentation

4.21.3.1 handleCloseRequest()

```
void millionaire.controller.RulesController.handleCloseRequest ( )
```

Kezeli az ablak bezárásának kérését.

Meghívja a nézet closeWindow() metódusát az ablak bezárásához és az erőforrások felszabadításához.

Definition at line 53 of file **RulesController.java**.

4.21.3.2 showRules()

```
void millionaire.controller.RulesController.showRules ( )
```

Megjeleníti a játékszabályok ablakot.

Láthatóvá teszi a játékszabályokat tartalmazó ablakot a felhasználói felületen.

Definition at line 63 of file **RulesController.java**.

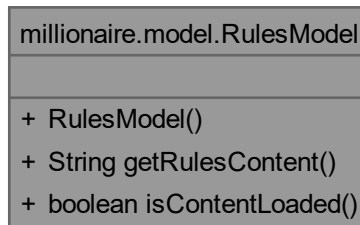
The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **RulesController.java**

4.22 millionaire.model.RulesModel Class Reference

Játékszabályok kezelését végző osztály.

Collaboration diagram for millionaire.model.RulesModel:



Public Member Functions

- **RulesModel ()**
Konstruktor a RulesModel létrehozásához.
- String **getRulesContent ()**
Visszaadja a betöltött játékszabályok szövegét.
- boolean **isContentLoaded ()**
Ellenőrzi a játékszabályok betöltésének sikerességét.

4.22.1 Detailed Description

Játékszabályok kezelését végző osztály.

Ez az osztály felelős:

- A játékszabályok betöltéséért text fájlból
- A szabályok tartalmának tárolásáért
- A betöltés sikerességének ellenőrzéséért

Definition at line 18 of file **RulesModel.java**.

4.22.2 Constructor & Destructor Documentation

4.22.2.1 RulesModel()

```
millionaire.model.RulesModel.RulesModel ( )
```

Konstruktor a RulesModel létrehozásához.

Létrehozáskor azonnal megkísérli betölteni a játékszabályokat.

Definition at line 30 of file **RulesModel.java**.

4.22.3 Member Function Documentation

4.22.3.1 getRulesContent()

```
String millionaire.model.RulesModel.getRulesContent ( )
```

Visszaadja a betöltött játékszabályok szövegét.

Returns

A szabályok szövege, vagy hibaüzenet ha a betöltés sikertelen volt

Definition at line **53** of file **RulesModel.java**.

4.22.3.2 isContentLoaded()

```
boolean millionaire.model.RulesModel.isContentLoaded ( )
```

Ellenőrzi a játékszabályok betöltésének sikerességét.

Returns

true ha a szabályok sikeresen betöltődtek, false ha hiba történt

Definition at line **61** of file **RulesModel.java**.

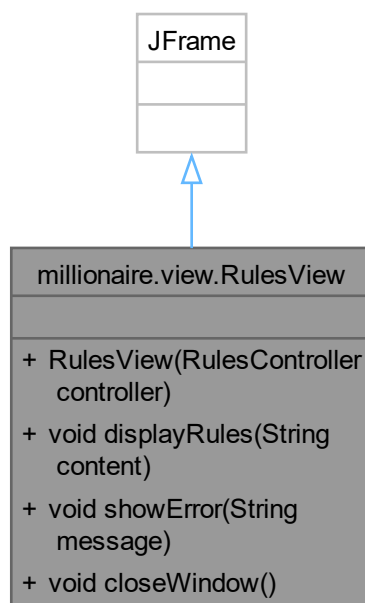
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **RulesModel.java**

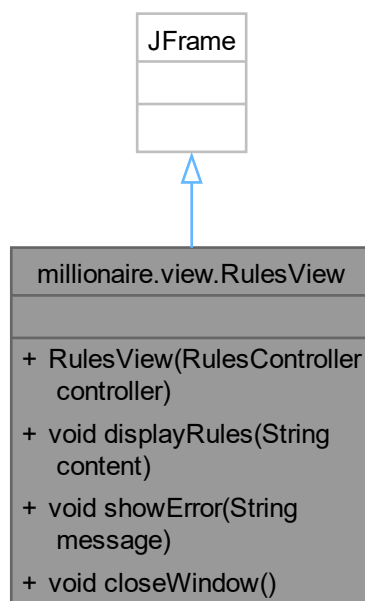
4.23 millionaire.view.RulesView Class Reference

A játékszabályok megjelenítésére szolgáló ablak.

Inheritance diagram for `millionaire.view.RulesView`:



Collaboration diagram for `millionaire.view.RulesView`:



Public Member Functions

- **RulesView** (**RulesController** controller)
Konstruktor a RulesView létrehozásához.
- void **displayRules** (String content)
Beállítja a játékszabályok szövegét.
- void **showError** (String message)
Hibaüzenet megjelenítése.
- void **closeWindow** ()
Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

4.23.1 Detailed Description

A játékszabályok megjelenítésére szolgáló ablak.

Ez az osztály felelős a játékszabályok megjelenítéséért egy görgethető szövegterületen. Az ablak tartalmazza:

- A játékszabályok szövegét
- Görgetősávot a hosszabb tartalom kezeléséhez
- Bezárás gombot

Definition at line 17 of file **RulesView.java**.

4.23.2 Constructor & Destructor Documentation

4.23.2.1 RulesView()

```
millionaire.view.RulesView.RulesView (
    RulesController controller )
```

Konstruktor a RulesView létrehozásához.

Parameters

<i>controller</i>	A szabályokat kezelő controller
-------------------	---------------------------------

Definition at line 33 of file **RulesView.java**.

4.23.3 Member Function Documentation

4.23.3.1 closeWindow()

```
void millionaire.view.RulesView.closeWindow ( )
```

Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

Definition at line 175 of file **RulesView.java**.

4.23.3.2 displayRules()

```
void millionaire.view.RulesView.displayRules (
    String content )
```

Beállítja a játékszabályok szövegét.

Parameters

<i>content</i>	A megjelenítendő szabályok szövege
----------------	------------------------------------

Definition at line **156** of file **RulesView.java**.

4.23.3.3 showError()

```
void millionaire.view.RulesView.showError (
    String message )
```

Hibaüzenet megjelenítése.

Parameters

<i>message</i>	A megjelenítendő hibaüzenet
----------------	-----------------------------

Definition at line **167** of file **RulesView.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **RulesView.java**

4.24 millionaire.controller.ScoreboardController Class Reference

A dicsőséglista kezelését végző vezérlő osztály.

Collaboration diagram for millionaire.controller.ScoreboardController:

millionaire.controller.Scoreboard Controller
+ ScoreboardController() + void addScore(PlayerModel player) + void showScoreboard() + void handleClose()

Public Member Functions

- **ScoreboardController** ()
Konstruktor a ScoreboardController létrehozásához.
- void **addScore** (**PlayerModel** player)
Új eredmény hozzáadása a dicsőséglistához.
- void **showScoreboard** ()
Megjeleníti a dicsőséglista ablakot.
- void **handleClose** ()
Kezeli az ablak bezárásának kérését.

4.24.1 Detailed Description

A dicsőséglista kezelését végző vezérlő osztály.

Ez az osztály felelős:

- A játékosok eredményeinek kezeléséért és megjelenítéséért
- Új eredmények hozzáadásáért a dicsőséglistához
- A dicsőséglista frissítéséért és megjelenítéséért
- A dicsőséglista ablak életciklusának kezeléséért

Definition at line 17 of file **ScoreboardController.java**.

4.24.2 Constructor & Destructor Documentation

4.24.2.1 ScoreboardController()

```
millionaire.controller.ScoreboardController.ScoreboardController ( )
```

Konstruktor a ScoreboardController létrehozásához.

Inicializálja a modellt és a nézetet, majd meghívja a refreshScoreboard() metódust a kezdeti megjelenítéshez.

Definition at line 29 of file **ScoreboardController.java**.

4.24.3 Member Function Documentation

4.24.3.1 addScore()

```
void millionaire.controller.ScoreboardController.addScore (
    PlayerModel player )
```

Új eredmény hozzáadása a dicsőséglistához.

A metódus:

1. Hozzáadja az új játékos eredményét a modellhez
2. Frissíti a dicsőséglista megjelenítését

Parameters

<i>player</i>	Az új eredménnyel rendelkező játékos modellje
---------------	---

Definition at line 44 of file **ScoreboardController.java**.

4.24.3.2 handleClose()

```
void millionaire.controller.ScoreboardController.handleClose ( )
```

Kezeli az ablak bezárásának kérését.

Meghívja a nézet closeWindow() metódusát az ablak bezárásához és az erőforrások felszabadításához.

Definition at line 78 of file **ScoreboardController.java**.

4.24.3.3 showScoreboard()

```
void millionaire.controller.ScoreboardController.showScoreboard ( )
```

Megjeleníti a dicsőséglista ablakot.

1. Frissíti a dicsőséglistát a legújabb adatokkal
2. Láthatóvá teszi a dicsőséglistát tartalmazó ablakot

Definition at line 67 of file **ScoreboardController.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/controller/ **ScoreboardController.java**

4.25 millionaire.model.ScoreboardModel Class Reference

Dicsőséglista kezelését végző osztály.

Collaboration diagram for millionaire.model.ScoreboardModel:



Public Member Functions

- **ScoreboardModel ()**
Konstruktor a ScoreboardModel létrehozásához.
- void **addScore (PlayerModel newPlayer)**
Új eredmény hozzáadása vagy meglévő frissítése.
- List< **PlayerModel** > **getPlayers ()**
Játékosok listájának lekérdezése.

4.25.1 Detailed Description

Dicsőségtábla kezelését végző osztály.

Ez az osztály felelős:

- A játékosok eredményeinek betöltéséért JSON fájlból
- Az eredmények perzisztens tárolásáért
- A játékosok rangsorolásáért
- Új eredmények hozzáadásáért és frissítéséért

Definition at line 24 of file **ScoreboardModel.java**.

4.25.2 Constructor & Destructor Documentation

4.25.2.1 ScoreboardModel()

```
millionaire.model.ScoreboardModel.ScoreboardModel ( )
```

Konstruktor a ScoreboardModel létrehozásához.

Inicializálja a listát és betölti a korábbi eredményeket.

Definition at line 39 of file **ScoreboardModel.java**.

4.25.3 Member Function Documentation

4.25.3.1 addScore()

```
void millionaire.model.ScoreboardModel.addScore (
    PlayerModel newPlayer )
```

Új eredmény hozzáadása vagy meglévő frissítése.

Ha a játékos már szerepel a listában és az új eredménye jobb, akkor frissíti a rekordját. Ha még nem szerepelt, vagy rosszabb az új eredménye, akkor az új eredményt menti el.

Parameters

<i>newPlayer</i>	Az új játékos eredménye
------------------	-------------------------

Definition at line **109** of file **ScoreboardModel.java**.

4.25.3.2 `getPlayers()`

```
List< PlayerModel > millionaire.model.ScoreboardModel.getPlayers ( )
```

Játékosok listájának lekérdezése.

Returns

A játékosok listájának másolata

Definition at line **160** of file **ScoreboardModel.java**.

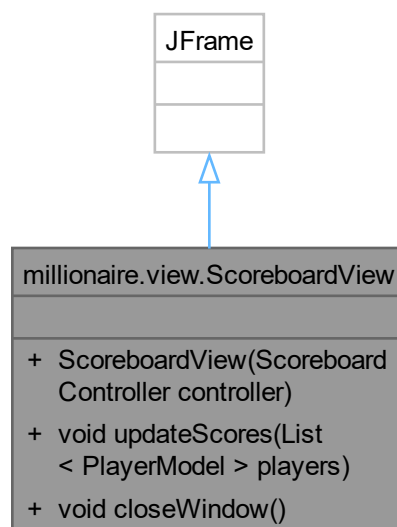
The documentation for this class was generated from the following file:

- `src/main/java/millionaire/model/ ScoreboardModel.java`

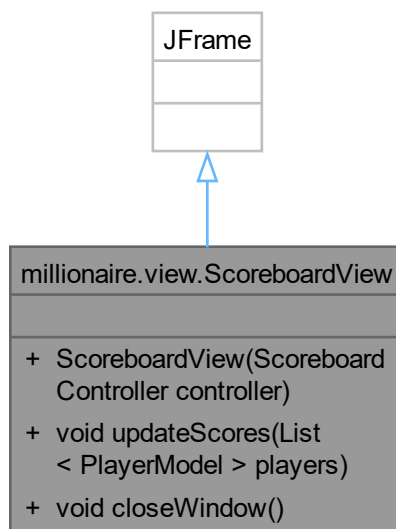
4.26 `millionaire.view.ScoreboardView` Class Reference

A dicsőséglista megjelenítésére szolgáló ablak.

Inheritance diagram for `millionaire.view.ScoreboardView`:



Collaboration diagram for millionaire.view.ScoreboardView:



Public Member Functions

- **ScoreboardView** (**ScoreboardController** controller)
Konstruktor a ScoreboardView létrehozásához.
- void **updateScores** (List< **PlayerModel** > players)
Frissíti a táblázat tartalmát.
- void **closeWindow** ()
Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

4.26.1 Detailed Description

A dicsőséglista megjelenítésére szolgáló ablak.

Ez az osztály felelős a játékosok eredményeinek táblázatos megjelenítéséért. Az ablak tartalmazza:

- A játékosok eredményeit tartalmazó táblázatot
- Görgetősávot a hosszabb lista kezeléséhez
- Bezárás gombot

Definition at line 20 of file **ScoreboardView.java**.

4.26.2 Constructor & Destructor Documentation

4.26.2.1 ScoreboardView()

```
millionaire.view.ScoreboardView.ScoreboardView (
    ScoreboardController controller )
```

Konstruktor a ScoreboardView létrehozásához.

Inicializálja a táblázatot és beállítja annak modelljét.

Parameters

<i>controller</i>	A dicsőséglistát kezelő controller
-------------------	------------------------------------

Definition at line 40 of file **ScoreboardView.java**.

4.26.3 Member Function Documentation

4.26.3.1 closeWindow()

```
void millionaire.view.ScoreboardView.closeWindow ( )
```

Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

Definition at line 208 of file **ScoreboardView.java**.

4.26.3.2 updateScores()

```
void millionaire.view.ScoreboardView.updateScores (
    List< PlayerModel > players )
```

Frissíti a táblázat tartalmát.

Törli a korábbi adatokat és megjeleníti az új eredményeket.

Parameters

<i>players</i>	A megjelenítendő játékosok listája
----------------	------------------------------------

Definition at line 190 of file **ScoreboardView.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **ScoreboardView.java**

4.27 millionaire.controller.TimerController Class Reference

Az időmérő vezérlője.

Collaboration diagram for millionaire.controller.TimerController:

millionaire.controller.Timer Controller
+ TimerController() + void startTimer() + void stopTimer() + TimerView getView() + Duration getElapsedTime()

Public Member Functions

- **TimerController ()**
Konstruktor a vezérlő létrehozásához.
- void **startTimer ()**
Időmérő indítása.
- void **stopTimer ()**
Időmérő megállítása.
- **TimerView getView ()**
Visszaadja az időmérő nézetét.
- Duration **getElapsedTime ()**
Visszaadja az eltelt időt.

4.27.1 Detailed Description

Az időmérő vezérlője.

Ez az osztály felelős:

- Az időmérő működésének vezérléséért
- A modell és view közötti kommunikációért

Definition at line 16 of file **TimerController.java**.

4.27.2 Constructor & Destructor Documentation

4.27.2.1 TimerController()

```
millionaire.controller.TimerController.TimerController ( )
```

Konstruktor a vezérlő létrehozásához.

Definition at line 24 of file **TimerController.java**.

4.27.3 Member Function Documentation

4.27.3.1 getElapsedTime()

`Duration millionaire.controller.TimerController.getElapsedTime ( )`

Visszaadja az eltelt időt.

Returns

Az eltelt idő Duration objektumként

Definition at line **68** of file **TimerController.java**.

4.27.3.2 getView()

`TimerView millionaire.controller.TimerController.getView ( )`

Visszaadja az időmérő nézetét.

Returns

Az időmérő nézete

Definition at line **60** of file **TimerController.java**.

4.27.3.3 startTimer()

`void millionaire.controller.TimerController.startTimer ( )`

Időmérő indítása.

Definition at line **43** of file **TimerController.java**.

4.27.3.4 stopTimer()

`void millionaire.controller.TimerController.stopTimer ( )`

Időmérő megállítása.

Definition at line **51** of file **TimerController.java**.

The documentation for this class was generated from the following file:

- `src/main/java/millionaire/controller/TimerController.java`

4.28 millionaire.model.TimerModel Class Reference

Az időmérő adatmodellje.

Collaboration diagram for millionaire.model.TimerModel:

millionaire.model.TimerModel
+ TimerModel() + void updateElapsedTime() + void start() + void stop() + Duration getElapsedTime() + Instant getStartTime()

Public Member Functions

- **TimerModel ()**
Konstruktor az időmérő létrehozásához.
- void **updateElapsedTime ()**
Frissíti az eltelt időt.
- void **start ()**
Elindítja az időmérőt.
- void **stop ()**
Megállítja az időmérőt.
- Duration **getElapsedTime ()**
Visszaadja az eltelt időt.
- Instant **getStartTime ()**
Visszaadja a kezdési időpontot.

4.28.1 Detailed Description

Az időmérő adatmodellje.

Ez az osztály felelős:

- A játékidő tárolásáért
- Az időmérés állapotának kezeléséért

Definition at line **14** of file **TimerModel.java**.

4.28.2 Constructor & Destructor Documentation

4.28.2.1 TimerModel()

```
millionaire.model.TimerModel.TimerModel ( )
```

Konstruktor az időmérő létrehozásához.

Definition at line 22 of file **TimerModel.java**.

4.28.3 Member Function Documentation

4.28.3.1 getElapsedTime()

```
Duration millionaire.model.TimerModel.getElapsedTime ( )
```

Visszaadja az eltelt időt.

Returns

Az eltelt idő Duration objektumként

Definition at line 55 of file **TimerModel.java**.

4.28.3.2 getStartTime()

```
Instant millionaire.model.TimerModel.getStartTime ( )
```

Visszaadja a kezdési időpontot.

Returns

A kezdési időpont Instant objektumként

Definition at line 63 of file **TimerModel.java**.

4.28.3.3 start()

```
void millionaire.model.TimerModel.start ( )
```

Elindítja az időmérőt.

Definition at line 40 of file **TimerModel.java**.

4.28.3.4 stop()

```
void millionaire.model.TimerModel.stop ( )
```

Megállítja az időmérőt.

Definition at line 47 of file **TimerModel.java**.

4.28.3.5 updateElapsedTime()

```
void millionaire.model.TimerModel.updateElapsedTime ( )
```

Frissíti az eltelt időt.

Definition at line **31** of file **TimerModel.java**.

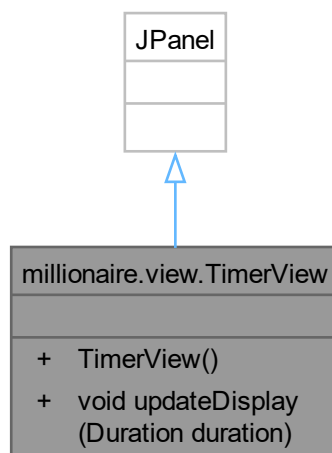
The documentation for this class was generated from the following file:

- src/main/java/millionaire/model/ **TimerModel.java**

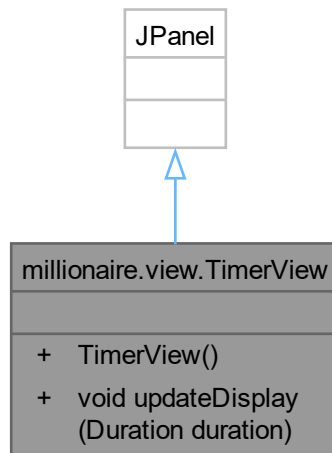
4.29 millionaire.view.TimerView Class Reference

Az időmérő megjelenítése.

Inheritance diagram for millionaire.view.TimerView:



Collaboration diagram for millionaire.view.TimerView:



Public Member Functions

- **TimerView ()**
Konstruktor a nézet létrehozásához.
- void **updateDisplay** (Duration duration)
Az idő kijelzésének frissítése.

4.29.1 Detailed Description

Az időmérő megjelenítése.

Ez az osztály felelős:

- Az időmérő grafikus megjelenítéséért
- Az idő formázott kijelzéséért

Definition at line 15 of file **TimerView.java**.

4.29.2 Constructor & Destructor Documentation

4.29.2.1 TimerView()

```
millionaire.view.TimerView.TimerView ( )
```

Konstruktor a nézet létrehozásához.

Definition at line 24 of file **TimerView.java**.

4.29.3 Member Function Documentation

4.29.3.1 updateDisplay()

```
void millionaire.view.TimerView.updateDisplay (
    Duration duration )
```

Az idő kijelzésének frissítése.

Parameters

<i>duration</i>	Az eltelt idő
-----------------	---------------

Definition at line **41** of file **TimerView.java**.

The documentation for this class was generated from the following file:

- src/main/java/millionaire/view/ **TimerView.java**

Chapter 5

File Documentation

5.1 src/main/java/millionaire/controller/AddQuestionController.java File Reference

Classes

- class **millionaire.controller.AddQuestionController**

Új kérdések hozzáadását vezérlő osztály.

5.2 AddQuestionController.java

Go to the documentation of this file.

```
00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.AddQuestionModel;
00005 import millionaire.model.QuestionRepository;
00006 import millionaire.view.AddQuestionView;
00007 import java.util.List;
00008
00018 public class AddQuestionController {
00022     private final AddQuestionModel model;
00023
00027     private final AddQuestionView view;
00028
00029
00034     public AddQuestionController(QuestionRepository repository) {
00035         this.model = new AddQuestionModel(repository);
00036         this.view = new AddQuestionView(this);
00037     }
00038
00039
00056     public void handleSubmit(String question, String optionA, String optionB, String optionC, String
optionD,
00057                             String correctAnswer, String difficulty, String category) {
00058         if (!model.validateInputs(question, optionA, optionB, optionC, optionD)) {
00059             view.showError("Minden mezőt ki kell tölteni!");
00060             return;
00061         }
00062
00063         boolean success = model.addQuestion(question, optionA, optionB, optionC, optionD,
correctAnswer, difficulty, category);
00064
00065         if (success) {
00066             view.showSuccess("A kérdés sikeresen hozzáadva!");
00067             view.clearFields();
00068         } else {
00069             view.showError("Hiba történt a kérdés mentése közben!");
00070         }
00071     }
```

```

00072
00073
00077     public void handleCancel() {
00078         view.closeWindow();
00079     }
00080
00081
00086     public List<String> getDifficulties() {
00087         return model.getDifficulties();
00088     }
00089
00090
00095     public List<String> getCategories() {
00096         return model.getCategories();
00097     }
00098
00099
00103     public void showDialog() {
00104         view.setVisible(true);
00105     }
00106 }

```

5.3 src/main/java/millionaire/controller/GameController.java File Reference

Classes

- class **millionaire.controller.GameController**

A játék fő vezérlő osztálya.

5.4 GameController.java

Go to the documentation of this file.

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.*;
00005 import millionaire.view.GameView;
00006 import javax.swing.*;
00007 import java.util.Timer;
00008 import java.util.TimerTask;
00009 import java.time.Duration;
00010
00021 public class GameController {
00022     private final GameModel model;
00023     private final GameView view;
00024     private final LifelineController lifelineController;
00025     private boolean isProcessingAnswer;
00033     public GameController(PlayerModel player, QuestionRepository questionRepository) {
00034         this.model = new GameModel(questionRepository, player);
00035         this.lifelineController = new LifelineController(this);
00036         this.view = new GameView(this, lifelineController);
00037         this.isProcessingAnswer = false;
00038
00039         initializeGame();
00040     }
00041
00042
00046     private void initializeGame() {
00047         updateGameState();
00048         view.setVisible(true);
00049     }
00050
00051
00061     public void handleAnswer(String selectedAnswer) {
00062         if (isProcessingAnswer) return;
00063         isProcessingAnswer = true;
00064
00065         Question currentQuestion = model.getCurrentQuestion();
00066         boolean isCorrect = currentQuestion.getCorrectAnswer().equals(selectedAnswer);
00067
00068         //

```

```

00069         view.showAnswerResult(selectedAnswer, currentQuestion.getCorrectAnswer(), isCorrect);
00070
00071         // Process the answer after a delay
00072         Timer timer = new Timer();
00073         timer.schedule(new TimerTask() {
00074             @Override
00075             public void run() {
00076                 SwingUtilities.invokeLater(() -> processAnswer(selectedAnswer));
00077                 timer.cancel();
00078             }
00079         }, 1500);
00080     }
00081
00082
00093     private void processAnswer(String selectedAnswer) {
00094         GameState result = model.processAnswer(selectedAnswer);
00095
00096         switch (result) {
00097             case CORRECT:
00098                 updateGameState();
00099                 break;
00100
00101             case WON:
00102             case LOST:
00103                 handleGameEnd(result);
00104                 break;
00105         }
00106
00107         isProcessingAnswer = false;
00108     }
00109
00110
00119     private void updateGameState() {
00120         Question currentQuestion = model.getCurrentQuestion();
00121         int currentLevel = model.getPlayer().getCurrentLevel();
00122
00123         view.updateQuestion(currentQuestion);
00124         view.updatePrizeLadder(currentLevel);
00125
00126         lifelineController.setGameContext(currentQuestion, currentLevel, view.getAnswerButtons());
00127     }
00128
00129
00140     private void handleGameEnd(GameState result) {
00141         PlayerModel player = model.getPlayer();
00142
00143         view.stopTimer();
00144
00145         long finalPrize = result == GameState.WON ?
00146             model.getCurrentPrize() :
00147             model.getLastSafePrize();
00148
00149         player.setPrize(finalPrize);
00150
00151         view.updateFinalPlaytime();
00152
00153         view.showGameEndDialog(result, player.getFormattedPrize());
00154
00155         saveScoreAndClose(player);
00156     }
00157
00158
00168     private void saveScoreAndClose(PlayerModel player) {
00169         ScoreboardController scoreboardController = new ScoreboardController();
00170         scoreboardController.addScore(player);
00171
00172         view.closeGame();
00173         new MainMenuController().showMenu();
00174     }
00175
00176
00181     public PlayerModel getPlayer() {
00182         return model.getPlayer();
00183     }
00184
00185
00190     public void updateGameTime(Duration elapsedTime) {
00191         model.updatePlayTime(elapsedTime);
00192     }
00193
00194
00199     private void checkTimeLimit(Duration currentTime) {
00200         if (model.isTimeUp(currentTime)) {
00201             handleGameEnd(GameState.LOST);
00202         }
00203     }
00204

```

```

00205
00210     public Question getCurrentQuestion() {
00211         return model.getCurrentQuestion();
00212     }
00213
00214
00219     public int getCurrentLevel() {
00220         return model.getPlayer().getCurrentLevel();
00221     }
00222
00223
00228     public void setAnswerButtonsEnabled(boolean enabled) {
00229         view.setAnswerButtonsEnabled(enabled);
00230     }
00231
00232
00237     public GameView getGameView() {
00238         return view;
00239     }
00240 }

```

5.5 src/main/java/millionaire/controller/LifelineController.java File Reference

Classes

- class **millionaire.controller.LifelineController**

A játék segítségait kezelő vezérlő osztály.

5.6 LifelineController.java

Go to the documentation of this file.

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.LifelineModel;
00005 import millionaire.model.Question;
00006 import millionaire.view.LifelineView;
00007 import javax.swing.JButton;
00008 import javax.swing.JPanel;
00009 import javax.swing.SwingUtilities;
00010
00011 import java.util.Map;
00012
00013
00023 public class LifelineController {
00024     private final LifelineModel model;
00025     private final LifelineView view;
00026     private final GameController gameController;
00027     private Question currentQuestion;
00028     private int currentLevel;
00029     private JButton[] answerButtons;
00036     public LifelineController(GameController gameController) {
00037         this.gameController = gameController;
00038         this.model = new LifelineModel();
00039         this.view = new LifelineView(gameController.getGameView(), this);
00040     }
00041
00042
00056     public void setGameContext(Question question, int level, JButton[] buttons) {
00057         this.currentQuestion = question;
00058         this.currentLevel = level;
00059         this.answerButtons = buttons;
00060         updateLifelineAvailability();
00061     }
00062
00063
00069     public void useFiftyFifty() {
00070         if (!model.isFiftyFiftyAvailable()) {
00071             return;
00072         }
00073         try {

```

```

00074         Map<String, Boolean> result = model.getFiftyFiftyResult(currentQuestion);
00075         view.updateAnswerButtons(answerButtons, result);
00076         updateLifelineAvailability();
00077     } catch (Exception e) {
00078         System.err.println("Error using fifty-fifty: " + e.getMessage());
00079     }
00080 }
00081
00082
00083 public void useAudienceHelp() {
00084     if (!model.isAudienceHelpAvailable()) {
00085         return;
00086     }
00087     try {
00088         Map<String, Integer> result = model.getAudienceResult(currentQuestion, currentLevel);
00089         view.showAudienceResults(result);
00090         updateLifelineAvailability();
00091     } catch (Exception e) {
00092         System.err.println("Error using audience help: " + e.getMessage());
00093     }
00094 }
00095
00096 private void updateLifelineAvailability() {
00097     SwingUtilities.invokeLater(() -> {
00098         view.updateButtonStates(
00099             model.isFiftyFiftyAvailable(),
00100             model.isAudienceHelpAvailable()
00101         );
00102     });
00103 }
00104
00105 public JPanel getLifelinePanel() {
00106     return view.createLifelinePanel();
00107 }
00108 }
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120
00121
00122
00123 }

```

5.7 src/main/java/millionaire/controller/MainMenuController.java File Reference

Classes

- class **millionaire.controller.MainMenuController**

A főmenü vezérlő osztálya.

5.8 MainMenuController.java

Go to the documentation of this file.

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.*;
00005 import millionaire.view.MainMenuView;
00006
00007
00008 public class MainMenuController {
00009     private final MainMenuModel model;
00010     private final MainMenuView view;
00011     public MainMenuController() {
00012         this.model = new MainMenuModel();
00013         this.view = new MainMenuView(this);
00014     }
00015
00016
00017
00018 public void startNewGame() {
00019     PlayerController playerController = new PlayerController(view);
00020     PlayerModel player = playerController.showPlayerCreationDialog();
00021
00022     if (player != null) {
00023         view.hideWindow();
00024     }
00025 }
00026
00027
00028
00029
00030
00031
00032
00033
00034
00035
00036
00037
00038
00039
00040
00041
00042

```

```

00043         GameController gameController = new GameController(player, model.getQuestionRepository());
00044     }
00045 }
00046
00047
00051 public void showAddQuestionDialog() {
00052     AddQuestionController addQuestionController =
00053         new AddQuestionController(model.getQuestionRepository());
00054     addQuestionController.showDialog();
00055 }
00056
00057
00061 public void showScoreboard() {
00062     ScoreboardController scoreboardController = new ScoreboardController();
00063     scoreboardController.showScoreboard();
00064 }
00065
00066
00070 public void showRules() {
00071     RulesController rulesController = new RulesController();
00072     rulesController.showRules();
00073 }
00074
00075
00079 public void exitGame() {
00080     System.exit(0);
00081 }
00082
00083
00087 public void showMenu() {
00088     view.showWindow();
00089 }
00090 }

```

5.9 src/main/java/millionaire/controller/PlayerController.java File Reference

Classes

- class **millionaire.controller.PlayerController**
A játékos létrehozását kezelő vezérlő osztály.

5.10 PlayerController.java

Go to the documentation of this file.

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.PlayerModel;
00005 import millionaire.view.PlayerView;
00006 import javax.swing.JFrame;
00007
00008
00017 public class PlayerController {
00018     private PlayerView view;
00019     private PlayerModel player;
00020     private boolean wasSuccessful;
00027     public PlayerController(JFrame parentFrame) {
00028         this.view = new PlayerView(parentFrame, this);
00029         this.wasSuccessful = false;
00030     }
00031
00032
00040     public void createPlayer(String name) {
00041         if (name.isEmpty()) {
00042             view.showError("A név nem lehet üres!");
00043             return;
00044         }
00045
00046         player = new PlayerModel(name);
00047         wasSuccessful = true;
00048         view.closeDialog();

```

```

00049     }
00050
00051
00055     public void cancelPlayerCreation() {
00056         wasSuccessful = false;
00057         view.closeDialog();
00058     }
00059
00060
00065     public PlayerModel showPlayerCreationDialog() {
00066         view.setVisible(true);
00067         return wasSuccessful ? player : null;
00068     }
00069 }

```

5.11 src/main/java/millionaire/controller/RulesController.java File Reference

Classes

- class **millionaire.controller.RulesController**
A játékszabályokat kezelő vezérlő osztály.

5.12 RulesController.java

Go to the documentation of this file.

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.RulesModel;
00005 import millionaire.view.RulesView;
00006
00015 public class RulesController {
00016     private final RulesModel model;
00017     private final RulesView view;
00025     public RulesController() {
00026         this.model = new RulesModel();
00027         this.view = new RulesView(this);
00028         initializeView();
00029     }
00030
00031
00039     private void initializeView() {
00040         if (model.isContentLoaded()) {
00041             view.displayRules(model.getRulesContent());
00042         } else {
00043             view.showError("Nem sikerült betölteni a játékszabályokat!");
00044         }
00045     }
00046
00047
00053     public void handleCloseRequest() {
00054         view.closeWindow();
00055     }
00056
00057
00063     public void showRules() {
00064         view.setVisible(true);
00065     }
00066 }

```

5.13 src/main/java/millionaire/controller/ScoreboardController.java File Reference

Classes

- class **millionaire.controller.ScoreboardController**
A dícsőséglista kezelését végző vezérlő osztály.

5.14 ScoreboardController.java

[Go to the documentation of this file.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.ScoreboardModel;
00005 import millionaire.model.PlayerModel;
00006 import millionaire.view.ScoreboardView;
00007
00017 public class ScoreboardController {
00018     private final ScoreboardModel model;
00019     private final ScoreboardView view;
00029     public ScoreboardController() {
00030         this.model = new ScoreboardModel();
00031         this.view = new ScoreboardView(this);
00032         refreshScoreboard();
00033     }
00034
00044     public void addScore(PlayerModel player) {
00045         model.addScore(player);
00046         refreshScoreboard();
00047     }
00048
00049
00056     private void refreshScoreboard() {
00057         view.updateScores(model.getPlayers());
00058     }
00059
00060
00067     public void showScoreboard() {
00068         refreshScoreboard();
00069         view.setVisible(true);
00070     }
00071
00072
00078     public void handleClose() {
00079         view.closeWindow();
00080     }
00081 }

```

5.15 src/main/java/millionaire/controller/TimerController.java File Reference

Classes

- class `millionaire.controller.TimerController`

Az időmérő vezérlője.

5.16 TimerController.java

[Go to the documentation of this file.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.TimerModel;
00005 import millionaire.view.TimerView;
00006 import java.time.Duration;
00007 import javax.swing.Timer;
00008
00016 public class TimerController {
00017     private final TimerModel model;
00018     private final TimerView view;
00019     private final Timer updateTimer;
00024     public TimerController() {
00025         this.model = new TimerModel();
00026         this.view = new TimerView();
00027
00028         // Timer létrehozása 1 másodperces frissítéssel
00029         this.updateTimer = new Timer(1000, e -> updateTime());

```

```

00030     }
00031
00035     private void updateTime() {
00036         model.updateElapsedTime();
00037         view.updateDisplay(model.getElapsedTime());
00038     }
00039
00043     public void startTimer() {
00044         model.start();
00045         updateTimer.start();
00046     }
00047
00051     public void stopTimer() {
00052         model.stop();
00053         updateTimer.stop();
00054     }
00055
00060     public TimerView getView() {
00061         return view;
00062     }
00063
00068     public Duration getElapsedTime() {
00069         return model.getElapsedTime();
00070     }
00071 }

```

5.17 src/main/java/millionaire/Main.java File Reference

Classes

- class **millionaire.Main**
A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.

5.18 Main.java

Go to the documentation of this file.

```

00001
00002 package millionaire;
00003
00004 import millionaire.controller.MainMenuController;
00005
00015 public class Main {
00016     private static boolean developerMode = false;
00022     public static boolean isDeveloperMode() {
00023         return developerMode;
00024     }
00025
00031     public static void main(String[] args) {
00032         if (args.length > 0 && args[0].equals("dev")) {
00033             developerMode = true;
00034             System.out.println("Developer mode activated");
00035         }
00036
00037         javax.swing.SwingUtilities.invokeLater(() -> {
00038             MainMenuController controller = new MainMenuController();
00039             controller.showMenu();
00040         });
00041     }
00042 }

```

5.19 src/main/java/millionaire/model/AddQuestionModel.java File Reference

Classes

- class **millionaire.model.AddQuestionModel**
Új kérdések hozzáadását kezelő modell osztály.

5.20 AddQuestionModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00004 import java.util.List;
00005
00012 public class AddQuestionModel {
00013     private final QuestionRepository repository;
00020     public AddQuestionModel(QuestionRepository repository) {
00021         this.repository = repository;
00022     }
00023
00024
00029     public List<String> getDifficulties() {
00030         return QuestionRepository.DIFFICULTIES;
00031     }
00032
00033
00038     public List<String> getCategories() {
00039         return QuestionRepository.CATEGORIES;
00040     }
00041
00042
00057     public boolean addQuestion(String questionText, String optionA, String optionB, String optionC,
String optionD,
                                String correctAnswer, String difficulty, String category) {
00058
00059         try {
00060             Question question = new Question.QuestionBuilder()
00061                 .difficulty(Integer.parseInt(difficulty))
00062                 .question(questionText)
00063                 .optionA(optionA)
00064                 .optionB(optionB)
00065                 .optionC(optionC)
00066                 .optionD(optionD)
00067                 .correctAnswer(correctAnswer)
00068                 .category(category)
00069                 .build();
00070
00071             repository.addQuestion(question);
00072             return true;
00073         } catch (Exception e) {
00074             return false;
00075         }
00076     }
00077
00078
00092     public boolean validateInputs(String question, String optionA, String optionB, String optionC,
String optionD) {
00093         return !question.trim().isEmpty() &&
00094             !optionA.trim().isEmpty() &&
00095             !optionB.trim().isEmpty() &&
00096             !optionC.trim().isEmpty() &&
00097             !optionD.trim().isEmpty();
00098     }
00099 }

```

5.21 src/main/java/millionaire/model/GameModel.java File Reference

Classes

- class **millionaire.model.GameModel**
A játék logikáját és állapotát kezelő osztály.

5.22 GameModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003

```

```

00004 import java.util.*;
00005 import java.time.Duration;
00006
00007
00017 public class GameModel {
00018
00020     private static final long[] PRIZES = {
00021         40_000_000L, 20_000_000L, 10_000_000L, 5_000_000L, 3_000_000L,
00022         1_500_000L, 800_000L, 500_000L, 300_000L, 200_000L,
00023         100_000L, 50_000L, 25_000L, 10_000L, 5_000L
00024     };
00025
00027     private static final int[] SAFE_SPOTS = {5, 8, 11, 13, 15};
00028
00029     private final QuestionRepository questionRepository;
00030     private final PlayerModel player;
00031     private Question currentQuestion;
00032     private final Random random;
00033     private boolean isGameOver;
00034     private final Duration timeLimit;
00042     public GameModel(QuestionRepository questionRepository, PlayerModel player) {
00043         this.questionRepository = questionRepository;
00044         this.player = player;
00045         this.random = new Random();
00046         this.isGameOver = false;
00047         this.timeLimit = null; // Duration.ofMinutes(X)
00048         this.currentQuestion = selectNextQuestion();
00049     }
00050
00051
00057     public Question selectNextQuestion() {
00058         List<Question> questions =
00059             questionRepository.getQuestionsByDifficulty(player.getCurrentLevel());
00059         if (questions.isEmpty()) {
00060             throw new IllegalStateException("No questions available for level " +
00061                 player.getCurrentLevel());
00061         }
00062         currentQuestion = questions.get(random.nextInt(questions.size()));
00063         return currentQuestion;
00064     }
00065
00066
00072     public GameState processAnswer(String selectedAnswer) {
00073         if (isGameOver || currentQuestion == null) {
00074             return GameState.LOST;
00075         }
00076
00077         boolean isCorrect = currentQuestion.getCorrectAnswer().equals(selectedAnswer);
00078
00079         if (!isCorrect) {
00080             isGameOver = true;
00081             return GameState.LOST;
00082         }
00083
00084         if (player.getCurrentLevel() == 15) {
00085             isGameOver = true;
00086             return GameState.WON;
00087         }
00088
00089         player.incrementLevel();
00090         currentQuestion = selectNextQuestion();
00091         return GameState.CORRECT;
00092     }
00093
00094
00099     public long getLastSafePrize() {
00100         for (int i = SAFE_SPOTS.length - 1; i >= 0; i--) {
00101             if (player.getCurrentLevel() >= SAFE_SPOTS[i]) {
00102                 return PRIZES[15 - SAFE_SPOTS[i]];
00103             }
00104         }
00105         return 0L;
00106     }
00107
00108
00113     public long getCurrentPrize() {
00114         return PRIZES[15 - player.getCurrentLevel()];
00115     }
00116
00117
00119     public PlayerModel getPlayer() { return player; }
00120
00121
00123     public Question getCurrentQuestion() { return currentQuestion; }
00124
00125
00127     public boolean isGameOver() { return isGameOver; }

```

```

00128
00129
00134     public static List<PrizeLevel> getPrizeLevels() {
00135         List<PrizeLevel> levels = new ArrayList<>();
00136         for (int i = 0; i < PRIZES.length; i++) {
00137             int level = 15 - i;
00138             long prize = PRIZES[i];
00139             boolean isSafeSpot = Arrays.stream(SAFE_SPOTS).anyMatch(spot -> spot == level);
00140             levels.add(new PrizeLevel(level, prize, isSafeSpot));
00141         }
00142         return levels;
00143     }
00144
00145
00150     public void updatePlayTime(Duration elapsedTime) {
00151         player.setPlaytime(elapsedTime);
00152     }
00153
00154
00160     public boolean isTimeUp(Duration currentTime) {
00161         return timeLimit != null && currentTime.compareTo(timeLimit) > 0;
00162     }
00163 }

```

5.23 src/main/java/millionaire/model/GameState.java File Reference

Classes

- enum **millionaire.model.GameState**
Enum osztály, amely a játék állapotait reprezentálja.

5.24 GameState.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00007 public enum GameState {
00012     CORRECT,
00013
00018     LOST,
00019
00024     WON;
00025
00030     public boolean isGameEnding() {
00031         return this == LOST || this == WON;
00032     }
00033 }

```

5.25 src/main/java/millionaire/model/LifelineModel.java File Reference

Classes

- class **millionaire.model.LifelineModel**
Segítségek (lifeline) logikáját kezelő osztály.

5.26 LifelineModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00004 import java.util.*;
00005
00012 public class LifelineModel {
00013     private final Random random = new Random();
00014     private boolean fiftyFiftyUsed = false;
00015     private boolean audienceHelpUsed = false;
00028     public Map<String, Boolean> getFiftyFiftyResult(Question question) {
00029         if (fiftyFiftyUsed) {
00030             throw new IllegalStateException("Fifty-fifty lifeline already used");
00031         }
00032
00033         String correctAnswer = question.getCorrectAnswer();
00034         Map<String, Boolean> visibilityMap = new HashMap<>();
00035         List<String> options = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
00036
00037         // Kivesszük a jó választ a lehetőségek közül..
00038         options.remove(correctAnswer);
00039         // Random választunk egy rossz választ amit megtartunk..
00040         String keepWrongAnswer = options.remove(random.nextInt(options.size()));
00041
00042         // Beállítjuk a válaszlehetőségek láthatóságát..
00043         for (String option : Arrays.asList("A", "B", "C", "D")) {
00044             visibilityMap.put(option, option.equals(correctAnswer) || option.equals(keepWrongAnswer));
00045         }
00046
00047         fiftyFiftyUsed = true;
00048         return visibilityMap;
00049     }
00050
00051
00063     public Map<String, Integer> getAudienceResult(Question question, int currentLevel) {
00064         if (audienceHelpUsed) {
00065             throw new IllegalStateException("Audience help already used");
00066         }
00067
00068         int correctPercentage = calculateCorrectPercentage(currentLevel);
00069         Map<String, Integer> percentages = distributePercentages(question.getCorrectAnswer(),
correctPercentage);
00070
00071         audienceHelpUsed = true;
00072         return percentages;
00073     }
00074
00075
00087     private int calculateCorrectPercentage(int currentLevel) {
00088         if (currentLevel <= 5) {
00089             return random.nextInt(71, 81);
00090         } else if (currentLevel <= 10) {
00091             return random.nextInt(61, 71);
00092         } else {
00093             return random.nextInt(51, 61);
00094         }
00095     }
00096
00097
00105     private Map<String, Integer> distributePercentages(String correctAnswer, int correctPercentage) {
00106         Map<String, Integer> percentages = new LinkedHashMap<>();
00107         List<String> wrongAnswers = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
00108         wrongAnswers.remove(correctAnswer);
00109
00110         percentages.put(correctAnswer, correctPercentage);
00111
00112         int remaining = 100 - correctPercentage;
00113         for (int i = 0; i < wrongAnswers.size(); i++) {
00114             int percentage;
00115             if (i == wrongAnswers.size() - 1) {
00116                 percentage = remaining;
00117             } else {
00118                 percentage = Math.min(remaining - (wrongAnswers.size() - i - 1),
random.nextInt(5, 26));
00119             }
00120             percentages.put(wrongAnswers.get(i), percentage);
00121             remaining -= percentage;
00122         }
00123
00124         return new TreeMap<>(percentages);
00125     }
00126 }
00127
00128

```

```

00133     public boolean isFiftyFiftyAvailable() {
00134         return !fiftyFiftyUsed;
00135     }
00136
00137
00142     public boolean isAudienceHelpAvailable() {
00143         return !audienceHelpUsed;
00144     }
00145 }

```

5.27 src/main/java/millionaire/model/MainMenuModel.java File Reference

Classes

- class **millionaire.model.MainMenuModel**

A főmenü modell osztálya.

5.28 MainMenuModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00010 public class MainMenuModel {
00011     private final QuestionRepository questionRepository;
00018     public MainMenuModel() {
00019         this.questionRepository = new QuestionRepository();
00020     }
00021
00026     public QuestionRepository getQuestionRepository() {
00027         return questionRepository;
00028     }
00029 }

```

5.29 src/main/java/millionaire/model/PlayerModel.java File Reference

Classes

- class **millionaire.model.PlayerModel**

5.30 PlayerModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00004 import java.text.NumberFormat;
00005 import java.time.Duration;
00006 import java.time.Instant;
00007 import java.util.Locale;
00008
00009 public class PlayerModel implements Comparable<PlayerModel> {
00010     private String name;
00011     private int currentLevel;
00012     private Duration playtime;
00013     private Instant startTime;
00014     private long prize;
00021     public PlayerModel(String name) {
00022         this.name = name;
00023         this.currentLevel = 1;

```

```
00024         this.playtime = Duration.ZERO;
00025         this.startTime = Instant.now();
00026         this.prize = 0;
00027     }
00028
00029
00037     public PlayerModel(String name, int level, long prize, Duration playtime) {
00038         this.name = name;
00039         this.currentLevel = level;
00040         this.prize = prize;
00041         this.playtime = playtime;
00042         this.startTime = Instant.now();
00043     }
00044
00045
00050     public String getName() { return name; }
00051
00052
00057     public int getCurrentLevel() { return currentLevel; }
00058
00059
00064     public Duration getPlaytime() { return playtime; }
00065
00066
00071     public Instant getStartTime() { return startTime; }
00072
00073
00078     public long getPrize() { return prize; }
00079
00080
00085     public void setName(String name) { this.name = name; }
00086
00087
00092     public void setCurrentLevel(int currentLevel) { this.currentLevel = currentLevel; }
00093
00094
00099     public void setPlaytime(Duration playtime) { this.playtime = playtime; }
00100
00105     public void setPrize(long prize) { this.prize = prize; }
00106
00107
00111     public void updatePlaytime() {
00112         if (this.playtime == null || this.playtime.equals(Duration.ZERO)) {
00113             this.playtime = Duration.between(startTime, Instant.now());
00114         }
00115         // Ha a Timer már beállította a játékidjét, akkor ne frissítsük
00116     }
00117
00118
00122     public void incrementLevel() { currentLevel++; }
00123
00124
00129     public String getFormattedPrize() {
00130         return NumberFormat.getNumberInstance(new Locale("hu", "HU")).format(prize) + " Ft";
00131     }
00132
00133
00138     public String getFormattedPlaytime() {
00139         if (playtime == null) {
00140             return "00:00";
00141         }
00142         long totalSeconds = playtime.getSeconds();
00143         long minutes = totalSeconds / 60;
00144         long seconds = totalSeconds % 60;
00145         return String.format("%02d:%02d", minutes, seconds);
00146     }
00147
00148
00154     @Override
00155     public int compareTo(PlayerModel other) {
00156         int levelComparison = Integer.compare(other.currentLevel, this.currentLevel);
00157         if (levelComparison != 0) {
00158             return levelComparison;
00159         }
00160         return this.playtime.compareTo(other.playtime);
00161     }
00162
00163
00164
00170     @Override
00171     public boolean equals(Object obj) {
00172         if (!(obj instanceof PlayerModel)) return false;
00173         PlayerModel other = (PlayerModel) obj;
00174         return this.name.equals(other.name);
00175     }
00176 }
```

5.31 src/main/java/millionaire/model/PrizeLevel.java File Reference

Classes

- class **millionaire.model.PrizeLevel**
Nyereményszintet reprezentáló osztály.

5.32 PrizeLevel.java

Go to the documentation of this file.

```
00001
00002 package millionaire.model;
00003
00013 public class PrizeLevel {
00014     private final int level;
00015     private final long prize;
00016     private final boolean isSafeSpot;
00025     public PrizeLevel(int level, long prize, boolean isSafeSpot) {
00026         this.level = level;
00027         this.prize = prize;
00028         this.isSafeSpot = isSafeSpot;
00029     }
00030
00035     public int getLevel() { return level; }
00036
00041     public long getPrize() { return prize; }
00042
00047     public boolean isSafeSpot() { return isSafeSpot; }
00048 }
```

5.33 src/main/java/millionaire/model/Question.java File Reference

Classes

- class **millionaire.model.Question**
Egy kérdést reprezentáló osztály.
- class **millionaire.model.Question.QuestionBuilder**
Builder osztály a Question példányok létrehozásához.

5.34 Question.java

Go to the documentation of this file.

```
00001
00002 package millionaire.model;
00003
00016 public class Question {
00017     private final int difficulty;
00018     private final String question;
00019     private final String optionA;
00020     private final String optionB;
00021     private final String optionC;
00022     private final String optionD;
00023     private final String correctAnswer;
00024     private final String category;
00031     private Question(QuestionBuilder builder) {
00032         this.difficulty = builder.difficulty;
00033         this.question = builder.question;
00034         this.optionA = builder.optionA;
00035         this.optionB = builder.optionB;
00036         this.optionC = builder.optionC;
00037         this.optionD = builder.optionD;
00038         this.correctAnswer = builder.correctAnswer;
```

```
00039         this.category = builder.category;
00040     }
00041
00042
00047     public int getDifficulty() { return difficulty; }
00048
00049
00054     public String getQuestion() { return question; }
00055
00056
00061     public String getOptionA() { return optionA; }
00062
00063
00068     public String getOptionB() { return optionB; }
00069
00070
00075     public String getOptionC() { return optionC; }
00076
00077
00082     public String getOptionD() { return optionD; }
00083
00084
00089     public String getCorrectAnswer() { return correctAnswer; }
00090
00091
00096     public String getCategory() { return category; }
00097
00098
00099
00100
00101
00108     public static class QuestionBuilder {
00109         private int difficulty;
00110         private String question;
00111         private String optionA;
00112         private String optionB;
00113         private String optionC;
00114         private String optionD;
00115         private String correctAnswer;
00116         private String category;
00124         public QuestionBuilder difficulty(int difficulty) {
00125             this.difficulty = difficulty;
00126             return this;
00127         }
00128
00129
00135         public QuestionBuilder question(String question) {
00136             this.question = question;
00137             return this;
00138         }
00139
00140
00146         public QuestionBuilder optionA(String optionA) {
00147             this.optionA = optionA;
00148             return this;
00149         }
00150
00151
00157         public QuestionBuilder optionB(String optionB) {
00158             this.optionB = optionB;
00159             return this;
00160         }
00161
00162
00168         public QuestionBuilder optionC(String optionC) {
00169             this.optionC = optionC;
00170             return this;
00171         }
00172
00173
00179         public QuestionBuilder optionD(String optionD) {
00180             this.optionD = optionD;
00181             return this;
00182         }
00183
00184
00190         public QuestionBuilder correctAnswer(String correctAnswer) {
00191             this.correctAnswer = correctAnswer;
00192             return this;
00193         }
00194
00195
00201         public QuestionBuilder category(String category) {
00202             this.category = category;
00203             return this;
00204         }
00205
```

```

00206
00212     public Question build() {
00213         validateQuestionData();
00214         return new Question(this);
00215     }
00216
00217
00222     private void validateQuestionData() {
00223         if (question == null || question.trim().isEmpty()) {
00224             throw new IllegalStateException("Question cannot be empty");
00225         }
00226         if (optionA == null || optionA.trim().isEmpty() ||
00227             optionB == null || optionB.trim().isEmpty() ||
00228             optionC == null || optionC.trim().isEmpty() ||
00229             optionD == null || optionD.trim().isEmpty()) {
00230             throw new IllegalStateException("All options must be provided");
00231         }
00232         if (correctAnswer == null || !correctAnswer.matches("[A-D]")) {
00233             throw new IllegalStateException("Correct answer must be A, B, C, or D");
00234         }
00235         if (difficulty < 1 || difficulty > 15) {
00236             throw new IllegalStateException("Difficulty must be between 1 and 15");
00237         }
00238         if (category == null || category.trim().isEmpty()) {
00239             throw new IllegalStateException("Category must be provided");
00240         }
00241     }
00242 }
00243 }

```

5.35 src/main/java/millionaire/model/QuestionRepository.java File Reference

Classes

- class **millionaire.model.QuestionRepository**
Kérdések perzisztens tárolását végző osztály.

5.36 QuestionRepository.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00004 import com.google.gson.Gson;
00005 import com.google.gson.GsonBuilder;
00006 import com.google.gson.JsonArray;
00007 import com.google.gson.JsonObject;
00008 import com.google.gson.JsonElement;
00009 import com.google.gson.JsonParser;
00010 import java.io.*;
00011 import java.nio.file.*;
00012 import java.util.*;
00013 import java.util.stream.Collectors;
00014
00024 public class QuestionRepository {
00025     private static final String JSON_FILE_PATH = "src/main/resources/loim.json";
00026     private final List<Question> questions;
00027     private final Gson gson;
00031     public static final List<String> CATEGORIES = Arrays.asList(
00032         "TÖRTÉNELEM", "SPORT", "FÖLDRAJZ", "IRODALOM", "TUDOMÁNY", "ZENE", "FILM", "KONYHA",
00033         "ÁLTALÁNOS"
00034     );
00035
00037     public static final List<String> DIFFICULTIES = Arrays.asList(
00038         "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15"
00039     );
00040
00041
00047     public QuestionRepository() {
00048         this.questions = new ArrayList<>();

```

```

00049         this.gson = new GsonBuilder().setPrettyPrinting().create();
00050         loadQuestions();
00051     }
00052
00053     private void loadQuestions() {
00064         try (Reader reader = Files.newBufferedReader(Paths.get(JSON_FILE_PATH))) {
00065             JSONArray jsonArray = JsonParser.parseReader(reader).getAsJsonArray();
00066
00067             for (JsonElement element : jsonArray) {
00068                 JsonObject questionObj = element.getAsJsonObject();
00069                 Question question = new Question.QuestionBuilder()
00070                     .difficulty(questionObj.get("Nehézség").getAsInt())
00071                     .question(questionObj.get("Kérdés").getString())
00072                     .optionA(questionObj.get("A").getString())
00073                     .optionB(questionObj.get("B").getString())
00074                     .optionC(questionObj.get("C").getString())
00075                     .optionD(questionObj.get("D").getString())
00076                     .correctAnswer(questionObj.get("Válasz").getString())
00077                     .category(questionObj.get("Kategória").getString())
00078                     .build();
00079                 questions.add(question);
00080             }
00081         } catch (IOException e) {
00082             System.err.println("Error loading questions: " + e.getMessage());
00083         }
00084     }
00085
00086     private void saveQuestions() {
00097         try {
00098             JSONArray jsonArray = new JSONArray();
00099             for (Question question : questions) {
00100                 JsonObject questionObj = new JsonObject();
00101                 questionObj.addProperty("Nehézség", question.getDifficulty());
00102                 questionObj.addProperty("Kérdés", question.getQuestion());
00103                 questionObj.addProperty("A", question.getOptionA());
00104                 questionObj.addProperty("B", question.getOptionB());
00105                 questionObj.addProperty("C", question.getOptionC());
00106                 questionObj.addProperty("D", question.getOptionD());
00107                 questionObj.addProperty("Válasz", question.getCorrectAnswer());
00108                 questionObj.addProperty("Kategória", question.getCategory());
00109                 jsonArray.add(questionObj);
00110             }
00111
00112             Files.createDirectories(Paths.get(JSON_FILE_PATH).getParent());
00113             Files.write(Paths.get(JSON_FILE_PATH),
00114                 gson.toJson(jsonArray).getBytes(),
00115                 StandardOpenOption.CREATE,
00116                 StandardOpenOption.TRUNCATE_EXISTING);
00117         } catch (IOException e) {
00118             throw new RuntimeException("Error saving questions: " + e.getMessage());
00119         }
00120     }
00121
00122     public void addQuestion(Question question) {
00123         questions.add(question);
00124         saveQuestions();
00125     }
00126
00127     public List<Question> getQuestionsByDifficulty(int difficulty) {
00128         return questions.stream()
00129             .filter(q -> q.getDifficulty() == difficulty)
00130             .collect(Collectors.toList());
00131     }
00132
00133     public List<Question> getQuestionsByCategory(String category) {
00134         return questions.stream().filter(q ->
00135             q.getCategory().equals(category)).collect(Collectors.toList());
00136     }
00137
00138     public List<Question> getAllQuestions() {
00139         return new ArrayList<>(questions);
00140     }
00141 }

```

5.37 src/main/java/millionaire/model/RulesModel.java File Reference

Classes

- class **millionaire.model.RulesModel**
Játékszabályok kezelését végző osztály.

5.38 RulesModel.java

Go to the documentation of this file.

```
00001
00002 package millionaire.model;
00003
00004 import java.io.IOException;
00005 import java.nio.charset.StandardCharsets;
00006 import java.nio.file.Files;
00007 import java.nio.file.Path;
00008 import java.nio.file.Paths;
00009
00018 public class RulesModel {
00020     private static final String RULES_FILE_PATH = "src/main/resources/rules.txt";
00021
00023     private String rulesContent;
00024
00030     public RulesModel() {
00031         loadRulesContent();
00032     }
00033
00040     private void loadRulesContent() {
00041         try {
00042             Path path = Paths.get(RULES_FILE_PATH);
00043             rulesContent = Files.readString(path, StandardCharsets.UTF_8);
00044         } catch (IOException e) {
00045             rulesContent = "Failed to load rules: " + e.getMessage();
00046         }
00047     }
00048
00053     public String getRulesContent() {
00054         return rulesContent;
00055     }
00056
00061     public boolean isContentLoaded() {
00062         return rulesContent != null && !rulesContent.startsWith("Failed to load rules");
00063     }
00064 }
```

5.39 src/main/java/millionaire/model/ScoreboardModel.java File Reference

Classes

- class **millionaire.model.ScoreboardModel**
Dicsőségtábla kezelését végző osztály.

5.40 ScoreboardModel.java

Go to the documentation of this file.

```
00001
00002 package millionaire.model;
00003
00004 import com.google.gson.Gson;
00005 import com.google.gson.GsonBuilder;
```

```

00006 import com.google.gson.JsonArray;
00007 import com.google.gson.JsonObject;
00008 import com.google.gson.JsonElement;
00009 import com.google.gson.JsonParser;
00010 import java.io.*;
00011 import java.nio.file.*;
00012 import java.time.Duration;
00013 import java.util.*;
00014
00024 public class ScoreboardModel {
00026     private static final String JSON_FILE_PATH = "src/main/resources/scoreboard.json";
00027
00029     private List<PlayerModel> players;
00030
00032     private final Gson gson;
00033
00039     public ScoreboardModel() {
00040         this.players = new ArrayList<>();
00041         this.gson = new GsonBuilder().setPrettyPrinting().create();
00042         initializeAndLoadScores();
00043     }
00044
00054     private void initializeAndLoadScores() {
00055         try {
00056             Files.createDirectories(Paths.get("src/main/resources"));
00057
00058             Path filePath = Paths.get(JSON_FILE_PATH);
00059             if (!Files.exists(filePath)) {
00060                 Files.write(filePath, [].getBytes());
00061                 return;
00062             }
00063
00064             String jsonContent = Files.readString(filePath);
00065             if (jsonContent.trim().isEmpty()) {
00066                 Files.write(filePath, [].getBytes());
00067                 return;
00068             }
00069
00070             JsonElement jsonElement = JsonParser.parseString(jsonContent);
00071             if (!jsonElement.isJsonArray()) {
00072                 Files.write(filePath, [].getBytes());
00073                 return;
00074             }
00075
00076             JsonArray jsonArray = jsonElement.getAsJsonArray();
00077             for (JsonElement element : jsonArray) {
00078                 JsonObject playerObj = element.getAsJsonObject();
00079                 PlayerModel player = new PlayerModel(
00080                     playerObj.get("name").getAsString(),
00081                     playerObj.get("level").getAsInt(),
00082                     playerObj.get("prize").getAsLong(),
00083                     Duration.ofSeconds(playerObj.get("playtime").getAsLong())
00084                 );
00085                 players.add(player);
00086             }
00087
00088             sortPlayers();
00089         } catch (Exception e) {
00090             System.err.println("Error initializing scoreboard: " + e.getMessage());
00091             players = new ArrayList<>();
00092             try {
00093                 Files.write(Paths.get(JSON_FILE_PATH), [].getBytes());
00094             } catch (IOException ioe) {
00095                 System.err.println("Could not create scoreboard file: " + ioe.getMessage());
00096             }
00097         }
00098     }
00099
00109     public void addScore(PlayerModel newPlayer) {
00110         Optional<PlayerModel> existingPlayer = players.stream()
00111             .filter(p -> p.getName().equals(newPlayer.getName()))
00112             .findFirst();
00113
00114         if (existingPlayer.isPresent()) {
00115             PlayerModel old = existingPlayer.get();
00116             if (shouldReplaceScore(newPlayer, old)) {
00117                 players.remove(old);
00118                 players.add(newPlayer);
00119             }
00120         } else {
00121             players.add(newPlayer);
00122         }
00123
00124         sortPlayers();
00125         saveScores();
00126     }
00127

```

```

00139     private boolean shouldReplaceScore(PlayerModel newPlayer, PlayerModel oldPlayer) {
00140         return newPlayer.getCurrentLevel() > oldPlayer.getCurrentLevel() ||
00141             (newPlayer.getCurrentLevel() == oldPlayer.getCurrentLevel() &&
00142              newPlayer.getPlaytime().compareTo(oldPlayer.getPlaytime()) < 0);
00143     }
00144
00152     private void sortPlayers() {
00153         Collections.sort(players);
00154     }
00155
00160     public List<PlayerModel> getPlayers() {
00161         return new ArrayList<>(players);
00162     }
00163
00170     private void saveScores() {
00171         try {
00172             JSONArray jsonArray = new JSONArray();
00173             for (PlayerModel player : players) {
00174                 JSONObject playerObj = new JSONObject();
00175                 playerObj.addProperty("name", player.getName());
00176                 playerObj.addProperty("level", player.getCurrentLevel());
00177                 playerObj.addProperty("prize", player.getPrize());
00178                 // Store the actual seconds value
00179                 playerObj.addProperty("playtime", player.getPlaytime().getSeconds());
00180                 jsonArray.add(playerObj);
00181             }
00182
00183             Files.createDirectories(Paths.get("src/main/resources"));
00184
00185             Files.write(
00186                 Paths.get(JSON_FILE_PATH),
00187                 gson.toJson(jsonArray).getBytes(),
00188                 StandardOpenOption.CREATE,
00189                 StandardOpenOption.TRUNCATE_EXISTING,
00190                 StandardOpenOption.WRITE
00191             );
00192         } catch (IOException e) {
00193             System.err.println("Error saving scoreboard: " + e.getMessage());
00194         }
00195     }
00196 }

```

5.41 src/main/java/millionaire/model/TimerModel.java File Reference

Classes

- class **millionaire.model.TimerModel**

Az időmérő adatmodellje.

5.42 TimerModel.java

Go to the documentation of this file.

```

00001
00002 package millionaire.model;
00003
00004 import java.time.Duration;
00005 import java.time.Instant;
00006
00014 public class TimerModel {
00015     private final Instant startTime;
00016     private Duration elapsedTime;
00017     private boolean isRunning;
00022     public TimerModel() {
00023         this.startTime = Instant.now();
00024         this.elapsedTime = Duration.ZERO;
00025         this.isRunning = false;
00026     }
00027
00031     public void updateElapsedTime() {
00032         if (isRunning) {
00033             elapsedTime = Duration.between(startTime, Instant.now());
00034         }
00035     }

```

```

00036
00040     public void start() {
00041         isRunning = true;
00042     }
00043
00047     public void stop() {
00048         isRunning = false;
00049     }
00050
00055     public Duration getElapsedTime() {
00056         return elapsedTime;
00057     }
00058
00063     public Instant getStartTime() {
00064         return startTime;
00065     }
00066 }

```

5.43 src/main/java/millionaire/view/AddQuestionView.java File Reference

Classes

- class **millionaire.view.AddQuestionView**

Új kérdések hozzáadására szolgáló grafikus felület.

5.44 AddQuestionView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.AddQuestionController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007 import java.util.List;
00008
00018 public class AddQuestionView extends JFrame {
00019     private static final int WINDOW_WIDTH = 800;
00020     private static final int WINDOW_HEIGHT = 600;
00021     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00026     private final AddQuestionController controller;
00028     private JTextField questionField;
00029     private JTextField optionAField;
00030     private JTextField optionBField;
00031     private JTextField optionCField;
00032     private JTextField optionDField;
00033     private JComboBox<String> correctAnswerCombo;
00034     private JComboBox<String> difficultyCombo;
00035     private JComboBox<String> categoryCombo;
00042     public AddQuestionView(AddQuestionController controller) {
00043         this.controller = controller;
00044         setupWindow();
00045         setupComponents();
00046         pack();
00047         setLocationRelativeTo(null);
00048     }
00049
00050
00061     private void setupWindow() {
00062         setTitle("Új kérdés hozzáadása");
00063         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00064         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00065         setResizable(false);
00066     }
00067
00068
00077     private void setupComponents() {
00078         JPanel mainPanel = new JPanel();
00079         mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
00080         mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

```

```

00081         mainPanel.setBackground(BACKGROUND_COLOR);
00082
00083         addTitle(mainPanel);
00084         addFormPanel(mainPanel);
00085         addButtonPanel(mainPanel);
00086
00087         add(mainPanel);
00088     }
00089
00090
00091     private void addTitle(JPanel mainPanel) {
00092         JLabel titleLabel = new JLabel("Új kérdés hozzáadása");
00093         titleLabel.setFont(new Font("Arial", Font.BOLD, 24));
00094         titleLabel.setForeground(TEXT_COLOR);
00095         titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
00096         mainPanel.add(titleLabel);
00097         mainPanel.add(Box.createRigidArea(new Dimension(0, 20)));
00098     }
00099
00100     private void addFormPanel(JPanel mainPanel) {
00101         JPanel formPanel = new JPanel(new GridBagLayout());
00102         formPanel.setBackground(BACKGROUND_COLOR);
00103         GridBagConstraints gbc = new GridBagConstraints();
00104         gbc.fill = GridBagConstraints.HORIZONTAL;
00105         gbc.insets = new Insets(5, 5, 5, 5);
00106
00107         questionField = new JTextField(40);
00108         optionAField = new JTextField(40);
00109         optionBField = new JTextField(40);
00110         optionCField = new JTextField(40);
00111         optionDField = new JTextField(40);
00112         correctAnswerCombo = new JComboBox<>(new String[]{"A", "B", "C", "D"});
00113         difficultyCombo = new JComboBox<>(controller.getDifficulties().toArray(new String[0]));
00114         categoryCombo = new JComboBox<>(controller.getCategories().toArray(new String[0]));
00115
00116         addFormRow(formPanel, gbc, 0, "Kérdés:", questionField);
00117         addFormRow(formPanel, gbc, 1, "A válasz:", optionAField);
00118         addFormRow(formPanel, gbc, 2, "B válasz:", optionBField);
00119         addFormRow(formPanel, gbc, 3, "C válasz:", optionCField);
00120         addFormRow(formPanel, gbc, 4, "D válasz:", optionDField);
00121         addFormRow(formPanel, gbc, 5, "Helyes válasz:", correctAnswerCombo);
00122         addFormRow(formPanel, gbc, 6, "Nehézség:", difficultyCombo);
00123         addFormRow(formPanel, gbc, 7, "Kategória:", categoryCombo);
00124
00125         mainPanel.add(formPanel);
00126         mainPanel.add(Box.createRigidArea(new Dimension(0, 20)));
00127     }
00128
00129     private void addFormRow(JPanel panel, GridBagConstraints gbc, int row, String labelText,
00130 JComponent component) {
00131         gbc.gridx = 0;
00132         gbc.gridy = row;
00133         gbc.gridwidth = 1;
00134
00135         JLabel label = new JLabel(labelText);
00136         label.setForeground(TEXT_COLOR);
00137         label.setPreferredSize(new Dimension(100, 25));
00138         panel.add(label, gbc);
00139
00140         gbc.gridx = 1;
00141         gbc.gridwidth = 2;
00142         panel.add(component, gbc);
00143     }
00144
00145     private void addButtonPanel(JPanel mainPanel) {
00146         JPanel buttonPanel = new JPanel();
00147         buttonPanel.setBackground(BACKGROUND_COLOR);
00148
00149         JButton submitButton = createStyledButton("Mentés");
00150         JButton cancelButton = createStyledButton("Mégse");
00151
00152         submitButton.addActionListener(e -> handleSubmit());
00153         cancelButton.addActionListener(e -> handleCancel());
00154
00155         buttonPanel.add(submitButton);
00156         buttonPanel.add(Box.createRigidArea(new Dimension(10, 0)));
00157         buttonPanel.add(cancelButton);
00158
00159         mainPanel.add(buttonPanel);
00160     }
00161
00162     private JButton createStyledButton(String text) {
00163         JButton button = new JButton(text);

```

```

00217         button.setPreferredSize(new Dimension(120, 40));
00218         button.setFont(new Font("Arial", Font.PLAIN, 14));
00219         button.setFocusPainted(false);
00220         button.setBackground(BUTTON_COLOR);
00221         button.setForeground(TEXT_COLOR);
00222         button.setBorder(BorderFactory.createRaisedBevelBorder());
00223
00224         button.addMouseListener(new java.awt.event.MouseAdapter() {
00225             @Override
00226             public void mouseEntered(java.awt.event.MouseEvent evt) {
00227                 button.setBackground(BUTTON_HOVER_COLOR);
00228             }
00229
00230             @Override
00231             public void mouseExited(java.awt.event.MouseEvent evt) {
00232                 button.setBackground(BUTTON_COLOR);
00233             }
00234         });
00235
00236         return button;
00237     }
00238
00239     private void handleSubmit() {
00240         controller.handleSubmit(
00241             questionField.getText(),
00242             optionAField.getText(),
00243             optionBField.getText(),
00244             optionCField.getText(),
00245             optionDField.getText(),
00246             (String) correctAnswerCombo.getSelectedItem(),
00247             (String) difficultyCombo.getSelectedItem(),
00248             (String) categoryCombo.getSelectedItem()
00249         );
00250     }
00251
00252     private void handleCancel() {
00253         controller.handleCancel();
00254     }
00255
00256     public void showError(String message) {
00257         JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
00258     }
00259
00260     public void showSuccess(String message) {
00261         JOptionPane.showMessageDialog(this, message, "Siker", JOptionPane.INFORMATION_MESSAGE);
00262     }
00263
00264     public void clearFields() {
00265         questionField.setText("");
00266         optionAField.setText("");
00267         optionBField.setText("");
00268         optionCField.setText("");
00269         optionDField.setText("");
00270         correctAnswerCombo.setSelectedIndex(0);
00271         difficultyCombo.setSelectedIndex(0);
00272         categoryCombo.setSelectedIndex(0);
00273     }
00274
00275     public void closeWindow() {
00276         dispose();
00277     }
00278 }

```

5.45 src/main/java/millionaire/view/MapView.java File Reference

Classes

- class **millionaire.view.MapView**

A játék fő grafikus felülete.

5.46 GameView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.Main;
00005 import millionaire.controller.GameController;
00006 import millionaire.controller.LifelineController;
00007 import millionaire.controller.TimerController;
00008 import millionaire.model.*;
00009 import javax.swing.*;
00010 import java.awt.*;
00011 import java.util.*;
00012 import java.util.List;
00013 import java.util.Timer;
00014
00026 public class GameView extends JFrame {
00027     private static final int WINDOW_WIDTH = 1440;
00028     private static final int WINDOW_HEIGHT = 720;
00029     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00030     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00031     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00032     private static final Color TEXT_COLOR = Color.WHITE;
00033     private static final Color HIGHLIGHT_COLOR = new Color(255, 215, 0);
00034     private static final Color CORRECT_ANSWER_COLOR = new Color(50, 205, 50);
00035     private static final Color WRONG_ANSWER_COLOR = new Color(220, 20, 20);
00037     private final GameController controller;
00038     private final TimerController timerController;
00039     private JLabel questionLabel;
00040     private JButton[] answerButtons;
00041     private final List<JPanel> prizeLadderRows;
00042     private final LifelineView lifelineView;
00043     private JLabel devHintLabel;
00052     public GameView(GameController controller, LifelineController lifelineController) {
00053         this.controller = controller;
00054         this.timerController = new TimerController();
00055         this.prizeLadderRows = new ArrayList<>();
00056         this.lifelineView = new LifelineView(this, lifelineController);
00057
00058         setupWindow();
00059         setupComponents();
00060         timerController.startTimer();
00061     }
00062
00063     private void setupWindow() {
00076         setTitle("Legyen Ön is Milliomos!");
00077         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00078         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
00079         setResizable(false);
00080         setLocationRelativeTo(null);
00081
00082         setLayout(new BorderLayout());
00083         getContentPane().setBackground(BACKGROUND_COLOR);
00084     }
00085
00086     private void setupComponents() {
00097         JPanel gamePanel = new JPanel(new BorderLayout(20, 20));
00098         gamePanel.setBackground(BACKGROUND_COLOR);
00099         gamePanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00100
00101         JPanel questionPanel = createQuestionPanel();
00102         gamePanel.add(questionPanel, BorderLayout.NORTH);
00103
00104         JPanel answersPanel = createAnswersPanel();
00105         gamePanel.add(answersPanel, BorderLayout.CENTER);
00106
00107         JPanel southPanel = new JPanel(new BorderLayout());
00108         southPanel.setBackground(BACKGROUND_COLOR);
00109
00110         southPanel.add(lifelineView.createLifelinePanel(), BorderLayout.CENTER);
00111
00112         if (Main.isDeveloperMode()) {
00113             devHintLabel = new JLabel();
00114             devHintLabel.setForeground(TEXT_COLOR);
00115             devHintLabel.setHorizontalAlignment(SwingConstants.CENTER);
00116             devHintLabel.setFont(new Font("Arial", Font.PLAIN, 12));
00117             southPanel.add(devHintLabel, BorderLayout.SOUTH);
00118         }
00119
00120         gamePanel.add(southPanel, BorderLayout.SOUTH);
00121
00122         JPanel rightPanel = new JPanel(new BorderLayout());

```

```
00123         rightPanel.setBackground(BACKGROUND_COLOR);
00124
00125         rightPanel.add(timerController.getView(), BorderLayout.NORTH);
00126
00127         rightPanel.add(createPrizeLadderPanel(), BorderLayout.CENTER);
00128
00129         add(gamePanel, BorderLayout.CENTER);
00130         add(rightPanel, BorderLayout.EAST);
00131     }
00132
00133
00142     private JPanel createQuestionPanel() {
00143         JPanel panel = new JPanel();
00144         panel.setBackground(BACKGROUND_COLOR);
00145         panel.setBorder(BorderFactory.createCompoundBorder(
00146             BorderFactory.createLineBorder(BUTTON_COLOR, 2),
00147             BorderFactory.createEmptyBorder(10, 10, 10, 10)
00148         ));
00149         panel.setPreferredSize(new Dimension(800, 100));
00150
00151         questionLabel = new JLabel();
00152         questionLabel.setForeground(TEXT_COLOR);
00153         questionLabel.setFont(new Font("Arial", Font.BOLD, 18));
00154         panel.add(questionLabel);
00155
00156         return panel;
00157     }
00158
00159
00170     private JPanel createAnswersPanel() {
00171         JPanel panel = new JPanel(new GridLayout(2, 2, 20, 20));
00172         panel.setBackground(BACKGROUND_COLOR);
00173
00174         answerButtons = new JButton[4];
00175         String[] letters = {"A", "B", "C", "D"};
00176
00177         for (int i = 0; i < 4; i++) {
00178             final String letter = letters[i];
00179             answerButtons[i] = createAnswerButton(letter);
00180             answerButtons[i].addActionListener(e -> controller.handleAnswer(letter));
00181             panel.add(answerButtons[i]);
00182         }
00183
00184         return panel;
00185     }
00186
00187
00194     private JButton createAnswerButton(String letter) {
00195         JButton button = new JButton(letter);
00196         styleButton(button);
00197         return button;
00198     }
00199
00200
00211     private JPanel createPrizeLadderPanel() {
00212         JPanel panel = new JPanel();
00213         panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
00214         panel.setBackground(BACKGROUND_COLOR);
00215         panel.setBorder(BorderFactory.createEmptyBorder(20, 10, 20, 20));
00216
00217         List<PrizeLevel> prizeLevels = GameModel.getPrizeLevels();
00218
00219         for (PrizeLevel level : prizeLevels) {
00220             JPanel prizeRow = createPrizeRow(level);
00221             prizeLadderRows.add(prizeRow);
00222             panel.add(prizeRow);
00223             if (prizeLevels.indexOf(level) < prizeLevels.size() - 1) {
00224                 panel.add(Box.createRigidArea(new Dimension(0, 5)));
00225             }
00226         }
00227
00228         return panel;
00229     }
00230
00231
00238     private JPanel createPrizeRow(PrizeLevel level) {
00239         JPanel panel = new JPanel();
00240         panel.setLayout(new BoxLayout(panel, BoxLayout.X_AXIS));
00241         panel.setBackground(BACKGROUND_COLOR);
00242
00243         JLabel levelLabel = new JLabel(String.valueOf(level.getLevel()));
00244         JLabel prizeLabel = new JLabel(formatPrize(level.getPrize()));
00245
00246         levelLabel.setForeground(TEXT_COLOR);
00247         prizeLabel.setForeground(TEXT_COLOR);
00248
00249         levelLabel.setFont(new Font("Arial", Font.BOLD, 14));
```

```

00250         prizeLabel.setFont(new Font("Arial", Font.BOLD, 14));
00251
00252         panel.add(levelLabel);
00253         panel.add(Box.createHorizontalStrut(10));
00254         panel.add(prizeLabel);
00255
00256         return panel;
00257     }
00258
00259     private String formatPrize(long prize) {
00260         return String.format("%,d Ft", prize).replace(",", ".");
00261     }
00262
00263     private void styleButton(JButton button) {
00264         button.setFont(new Font("Arial", Font.BOLD, 16));
00265         button.setForeground(TEXT_COLOR);
00266         button.setBackground(BUTTON_COLOR);
00267         button.setBorder(BorderFactory.createCompoundBorder(
00268             BorderFactory.createLineBorder(BUTTON_COLOR, 2),
00269             BorderFactory.createEmptyBorder(10, 10, 10, 10)
00270         ));
00271         button.setFocusPainted(false);
00272
00273         button.addMouseListener(new java.awt.event.MouseAdapter() {
00274             @Override
00275             public void mouseEntered(java.awt.event.MouseEvent evt) {
00276                 if (button.isEnabled()) {
00277                     button.setBackground(BUTTON_HOVER_COLOR);
00278                 }
00279             }
00280             @Override
00281             public void mouseExited(java.awt.event.MouseEvent evt) {
00282                 if (button.isEnabled()) {
00283                     button.setBackground(BUTTON_COLOR);
00284                 }
00285             }
00286         });
00287     }
00288
00289     public void updateQuestion(Question question) {
00290         questionLabel.setText(question.getQuestion());
00291
00292         answerButtons[0].setText("A: " + question.getOptionA());
00293         answerButtons[1].setText("B: " + question.getOptionB());
00294         answerButtons[2].setText("C: " + question.getOptionC());
00295         answerButtons[3].setText("D: " + question.getOptionD());
00296
00297         if (Main.isDeveloperMode()) {
00298             devHintLabel.setText("Válasz: " + question.getCorrectAnswer());
00299         }
00300
00301         resetAnswerButtons();
00302     }
00303
00304     public void showAnswerResult(String selectedAnswer, String correctAnswer, boolean isCorrect) {
00305         setAnswerButtonsEnabled(false);
00306
00307         for (JButton button : answerButtons) {
00308             String buttonLetter = button.getText().substring(0, 1);
00309             if (buttonLetter.equals(selectedAnswer)) {
00310                 button.setBackground(isCorrect ? CORRECT_ANSWER_COLOR : WRONG_ANSWER_COLOR);
00311             } else if (buttonLetter.equals(correctAnswer) && !isCorrect) {
00312                 button.setBackground(CORRECT_ANSWER_COLOR);
00313             }
00314         }
00315     }
00316
00317     public void updatePrizeLadder(int currentLevel) {
00318         for (int i = 0; i < prizeLadderRows.size(); i++) {
00319             JPanel row = prizeLadderRows.get(i);
00320             int rowLevel = 15 - i;
00321
00322             if (rowLevel == currentLevel) {
00323                 row.setBackground(HIGHLIGHT_COLOR);
00324                 for (Component comp : row.getComponents()) {
00325                     if (comp instanceof JLabel) {
00326                         ((JLabel) comp).setForeground(Color.BLACK);
00327                     }
00328                 }
00329             } else {
00330                 row.setBackground(BACKGROUND_COLOR);
00331                 for (Component comp : row.getComponents()) {

```

```

00384             if (comp instanceof JLabel) {
00385                 ((JLabel) comp).setForeground(TEXT_COLOR);
00386             }
00387         }
00388     }
00389 }
00390
00391
00392
00400 private void resetAnswerButtons() {
00401     for (JButton button : answerButtons) {
00402         button.setBackground(BUTTON_COLOR);
00403         button.setEnabled(true);
00404     }
00405 }
00406
00407
00413 public void setAnswerButtonsEnabled(boolean enabled) {
00414     for (JButton button : answerButtons) {
00415         button.setEnabled(enabled);
00416     }
00417 }
00418
00419
00425 public JButton[] getAnswerButtons() {
00426     return answerButtons;
00427 }
00428
00429
00433 public void stopTimer() {
00434     timerController.stopTimer();
00435 }
00436
00437
00441 public void updateFinalPlaytime() {
00442     controller.getPlayer().setPlaytime(timerController.getElapsedTime());
00443 }
00444
00445
00456 public void showGameEndDialog(GameState state, String prize) {
00457     String message = state == GameState.WON ?
00458         "Gratulálok! Megnyerted a főnyereményt: " + prize :
00459         "Sajnos helytelen válasz! Nyereményed: " + prize;
00460
00461     String title = state == GameState.WON ? "Győzelem!" : "Játék vége";
00462
00463     JOptionPane.showMessageDialog(this, message, title, JOptionPane.INFORMATION_MESSAGE);
00464 }
00465
00466
00473 public void closeGame() {
00474     timerController.stopTimer();
00475     controller.getPlayer().updatePlaytime();
00476     dispose();
00477 }
00478 }

```

5.47 src/main/java/millionaire/view/LifelineView.java File Reference

Classes

- class **millionaire.view.LifelineView**
A játék segítségait megjelenítő és kezelő felület.

5.48 LifelineView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.LifelineController;
00005 import javax.swing.*;
00006 import java.awt.*;

```

```

00007 import java.util.Map;
00008
00018 public class LifelineView {
00019     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00020     private static final Color TEXT_COLOR = Color.WHITE;
00021     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00022     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00023     private static final Color DISABLED_COLOR = new Color(128, 128, 128);
00025     private final JButton fiftyFiftyButton;
00026     private final JButton audienceButton;
00027     private final LifelineController controller;
00028     private final JFrame parentFrame;
00036     public LifelineView(JFrame parentFrame, LifelineController controller) {
00037         this.parentFrame = parentFrame;
00038         this.controller = controller;
00039
00040         this.fiftyFiftyButton = createLifelineButton("Felezés", e -> controller.useFiftyFifty());
00041         this.audienceButton = createLifelineButton("Közönség szavazás", e ->
controller.useAudienceHelp());
00042     }
00043
00044
00054     private JButton createLifelineButton(String text, java.awt.event.ActionListener listener) {
00055         JButton button = new JButton(text);
00056         styleButton(button);
00057         button.addActionListener(listener);
00058         return button;
00059     }
00060
00061
00073     private void styleButton(JButton button) {
00074         button.setFont(new Font("Arial", Font.BOLD, 14));
00075         button.setForeground(Color.WHITE);
00076         button.setBackground(BUTTON_COLOR);
00077         button.setBorder(BorderFactory.createRaisedBevelBorder());
00078         button.setFocusPainted(false);
00079
00080         button.addMouseListener(new java.awt.event.MouseAdapter() {
00081             @Override
00082             public void mouseEntered(java.awt.event.MouseEvent evt) {
00083                 if (button.isEnabled()) {
00084                     button.setBackground(BUTTON_HOVER_COLOR);
00085                 }
00086             }
00087
00088             @Override
00089             public void mouseExited(java.awt.event.MouseEvent evt) {
00090                 if (button.isEnabled()) {
00091                     button.setBackground(BUTTON_COLOR);
00092                 }
00093             }
00094         });
00095     }
00096
00097
00106     public JPanel createLifelinePanel() {
00107         JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 20, 10));
00108         panel.setBackground(new Color(90, 90, 90));
00109         panel.add(fiftyFiftyButton);
00110         panel.add(audienceButton);
00111         return panel;
00112     }
00113
00114
00126     public void updateButtonStates(boolean fiftyFiftyAvailable, boolean audienceHelpAvailable) {
00127         fiftyFiftyButton.setEnabled(fiftyFiftyAvailable);
00128         audienceButton.setEnabled(audienceHelpAvailable);
00129
00130         fiftyFiftyButton.setBackground(fiftyFiftyAvailable ? BUTTON_COLOR : DISABLED_COLOR);
00131         audienceButton.setBackground(audienceHelpAvailable ? BUTTON_COLOR : DISABLED_COLOR);
00132     }
00133
00134
00145     public void updateAnswerButtons(JButton[] answerButtons, Map<String, Boolean> visibilityMap) {
00146         for (JButton button : answerButtons) {
00147             String option = button.getText().substring(0, 1);
00148             if (!visibilityMap.get(option)) {
00149                 button.setEnabled(false);
00150                 button.setText(option + ": ");
00151             }
00152         }
00153     }
00154
00155
00163     public void showAudienceResults(Map<String, Integer> percentages) {
00164         JDialog dialog = createAudienceDialog(percentages);

```

```

00165         dialog.setVisible(true);
00166     }
00167
00168
00181     private JDialog createAudienceDialog(Map<String, Integer> percentages) {
00182         JDialog dialog = new JDialog(parentFrame, "Közönség szavazás", true);
00183         dialog.setLayout(new BorderLayout());
00184
00185         JPanel barsPanel = new JPanel(new GridLayout(4, 1, 5, 5));
00186         barsPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00187
00188         for (Map.Entry<String, Integer> entry : percentages.entrySet()) {
00189             barsPanel.add(createPercentageBar(entry.getKey(), entry.getValue()));
00190         }
00191
00192         JButton closeButton = new JButton("Bezár");
00193         closeButton.addActionListener(e -> dialog.dispose());
00194
00195         dialog.add(barsPanel, BorderLayout.CENTER);
00196         dialog.add(closeButton, BorderLayout.SOUTH);
00197
00198         dialog.setSize(300, 200);
00199         dialog.setLocationRelativeTo(parentFrame);
00200
00201         return dialog;
00202     }
00203
00204
00217     private JPanel createPercentageBar(String letter, int percentage) {
00218         JPanel panel = new JPanel(new BorderLayout(10, 0));
00219
00220         JLabel label = new JLabel(String.format("%s: %d%%", letter, percentage));
00221         JProgressBar progressBar = new JProgressBar(0, 100);
00222         progressBar.setValue(percentage);
00223         progressBar.setStringPainted(false);
00224
00225         panel.add(label, BorderLayout.WEST);
00226         panel.add(progressBar, BorderLayout.CENTER);
00227
00228         return panel;
00229     }
00230 }

```

5.49 src/main/java/millionaire/view/MainMenuView.java File Reference

Classes

- class **millionaire.view.MainMenuView**

A játék főmenüjének grafikus felülete.

5.50 MainMenuView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.MainMenuController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00017 public class MainMenuView extends JFrame {
00018     private static final int WINDOW_WIDTH = 1280;
00019     private static final int WINDOW_HEIGHT = 720;
00020     private static final String GAME_TITLE = "Legyen Ön is Milliomos!";
00021     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00026     private final MainMenuController controller;
00036     public MainMenuView(MainMenuController controller) {
00037         this.controller = controller;
00038         setupWindow();
00039         setupComponents();

```

```

00040     }
00041
00042
00053     private void setupWindow() {
00054         setTitle(GAME_TITLE);
00055         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00056         setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
00057         setResizable(false);
00058         setLocationRelativeTo(null);
00059     }
00060
00061
00069     private void setupComponents() {
00070         JPanel mainPanel = createMainPanel();
00071         JLabel titleLabel = createTitleLabel();
00072         JPanel buttonPanel = createButtonPanel();
00073
00074         mainPanel.add(titleLabel);
00075         mainPanel.add(buttonPanel);
00076
00077         add(mainPanel);
00078     }
00079
00080
00089     private JPanel createMainPanel() {
00090         JPanel panel = new JPanel();
00091         panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00092         panel.setBorder(BorderFactory.createEmptyBorder(50, 25, 50, 25));
00093         panel.setBackground(BACKGROUND_COLOR);
00094         return panel;
00095     }
00096
00097
00106     private JLabel createTitleLabel() {
00107         JLabel label = new JLabel(GAME_TITLE);
00108         label.setFont(new Font("Arial", Font.BOLD, 36));
00109         label.setForeground(TEXT_COLOR);
00110         label.setAlignmentX(Component.CENTER_ALIGNMENT);
00111         return label;
00112     }
00113
00114
00127     private JPanel createButtonPanel() {
00128         JPanel panel = new JPanel();
00129         panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00130         panel.setBorder(BorderFactory.createEmptyBorder(50, 0, 0, 0));
00131         panel.setBackground(BACKGROUND_COLOR);
00132
00133         String[] buttonLabels = {"Új Játék", "Kérdés hozzáadása", "Dicsőséglista", "Kézikönyv",
00134 "Kilépés"};
00135
00136         for (String label : buttonLabels) {
00137             JButton button = createMenuButton(label);
00138             panel.add(button);
00139             panel.add(Box.createRigidArea(new Dimension(0, 20)));
00140         }
00141
00142         return panel;
00143     }
00144
00154     private JButton createMenuButton(String text) {
00155         JButton button = new JButton(text);
00156         styleButton(button);
00157         button.addActionListener(e -> handleButtonClick(text));
00158         return button;
00159     }
00160
00161
00175     private void styleButton(JButton button) {
00176         button.setPreferredSize(new Dimension(200, 50));
00177         button.setMaximumSize(new Dimension(200, 50));
00178         button.setFont(new Font("Arial", Font.PLAIN, 18));
00179         button.setFocusPainted(false);
00180         button.setBackground(BUTTON_COLOR);
00181         button.setForeground(TEXT_COLOR);
00182         button.setBorder(BorderFactory.createRaisedBevelBorder());
00183         button.setAlignmentX(Component.CENTER_ALIGNMENT);
00184
00185         button.addMouseListener(new java.awt.event.MouseAdapter() {
00186             @Override
00187             public void mouseEntered(java.awt.event.MouseEvent evt) {
00188                 button.setBackground(BUTTON_HOVER_COLOR);
00189             }
00190
00191             @Override
00192             public void mouseExited(java.awt.event.MouseEvent evt) {

```

```

00193         button.setBackground(BUTTON_COLOR);
00194     }
00195     });
00196 }
00197
00198
00212 private void handleClick(String buttonText) {
00213     switch (buttonText) {
00214         case "Új Játék":
00215             controller.startNewGame();
00216             break;
00217         case "Kérdés hozzáadása":
00218             controller.showAddQuestionDialog();
00219             break;
00220         case "Dicsőséglista":
00221             controller.showScoreboard();
00222             break;
00223         case "Kézikönyv":
00224             controller.showRules();
00225             break;
00226         case "Kilépés":
00227             controller.exitGame();
00228             break;
00229     }
00230 }
00231
00232
00238 public void hideWindow() {
00239     setVisible(false);
00240 }
00241
00242
00248 public void showWindow() {
00249     setVisible(true);
00250 }
00251 }

```

5.51 src/main/java/millionaire/view/PlayerView.java File Reference

Classes

- class **millionaire.view.PlayerView**
Játékos létrehozására szolgáló dialógusablak.

5.52 PlayerView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.PlayerController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00016 public class PlayerView extends JDialog {
00017     private static final int DIALOG_WIDTH = 300;
00018     private static final int DIALOG_HEIGHT = 150;
00020     private final PlayerController controller;
00021     private JTextField nameField;
00022     private JButton okButton;
00023     private JButton cancelButton;
00035     public PlayerView(JFrame parent, PlayerController controller) {
00036         super(parent, "Új Játék", true);
00037         this.controller = controller;
00038         setupUI();
00039     }
00040
00041
00053     private void setupUI() {
00054         setSize(DIALOG_WIDTH, DIALOG_HEIGHT);
00055         setLocationRelativeTo(getParent());
00056
00057         JPanel panel = new JPanel();

```

```

00058     panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00059     panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00060
00061     JLabel label = new JLabel("Kérem adja meg a nevét:");
00062     nameField = new JTextField(20);
00063     okButton = new JButton("OK");
00064     cancelButton = new JButton("Mégse");
00065
00066     label.setAlignmentX(Component.CENTER_ALIGNMENT);
00067     nameField.setMaximumSize(new Dimension(200, 25));
00068     nameField.setAlignmentX(Component.CENTER_ALIGNMENT);
00069
00070     JPanel buttonPanel = new JPanel();
00071     buttonPanel.add(okButton);
00072     buttonPanel.add(cancelButton);
00073
00074     okButton.addActionListener(e -> handleOkButton());
00075     cancelButton.addActionListener(e -> handleCancelButton());
00076
00077     panel.add(label);
00078     panel.add(Box.createRigidArea(new Dimension(0, 10)));
00079     panel.add(nameField);
00080     panel.add(Box.createRigidArea(new Dimension(0, 10)));
00081     panel.add(buttonPanel);
00082
00083     add(panel);
00084 }
00085
00086
00092 private void handleOkButton() {
00093     String name = nameField.getText().trim();
00094     controller.createPlayer(name);
00095 }
00096
00097
01003 private void handleCancelButton() {
01004     controller.cancelPlayerCreation();
01005 }
01006
01007
01115 public void showError(String message) {
01116     JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
01117 }
01118
01119
01227 public String getPlayerName() {
01228     return nameField.getText().trim();
01229 }
01230
01231
01337 public void closeDialog() {
01338     dispose();
01339 }
01340 }

```

5.53 src/main/java/millionaire/view/RulesView.java File Reference

Classes

- class **millionaire.view.RulesView**
A játékszabályok megjelenítésére szolgáló ablak.

5.54 RulesView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.RulesController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00017 public class RulesView extends JFrame {

```

```

00018     private static final int WINDOW_WIDTH = 800;
00019     private static final int WINDOW_HEIGHT = 600;
00020     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00021     private static final Color TEXT_COLOR = Color.WHITE;
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00025     private final RulesController controller;
00026     private JTextArea rulesArea;
00033     public RulesView(RulesController controller) {
00034         this.controller = controller;
00035         setupWindow();
00036         setupComponents();
00037     }
00038
00039
00050     private void setupWindow() {
00051         setTitle("Játékszabályok");
00052         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00053         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00054         setLocationRelativeTo(null);
00055         setResizable(true);
00056     }
00057
00058
00067     private void setupComponents() {
00068         JPanel mainPanel = createMainPanel();
00069         JScrollPane scrollPane = createRulesTextArea();
00070         JButton closeButton = createCloseButton();
00071
00072         mainPanel.add(scrollPane, BorderLayout.CENTER);
00073         mainPanel.add(closeButton, BorderLayout.SOUTH);
00074
00075         add(mainPanel);
00076     }
00077
00078
00084     private JPanel createMainPanel() {
00085         JPanel panel = new JPanel(new BorderLayout(10, 10));
00086         panel.setBackground(BACKGROUND_COLOR);
00087         panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00088         return panel;
00089     }
00090
00091
00097     private JScrollPane createRulesTextArea() {
00098         rulesArea = new JTextArea();
00099         rulesArea.setEditable(false);
00100         rulesArea.setBackground(BACKGROUND_COLOR);
00101         rulesArea.setForeground(TEXT_COLOR);
00102         rulesArea.setFont(new Font("Arial", Font.PLAIN, 14));
00103         rulesArea.setLineWrap(true);
00104         rulesArea.setWrapStyleWord(true);
00105
00106         JScrollPane scrollPane = new JScrollPane(rulesArea);
00107         scrollPane.setBorder(BorderFactory.createEmptyBorder());
00108         return scrollPane;
00109     }
00110
00111
00117     private JButton createCloseButton() {
00118         JButton button = new JButton("Bezárás");
00119         styleButton(button);
00120         button.addActionListener(e -> controller.handleCloseRequest());
00121         return button;
00122     }
00123
00124
00130     private void styleButton(JButton button) {
00131         button.setFont(new Font("Arial", Font.PLAIN, 14));
00132         button.setFocusPainted(false);
00133         button.setBackground(BUTTON_COLOR);
00134         button.setForeground(TEXT_COLOR);
00135         button.setBorder(BorderFactory.createRaisedBevelBorder());
00136
00137         button.addMouseListener(new java.awt.event.MouseAdapter() {
00138             @Override
00139             public void mouseEntered(java.awt.event.MouseEvent evt) {
00140                 button.setBackground(BUTTON_HOVER_COLOR);
00141             }
00142
00143             @Override
00144             public void mouseExited(java.awt.event.MouseEvent evt) {
00145                 button.setBackground(BUTTON_COLOR);
00146             }
00147         });
00148     }
00149

```

```

00150
00156     public void displayRules(String content) {
00157         rulesArea.setText(content);
00158         rulesArea.setCaretPosition(0);
00159     }
00160
00161
00167     public void showError(String message) {
00168         JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
00169     }
00170
00171
00175     public void closeWindow() {
00176         dispose();
00177     }
00178 }

```

5.55 src/main/java/millionaire/view/ScoreboardView.java File Reference

Classes

- class **millionaire.view.ScoreboardView**

A dicsőséglista megjelenítésére szolgáló ablak.

5.56 ScoreboardView.java

Go to the documentation of this file.

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.ScoreboardController;
00005 import millionaire.model.PlayerModel;
00006 import javax.swing.*;
00007 import javax.swing.table.DefaultTableModel;
00008 import java.awt.*;
00009 import java.util.List;
00010
00020 public class ScoreboardView extends JFrame {
00021     private static final int WINDOW_WIDTH = 800;
00022     private static final int WINDOW_HEIGHT = 600;
00023     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00025     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00026     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00028     private final transient ScoreboardController controller;
00029     private final JTable scoreTable;
00030     private final DefaultTableModel tableModel;
00040     public ScoreboardView(ScoreboardController controller) {
00041         this.controller = controller;
00042         this.tableModel = createTableModel();
00043         this.scoreTable = createTable();
00044
00045         setupWindow();
00046         setupComponents();
00047     }
00048
00049
00053     private void setupWindow() {
00054         setTitle("Dicsőséglista");
00055         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00056         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00057         setLocationRelativeTo(null);
00058         setResizable(true);
00059     }
00060
00061
00074     private DefaultTableModel createTableModel() {
00075         return new DefaultTableModel(
00076             new String[]{"Helyezés", "Név", "Megválaszolt kérdések", "Nyeremény", "Játékidő"},
00077             0
00078         ) {
00079             @Override
00080             public boolean isCellEditable(int row, int column) {

```

```

00081         return false;
00082     }
00083 };
00084 }
00085
00086
00091 private JTable createTable() {
00092     JTable table = new JTable(tableModel);
00093     styleTable(table);
00094     return table;
00095 }
00096
00097
00110 private void styleTable(JTable table) {
00111     table.setBackground(BACKGROUND_COLOR);
00112     table.setForeground(TEXT_COLOR);
00113     table.setFont(new Font("Arial", Font.PLAIN, 14));
00114     table.getTableHeader().setFont(new Font("Arial", Font.BOLD, 14));
00115     table.setRowHeight(25);
00116     table.setGridColor(new Color(70, 130, 180));
00117
00118     // Nincs átrendezés és átméretezés..
00119     table.getTableHeader().setReorderingAllowed(false);
00120     table.getTableHeader().setResizingAllowed(false);
00121 }
00122
00123
00127 private void setupComponents() {
00128     JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
00129     mainPanel.setBackground(BACKGROUND_COLOR);
00130     mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00131
00132     JScrollPane scrollPane = new JScrollPane(scoreTable);
00133     scrollPane.getViewport().setBackground(BACKGROUND_COLOR);
00134
00135     JButton closeButton = createCloseButton();
00136
00137     mainPanel.add(scrollPane, BorderLayout.CENTER);
00138     mainPanel.add(closeButton, BorderLayout.SOUTH);
00139
00140     add(mainPanel);
00141 }
00142
00143
00149 private JButton createCloseButton() {
00150     JButton button = new JButton("Bezárás");
00151     styleButton(button);
00152     button.addActionListener(e -> controller.handleClose());
00153     return button;
00154 }
00155
00156
00162 private void styleButton(JButton button) {
00163     button.setFont(new Font("Arial", Font.PLAIN, 14));
00164     button.setFocusPainted(false);
00165     button.setBackground(BUTTON_COLOR);
00166     button.setForeground(TEXT_COLOR);
00167     button.setBorder(BorderFactory.createRaisedBevelBorder());
00168
00169     button.addMouseListener(new java.awt.event.MouseAdapter() {
00170         @Override
00171         public void mouseEntered(java.awt.event.MouseEvent evt) {
00172             button.setBackground(BUTTON_HOVER_COLOR);
00173         }
00174
00175         @Override
00176         public void mouseExited(java.awt.event.MouseEvent evt) {
00177             button.setBackground(BUTTON_COLOR);
00178         }
00179     });
00180 }
00181
00182
00190 public void updateScores(List<PlayerModel> players) {
00191     tableModel.setRowCount(0);
00192     int rank = 1;
00193     for (PlayerModel player : players) {
00194         tableModel.addRow(new Object[]{
00195             rank++,
00196             player.getName(),
00197             player.getCurrentLevel(),
00198             player.getFormattedPrize(),
00199             player.getFormattedPlaytime()
00200         });
00201     }
00202 }
00203

```

```
00204
00208     public void closeWindow() {
00209         dispose();
00210     }
00211 }
```

5.57 src/main/java/millionaire/view/TimerView.java File Reference

Classes

- class **millionaire.view.TimerView**

Az időmérő megjelenítése.

5.58 TimerView.java

Go to the documentation of this file.

```
00001
00002 package millionaire.view;
00003
00004 import javax.swing.*;
00005 import java.awt.*;
00006 import java.time.Duration;
00007
00015 public class TimerView extends JPanel {
00016     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00017     private static final Color TEXT_COLOR = Color.WHITE;
00018
00019     private final JLabel timeLabel;
00024     public TimerView() {
00025         setBackground(BACKGROUND_COLOR);
00026         setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00027         setPreferredSize(new Dimension(200, 50));
00028
00029         timeLabel = new JLabel("Játékidő: 00:00");
00030         timeLabel.setForeground(TEXT_COLOR);
00031         timeLabel.setFont(new Font("Arial", Font.BOLD, 16));
00032         timeLabel.setHorizontalAlignment(SwingConstants.CENTER);
00033
00034         add(timeLabel);
00035     }
00036
00041     public void updateDisplay(Duration duration) {
00042         long seconds = duration.getSeconds();
00043         long minutes = seconds / 60;
00044         seconds = seconds % 60;
00045
00046         timeLabel.setText(String.format("Játékidő: %02d:%02d", minutes, seconds));
00047     }
00048 }
```

Index

- addQuestion
 - millionaire.model.AddQuestionModel, 11
 - millionaire.model.QuestionRepository, 68
- AddQuestionController
 - millionaire.controller.AddQuestionController, 8
- AddQuestionModel
 - millionaire.model.AddQuestionModel, 11
- AddQuestionView
 - millionaire.view.AddQuestionView, 16
- addScore
 - millionaire.controller.ScoreboardController, 77
 - millionaire.model.ScoreboardModel, 79
- cancelPlayerCreation
 - millionaire.controller.PlayerController, 49
- CATEGORIES
 - millionaire.model.QuestionRepository, 69
- clearFields
 - millionaire.view.AddQuestionView, 16
- closeDialog
 - millionaire.view.PlayerView, 60
- closeGame
 - millionaire.view.GameView, 30
- closeWindow
 - millionaire.view.AddQuestionView, 16
 - millionaire.view.RulesView, 75
 - millionaire.view.ScoreboardView, 83
- compareTo
 - millionaire.model.PlayerModel, 54
- CORRECT
 - millionaire.model.GameState, 26
- createLifelinePanel
 - millionaire.view.LifelineView, 39
- createPlayer
 - millionaire.controller.PlayerController, 49
- DIFFICULTIES
 - millionaire.model.QuestionRepository, 69
- displayRules
 - millionaire.view.RulesView, 75
- equals
 - millionaire.model.PlayerModel, 54
- exitGame
 - millionaire.controller.MainMenuController, 43
- GameController
 - millionaire.controller.GameController, 18
- GameModel
 - millionaire.model.GameModel, 22
- GameView
 - millionaire.view.GameView, 29
- getAllQuestions
 - millionaire.model.QuestionRepository, 68
- getAnswerButtons
 - millionaire.view.GameView, 30
- getAudienceResult
 - millionaire.model.LifelineModel, 36
- getCategories
 - millionaire.controller.AddQuestionController, 8
 - millionaire.model.AddQuestionModel, 12
- getCategory
 - millionaire.model.Question, 64
- getCorrectAnswer
 - millionaire.model.Question, 64
- getCurrentLevel
 - millionaire.controller.GameController, 19
 - millionaire.model.PlayerModel, 55
- getCurrentPrize
 - millionaire.model.GameModel, 22
- getCurrentQuestion
 - millionaire.controller.GameController, 19
 - millionaire.model.GameModel, 22
- getDifficulties
 - millionaire.controller.AddQuestionController, 9
 - millionaire.model.AddQuestionModel, 12
- getDifficulty
 - millionaire.model.Question, 65
- getElapsedTime
 - millionaire.controller.TimerController, 85
 - millionaire.model.TimerModel, 87
- getFiftyFiftyResult
 - millionaire.model.LifelineModel, 36
- getFormattedPlaytime
 - millionaire.model.PlayerModel, 55
- getFormattedPrize
 - millionaire.model.PlayerModel, 55
- getGameView
 - millionaire.controller.GameController, 19
- getLastSafePrize
 - millionaire.model.GameModel, 23
- getLevel
 - millionaire.model.PrizeLevel, 62
- getLifelinePanel
 - millionaire.controller.LifelineController, 34
- getName
 - millionaire.model.PlayerModel, 55
- getOptionA
 - millionaire.model.Question, 65

- getOptionB
 - millionaire.model.Question, 65
- getOptionC
 - millionaire.model.Question, 65
- getOptionD
 - millionaire.model.Question, 66
- getPlayer
 - millionaire.controller.GameController, 19
 - millionaire.model.GameModel, 23
- getPlayerName
 - millionaire.view.PlayerView, 60
- getPlayers
 - millionaire.model.ScoreboardModel, 80
- getPlaytime
 - millionaire.model.PlayerModel, 55
- getPrize
 - millionaire.model.PlayerModel, 56
 - millionaire.model.PrizeLevel, 62
- getPrizeLevels
 - millionaire.model.GameModel, 23
- getQuestion
 - millionaire.model.Question, 66
- getQuestionRepository
 - millionaire.model.MainMenuModel, 46
- getQuestionsByCategory
 - millionaire.model.QuestionRepository, 69
- getQuestionsByDifficulty
 - millionaire.model.QuestionRepository, 69
- getRulesContent
 - millionaire.model.RulesModel, 73
- getStartTime
 - millionaire.model.PlayerModel, 56
 - millionaire.model.TimerModel, 87
- getView
 - millionaire.controller.TimerController, 85
- handleAnswer
 - millionaire.controller.GameController, 19
- handleCancel
 - millionaire.controller.AddQuestionController, 9
- handleClose
 - millionaire.controller.ScoreboardController, 78
- handleCloseRequest
 - millionaire.controller.RulesController, 71
- handleSubmit
 - millionaire.controller.AddQuestionController, 9
- hideWindow
 - millionaire.view.MainMenuView, 48
- incrementLevel
 - millionaire.model.PlayerModel, 56
- isAudienceHelpAvailable
 - millionaire.model.LifelineModel, 37
- isContentLoaded
 - millionaire.model.RulesModel, 73
- isDeveloperMode
 - millionaire.Main, 41
- isFiftyFiftyAvailable
 - millionaire.model.LifelineModel, 37
- isGameEnding
 - millionaire.model.GameState, 26
- isGameOver
 - millionaire.model.GameModel, 23
- isSafeSpot
 - millionaire.model.PrizeLevel, 63
- isTimeUp
 - millionaire.model.GameModel, 23
- LifelineController
 - millionaire.controller.LifelineController, 34
- LifelineView
 - millionaire.view.LifelineView, 39
- LOST
 - millionaire.model.GameState, 26
- main
 - millionaire.Main, 41
- MainMenuController
 - millionaire.controller.MainMenuController, 43
- MainMenuModel
 - millionaire.model.MainMenuModel, 45
- MainMenuView
 - millionaire.view.MainMenuView, 47
- millionaire.controller.AddQuestionController, 7
 - AddQuestionController, 8
 - getCategories, 8
 - getDifficulties, 9
 - handleCancel, 9
 - handleSubmit, 9
 - showDialog, 10
- millionaire.controller.GameController, 17
 - GameController, 18
 - getCurrentLevel, 19
 - getCurrentQuestion, 19
 - getGameView, 19
 - getPlayer, 19
 - handleAnswer, 19
 - setAnswerButtonsEnabled, 20
 - updateGameTime, 20
- millionaire.controller.LifelineController, 32
 - getLifelinePanel, 34
 - LifelineController, 34
 - setGameContext, 34
 - useAudienceHelp, 35
 - useFiftyFifty, 35
- millionaire.controller.MainMenuController, 42
 - exitGame, 43
 - MainMenuController, 43
 - showAddQuestionDialog, 43
 - showMenu, 44
 - showRules, 44
 - showScoreboard, 44
 - startNewGame, 44
- millionaire.controller.PlayerController, 48
 - cancelPlayerCreation, 49
 - createPlayer, 49
 - PlayerController, 49
 - showPlayerCreationDialog, 50

- millionaire.controller.RulesController, 70
 - handleCloseRequest, 71
 - RulesController, 71
 - showRules, 71
- millionaire.controller.ScoreboardController, 76
 - addScore, 77
 - handleClose, 78
 - ScoreboardController, 77
 - showScoreboard, 78
- millionaire.controller.TimerController, 83
 - getElapsedTime, 85
 - getView, 85
 - startTimer, 85
 - stopTimer, 85
 - TimerController, 84
- millionaire.Main, 40
 - isDeveloperMode, 41
 - main, 41
- millionaire.model.AddQuestionModel, 10
 - addQuestion, 11
 - AddQuestionModel, 11
 - getCategories, 12
 - getDifficulties, 12
 - validateInputs, 12
- millionaire.model.GameModel, 20
 - GameModel, 22
 - getCurrentPrize, 22
 - getCurrentQuestion, 22
 - getLastSafePrize, 23
 - getPlayer, 23
 - getPrizeLevels, 23
 - isGameOver, 23
 - isTimeUp, 23
 - processAnswer, 24
 - selectNextQuestion, 24
 - updatePlayTime, 24
- millionaire.model.GameState, 25
 - CORRECT, 26
 - isGameEnding, 26
 - LOST, 26
 - WON, 26
- millionaire.model.LifelineModel, 35
 - getAudienceResult, 36
 - getFiftyFiftyResult, 36
 - isAudienceHelpAvailable, 37
 - isFiftyFiftyAvailable, 37
- millionaire.model.MainMenuModel, 45
 - getRepository, 46
 - MainMenuModel, 45
- millionaire.model.PlayerModel, 51
 - compareTo, 54
 - equals, 54
 - getCurrentLevel, 55
 - getFormattedPlaytime, 55
 - getFormattedPrize, 55
 - getName, 55
 - getPlaytime, 55
 - getPrize, 56
 - getStartTime, 56
 - incrementLevel, 56
 - PlayerModel, 53
 - setCurrentLevel, 56
 - setName, 57
 - setPlaytime, 57
 - setPrize, 57
 - updatePlaytime, 57
- millionaire.model.PrizeLevel, 61
 - getLevel, 62
 - getPrize, 62
 - isSafeSpot, 63
 - PrizeLevel, 62
- millionaire.model.Question, 63
 - getCategory, 64
 - getCorrectAnswer, 64
 - getDifficulty, 65
 - getOptionA, 65
 - getOptionB, 65
 - getOptionC, 65
 - getOptionD, 66
 - getQuestion, 66
- millionaire.model.QuestionRepository, 67
 - addQuestion, 68
 - CATEGORIES, 69
 - DIFFICULTIES, 69
 - getAllQuestions, 68
 - getQuestionsByCategory, 69
 - getQuestionsByDifficulty, 69
 - QuestionRepository, 68
- millionaire.model.RulesModel, 72
 - getRulesContent, 73
 - isContentLoaded, 73
 - RulesModel, 72
- millionaire.model.ScoreboardModel, 78
 - addScore, 79
 - getPlayers, 80
 - ScoreboardModel, 79
- millionaire.model.TimerModel, 86
 - getElapsedTime, 87
 - getStartTime, 87
 - start, 87
 - stop, 87
 - TimerModel, 87
 - updateElapsedTime, 87
- millionaire.view.AddQuestionView, 13
 - AddQuestionView, 16
 - clearFields, 16
 - closeWindow, 16
 - showError, 16
 - showSuccess, 17
- millionaire.view.GameView, 27
 - closeGame, 30
 - GameView, 29
 - getAnswerButtons, 30
 - setAnswerButtonsEnabled, 30
 - showAnswerResult, 30
 - showGameEndDialog, 31

- stopTimer, 31
- updateFinalPlaytime, 31
- updatePrizeLadder, 31
- updateQuestion, 32
- millionaire.view.LifelineView, 38
 - createLifelinePanel, 39
 - LifelineView, 39
 - showAudienceResults, 39
 - updateAnswerButtons, 39
 - updateButtonStates, 40
- millionaire.view.MainMenuView, 46
 - hideWindow, 48
 - MainMenuView, 47
 - showWindow, 48
- millionaire.view.PlayerView, 58
 - closeDialog, 60
 - getPlayerName, 60
 - PlayerView, 60
 - showError, 60
- millionaire.view.RulesView, 73
 - closeWindow, 75
 - displayRules, 75
 - RulesView, 75
 - showError, 76
- millionaire.view.ScoreboardView, 80
 - closeWindow, 83
 - ScoreboardView, 82
 - updateScores, 83
- millionaire.view.TimerView, 88
 - TimerView, 89
 - updateDisplay, 90
- PlayerController
 - millionaire.controller.PlayerController, 49
- PlayerModel
 - millionaire.model.PlayerModel, 53
- PlayerView
 - millionaire.view.PlayerView, 60
- PrizeLevel
 - millionaire.model.PrizeLevel, 62
- processAnswer
 - millionaire.model.GameModel, 24
- QuestionRepository
 - millionaire.model.QuestionRepository, 68
- RulesController
 - millionaire.controller.RulesController, 71
- RulesModel
 - millionaire.model.RulesModel, 72
- RulesView
 - millionaire.view.RulesView, 75
- ScoreboardController
 - millionaire.controller.ScoreboardController, 77
- ScoreboardModel
 - millionaire.model.ScoreboardModel, 79
- ScoreboardView
 - millionaire.view.ScoreboardView, 82
- selectNextQuestion
 - millionaire.model.GameModel, 24
- setAnswerButtonsEnabled
 - millionaire.controller.GameController, 20
 - millionaire.view.GameView, 30
- setCurrentLevel
 - millionaire.model.PlayerModel, 56
- setGameContext
 - millionaire.controller.LifelineController, 34
- setName
 - millionaire.model.PlayerModel, 57
- setPlaytime
 - millionaire.model.PlayerModel, 57
- setPrize
 - millionaire.model.PlayerModel, 57
- showAddQuestionDialog
 - millionaire.controller.MainMenuController, 43
- showAnswerResult
 - millionaire.view.GameView, 30
- showAudienceResults
 - millionaire.view.LifelineView, 39
- showDialog
 - millionaire.controller.AddQuestionController, 10
- showError
 - millionaire.view.AddQuestionView, 16
 - millionaire.view.PlayerView, 60
 - millionaire.view.RulesView, 76
- showGameEndDialog
 - millionaire.view.GameView, 31
- showMenu
 - millionaire.controller.MainMenuController, 44
- showPlayerCreationDialog
 - millionaire.controller.PlayerController, 50
- showRules
 - millionaire.controller.MainMenuController, 44
 - millionaire.controller.RulesController, 71
- showScoreboard
 - millionaire.controller.MainMenuController, 44
 - millionaire.controller.ScoreboardController, 78
- showSuccess
 - millionaire.view.AddQuestionView, 17
- showWindow
 - millionaire.view.MainMenuView, 48
- src/main/java/millionaire/controller/AddQuestionController.java, 91
- src/main/java/millionaire/controller/GameController.java, 92
- src/main/java/millionaire/controller/LifelineController.java, 94
- src/main/java/millionaire/controller/MainMenuController.java, 95
- src/main/java/millionaire/controller/PlayerController.java, 96
- src/main/java/millionaire/controller/RulesController.java, 97
- src/main/java/millionaire/controller/ScoreboardController.java, 97, 98

- src/main/java/millionaire/controller/TimerController.java, 98
- src/main/java/millionaire/Main.java, 99
- src/main/java/millionaire/model/AddQuestionModel.java, 99, 100
- src/main/java/millionaire/model/GameModel.java, 100
- src/main/java/millionaire/model/GameState.java, 102
- src/main/java/millionaire/model/LifelineModel.java, 102, 103
- src/main/java/millionaire/model/MainMenuModel.java, 104
- src/main/java/millionaire/model/PlayerModel.java, 104
- src/main/java/millionaire/model/PrizeLevel.java, 106
- src/main/java/millionaire/model/Question.java, 106
- src/main/java/millionaire/model/QuestionRepository.java, 108
- src/main/java/millionaire/model/RulesModel.java, 110
- src/main/java/millionaire/model/ScoreboardModel.java, 110
- src/main/java/millionaire/model/TimerModel.java, 112
- src/main/java/millionaire/view/AddQuestionView.java, 113
- src/main/java/millionaire/view/MapView.java, 115, 116
- src/main/java/millionaire/view/LifelineView.java, 119
- src/main/java/millionaire/view/MainMenuView.java, 121
- src/main/java/millionaire/view/PlayerView.java, 123
- src/main/java/millionaire/view/RulesView.java, 124
- src/main/java/millionaire/view/ScoreboardView.java, 126
- src/main/java/millionaire/view/TimerView.java, 128
- start
 - millionaire.model.TimerModel, 87
- startNewGame
 - millionaire.controller.MainMenuController, 44
- startTimer
 - millionaire.controller.TimerController, 85
- stop
 - millionaire.model.TimerModel, 87
- stopTimer
 - millionaire.controller.TimerController, 85
 - millionaire.view.GameView, 31
- TimerController
 - millionaire.controller.TimerController, 84
- TimerModel
 - millionaire.model.TimerModel, 87
- TimerView
 - millionaire.view.TimerView, 89
- updateAnswerButtons
 - millionaire.view.LifelineView, 39
- updateButtonStates
 - millionaire.view.LifelineView, 40
- updateDisplay
 - millionaire.view.TimerView, 90
- updateElapsedTime
 - millionaire.model.TimerModel, 87
- updateFinalPlaytime
 - millionaire.view.GameView, 31
- updateGameTime
 - millionaire.controller.GameController, 20
- updatePlayTime
 - millionaire.model.GameModel, 24
- updatePlaytime
 - millionaire.model.PlayerModel, 57
- updatePrizeLadder
 - millionaire.view.GameView, 31
- updateQuestion
 - millionaire.view.GameView, 32
- updateScores
 - millionaire.view.ScoreboardView, 83
- useAudienceHelp
 - millionaire.controller.LifelineController, 35
- useFiftyFifty
 - millionaire.controller.LifelineController, 35
- validateInputs
 - millionaire.model.AddQuestionModel, 12
- WON
 - millionaire.model.GameState, 26